Agent 开发面试题

全量参考答案（易懂版） · 直白表达 · 保留必要术语

本答案册覆盖原文公司面试部分的 642 道编号问题。答案已改为更直接、容易理解的表述；带【】的内容必须替换为你的真实经历和数据。框架与产品会持续变化，回答时要说明适用条件，不要把结论说成永远成立。

目录

使用说明 4

重点准备 5

1. 单 Agent 的完整执行链 5
2. Tool、Skill 与 MCP 5
3. 上下文与 Memory 5
4. RAG 全链路与故障定位 6
5. 可靠性、权限与沙箱 6
6. 评测、回归与指标口径 7
7. 后端基础与编码 7

一、字节跳动 8

一面 8

二面 10

三面 12

二、千问 15

一面 15

二面 19

三面 22

三、淘天 26

一面 26

二面 30

四、小红书 34

方向1一面 34

方向1二面 36

方向1三面 39

方向2一面 42

方向2二面 44

五、美团 48

一面 48

六、拼多多 53

一面 53

七、腾讯 58

一面 58

八、蚂蚁 61

一面 61

二面 66

九、百度 71

一面 71

二面 74

三面 77

十、深信服 82

一面 82

十一、同花顺 84

一面 84

编码题参考实现 89

附录 A：Two Sum（Python） 89

附录 B：原地合并有序数组（Python） 89

附录 C：简化 ReAct Loop（伪代码） 89

附录 D：Plan-and-Execute 核心结构（Python） 89

附录 E：重排数组使相邻不同（Python） 89

附录 F：二叉树层序遍历（Python） 90

附录 G：LRU Cache（Python） 90

附录 H：线程池执行流程（伪代码） 90

附录 I：Python 协程实现 91

附录 J：Java 单例 91

答案边界 91

使用说明

1. 个人经历题：替换占位符，并准备可核验的时间线、职责边界和指标口径。
2. 技术设计题：先讲约束，再讲主流程、异常路径和验证指标；答案不是唯一方案。
3. 同类问题在不同公司反复出现，保留重复题是为了让每道原题后都能直接看到答案。
4. 编码题的完整实现集中在文末附录。

重点准备

1. 单 Agent 的完整执行链

- 先能在 2-3 分钟内讲清一条请求：入口鉴权与会话创建 → 加载状态/上下文 → 模型决策 → 工具执行 → Observation → 完成校验 → 持久化结果与事件。任何一步都要能说明超时、取消、预算耗尽和恢复位置。
- ReAct：模型每轮决定继续调用工具还是结束，适合路径未知的探索任务；Plan-and-Execute：先生成有依赖的步骤，再按计划执行与重规划，适合可分解长任务；Workflow：节点和连线确定，适合审计严格、失败代价高的流程。
- Sub-agent
  只在职责可分、上下文需要隔离、可并行，或需要不同模型/权限时引入。任务只是"复杂"并不足以使用多 Agent；先把单 Agent 的正常与异常路径做扎实。
- 练习产物：一张调用图、一张故障时序图，以及可运行的简化 ReAct 与 Plan-and-Execute 代码。完整实现见附录 C、D。

模式	最小执行图	重点异常
ReAct	Context → Model → Tool → Observation → Final	无进展循环、工具失败、预算耗尽
Plan-and-Execute	生成计划 → 按依赖执行步骤 → 必要时重规划 → 汇总	计划过时、依赖失败、结果引用错误
Workflow	校验 DSL → 编译执行图 → 按固定节点执行 → 输出结果	非法连线、节点超时、补偿与回滚

2. Tool、Skill 与 MCP

- Tool 是原子动作及其输入输出契约；Skill
  是何时使用、怎样组合能力的可复用方法，可包含说明、Workflow、脚本和资源；MCP 是 Host 与工具服务之间的发现、Schema 和调用协议。三者是不同层次，不是三个同义词。
- 一次能力调用应能讲完整：权限/场景过滤 → 候选能力发现 → 渐进式加载描述 → 模型选择 → JSON Schema 校验 → 执行层再鉴权 → 超时/重试/幂等执行 → 结构化 Observation → 审计与错误返回。
- 常见追问：工具没展示给模型是否等于无权限（不是）；Skill 能否替代 MCP（不能完全替代协议与跨 Host 复用）；MCP 是否天然解决治理（不是，认证、审计、限流仍由平台承担）。

概念	主要职责	典型内容
Tool	执行一个原子动作	Schema、权限、超时、幂等、适配器
Skill	指导并组合能力	说明、规则、Workflow、脚本、产物要求
MCP	统一发现与调用协议	Tools/Resources/Prompts、传输与能力协商

3. 上下文与 Memory

- 上下文解决“当前这次推理给模型看什么”，Memory 解决“跨步骤或跨会话长期保留什么”。两者都要有写入、读取和删除策略，但生命周期、授权和错误影响不同。
- Token 预算可以按系统/策略、当前任务、近期对话、检索证据、工具结果和输出预留分区。达到阈值后先去重和裁剪低价值日志，再做结构化摘要；目标、约束、未完成项、关键证据、来源和副作用记录不能丢。
- 长期记忆写入前检查价值、可信程度、敏感性和用户授权，并记录来源、时间与版本。新旧信息冲突时，按来源可靠性、时效和用户确认处理。旧记录应标记为已被替代，而不是直接覆盖；错误记忆必须支持更正、删除和审计。
- 压缩后必须用同一批任务做回归：完成率是否下降、引用是否仍能定位、需要的原文能否从外部存储按 ID 回取。

对象	保留重点	常见治理
当前上下文	目标、约束、近期事件、必要证据	预算、摘要、回取、隔离
长期 Memory	稳定事实、偏好、来源、版本	授权、冲突、纠错、遗忘

4. RAG 全链路与故障定位

- 完整链路：权限校验 → 文档解析/OCR → 结构化与切分 → 去重和元数据 → 全文/向量索引 → 查询理解 → 权限过滤 → 多路召回 → RRF 融合 → Rerank → 上下文组装 → 带引用生成 → 评测反馈。
- 失败排查不要直接改 Prompt。先确认正确证据是否存在：不存在看采集/解析；存在但 Chunk 不完整看切分；正确 Chunk 未进候选看召回/过滤/索引；进候选后丢失看融合与重排；已进入上下文但回答错看组装和生成。
- 为每个阶段保留可观察中间结果和版本哈希，使用固定查询集逐层对照。指标分层：解析覆盖率、Recall@K、MRR/NDCG、上下文命中率、答案忠实性与引用正确率。

现象	优先检查	验证方法
答案缺关键事实	解析/切分	在原文和 Chunk 库检索该事实
正确 Chunk 未返回	索引/过滤/召回	旁路各召回器并核对 tenant/filter
候选有但最终没有	融合/Rerank	比较融合前后排名与打分
上下文有但回答错	组装/生成	检查截断、指令和引用约束

5. 可靠性、权限与沙箱

- 围绕五个故障练时序：任务重复投递、Worker 退出、外部结果未知、用户取消、等待人工确认。每个故障都回答：状态存在哪里、当前执行权属于谁、怎样检测、是否能重试、如何避免重复副作用。
- Checkpoint 只保证内部状态可恢复，不保证外部动作 exactly-once。副作用工具需要稳定幂等键、外部状态查询、唯一约束和补偿；结果未知时先进入 UNKNOWN/对账，不能盲目重试。
- Worker 用租约认领任务，写状态时校验单调 fencing token/CAS；取消是持久化事件，Worker 在安全点协作终止。人工等待保存 wait_id、允许输入和恢复节点。
- 权限要贯穿入口、运行时和工具层；沙箱覆盖文件、网络、进程、CPU/内存/时长与密钥。工具描述不可见不等于没有权限，Prompt 中的禁止事项也不能替代执行层鉴权。

故障	关键状态	正确动作
重复投递	run_id、幂等键、版本	去重或返回既有结果
Worker 退出	检查点、租约、防旧写令牌	租约过期后从已提交位置恢复
外部结果未知	外部任务编号、未知状态	先查询或对账，再补偿或人工确认
用户取消	取消事件、当前安全点	传递取消信号并记录已经发生的外部操作
等待人工	等待点编号、候选动作、期限	校验身份和版本后恢复

6. 评测、回归与指标口径

- 评测集来自真实任务抽样、专家设计边界例和生产 Bad Case；每例保存目标、约束、允许的多种答案/轨迹和必要证据。参考答案不是只写一段文本，而要说明什么算完成。
- 轨迹检查关注必调工具、禁止动作、参数合法性、证据使用和完成条件。LLM-as-a-Judge 使用明确 Rubric、盲测、多 Judge 或人工抽检校准，并监控与人工标签的一致率。
- 发布流程：离线回归 → 安全/故障注入 → 小流量影子或 A/B → 监控阈值 → 可一键回滚。报告样本量、置信区间、模型/Prompt/工具版本，避免只报一个平均分。
- 指标必须拆开：工具调用成功率、任务完成率、答案质量通过率、P50/P95 延迟、TTFT、Token 与单位任务成本。工具失败后绕路成功，任务可成功但工具调用仍计失败；工具全成功也不代表答案正确。

指标	回答的问题	不可混淆
工具调用成功率	基础能力是否健康	不等于任务完成
任务完成率	目标是否完成	不等于答案质量合格
答案质量通过率	答案或产物是否正确	需要独立评分标准或人工校准
延迟和成本	体验与资源效率	需要注明阶段、分位数和样本量

7. 后端基础与编码

- Python：asyncio 适合大量非阻塞 I/O；线程适合阻塞 I/O 和兼容同步库；进程适合 CPU 密集任务并绕过 GIL。协程中调用阻塞函数会卡住事件循环，应移入线程池/进程池或改为异步客户端。
- 后端组件：数据库重点准备索引、事务、慢查询和连接池；缓存准备一致性、穿透/击穿/雪崩；队列准备至少一次投递、幂等消费、顺序与积压；SSE 准备 event_id、断线续传、去重、心跳和背压。
- 编码练习应关闭 AI 补全，覆盖数组/哈希、堆、队列、树、LRU，以及 ReAct Loop、Plan-and-Execute、异步任务与取消。先写输入输出和主流程，再补边界、复杂度和最小测试。

并发方式	适用场景	主要风险
协程	高并发非阻塞 I/O	阻塞调用卡事件循环
线程	阻塞 I/O、同步 SDK	竞态、上下文切换
进程	CPU 密集、强隔离	序列化、进程间通信成本

一、字节跳动

一面

1. 为什么选择 LangGraph？与其他 Agent 框架相比，看中了哪些能力？

参考答案：选择依据是显式状态图、条件边、Checkpoint/中断恢复、流式事件和可插拔节点，适合长任务与人机协作。把它限定在核心编排层，工具治理、权限、租约、评测和业务扩展由自有层实现，避免深度绑定。同时用同一任务比较自研 Loop、LangGraph 与其他框架的复杂度和运行指标。

2. ReAct 循环在什么情况下会迟迟不结束？

参考答案：ReAct 迟迟不结束，通常是因为模型一直认为任务还没有完成。常见原因是完成条件不清楚、工具结果不足或持续报错、同一工具被反复调用、每轮不断重规划、上下文丢失关键目标，或提示词过度要求继续检查。处理时要明确完成条件，并加入最大轮次、重复调用检测、无进展检测和工具重试上限。

3. 追问：如何限制循环次数、上下文规模和工具调用范围？

参考答案：循环次数由最大轮次、总时长和无进展检测共同限制。上下文设置 Token 预算，超过阈值后裁剪、摘要并保留关键状态。工具范围先按用户权限、任务阶段和风险等级过滤，执行前再做服务端鉴权。三类限制都应记录触发原因并返回可诊断状态。

4. Tool Calling 的协议怎么设计？模型看到的工具描述与实际执行实现如何对应？

参考答案：注册表保存稳定 tool_id、版本、JSON 参数格式、权限、超时/重试、幂等性和执行适配器。模型只看到经权限与场景过滤后的短描述。解析后在服务端做参数格式校验、参数规范化、鉴权和风险审批，再执行并返回结构化工具结果（Observation）。用调用正确率、参数合法率和任务完成率评估。

5. Agent 执行到一半进程崩溃，恢复后怎样找到正确的执行位置？

参考答案：以 task_id/run_id 保存不可变输入、当前节点、步骤状态、上下文摘要、工具调用记录、等待事件和版本号。每个状态跃迁在同一事务中写入数据库。恢复时从最后一个已提交边界继续，并以租约/版本校验防止双执行。外部副作用还要靠幂等键、状态查询或补偿，Checkpoint 本身不保证外部操作只执行一次。

6. 追问：如果外部工具已经执行过了，恢复时怎么避免重复调用、重复产生副作用？

参考答案：为每次业务操作生成稳定 幂等键，并在工具侧建立唯一约束和结果表。超时且结果未知时先按 key 查询外部系统，不盲目重试。无法查询时把状态置为 UNKNOWN，走对账、补偿或人工确认。Harness 负责传播 key 和策略，业务工具负责真正的原子去重。

7. 面对理解需求、检索资料、生成报告等多步骤任务，会选择单 Agent 加工具，还是多 Agent？

参考答案：只有在职责可分解、上下文可隔离、可并行或需要不同权限/模型时才引入多 Agent。否则单 Agent + 工具更简单。Manager 生成任务契约，任务交接（Handoff）传目标、输入、约束、证据、预算和期望输出。用最大深度、调用图去环、租约和独立状态空间避免循环与冲突。

8. 追问：Manager-Worker 这样的架构适合什么情况？与单 Agent 的取舍是什么？

参考答案：只有在职责可分解、上下文可隔离、可并行或需要不同权限/模型时才引入多 Agent。否则单 Agent + 工具更简单。Manager 生成任务契约，任务交接（Handoff）传目标、输入、约束、证据、预算和期望输出。用最大深度、调用图去环、租约和独立状态空间避免循环与冲突。

9. Agent 运行时记录了哪些 Metrics、Trace 和 Log？

参考答案：每次请求绑定 trace_id/run_id/租户编号，Trace 记录模型、工具、节点和等待时段，Metric 统计成功率、完成率、P50/P95 延迟、Token/成本、重试和队列积压，Log 记录结构化错误但脱敏。可观测解决“发生了什么”，评测再把轨迹与任务质量标签连接起来。

10. RAG 使用什么向量存储？

参考答案：应直接说明实际使用的向量库及选择原因。选择主要看数据规模、过滤能力、索引更新、延迟、可用性、运维成本和私有化要求。还要说明向量库只负责向量召回，文档元数据、权限和原文通常仍由关系库或对象存储管理。

11. 检索返回了很多候选片段，但上下文空间有限，如何排序、压缩和组装？

参考答案：先按权限和元数据过滤，再结合向量分数、关键词分数和重排模型排序。去掉重复片段，并保留标题、来源和相邻上下文。根据 Token 预算选择高价值片段，过长内容做结构化摘要，最后按问题所需信息组织，而不是简单按分数拼接。

12. Agent 如何部署？逻辑上的 Agent 与进程、Worker 是什么关系？

参考答案：Agent 是业务上的执行单元，进程和 Worker 是运行它的计算资源。Agent 保存模型、提示词、工具权限、记忆引用和任务状态。进程是操作系统正在运行的程序。Worker 是领取并执行任务的工作单元，通常运行在某个进程或容器中。三者通常不是一一对应：请求到达后，调度器把 Agent 的任务交给空闲 Worker，Worker 从外部存储读取配置和状态，执行任务并写回新状态，结束后还可以继续处理其他 Agent 的任务。因此部署时一般分别部署入口服务、任务队列、共享 Worker 池和状态存储。只有需要强隔离、专用硬件或长期独占资源时，才为某个 Agent 单独分配进程或容器。

13. 追问：每个 Agent 都需要一个独立运行实例吗？

参考答案：不需要。大多数 Agent 共享一组 Worker，由调度器按任务分配。Agent 的配置和状态保存在外部存储中，所以任务可以换一个 Worker 继续执行。只有需要强隔离、专用硬件或长期独占资源时，才需要独立的进程或容器。

14. 后端平台与 AI 平台如何协作，哪些能力可以复用？

参考答案：后端平台提供账号、权限、租户、数据访问、消息、存储、发布和运维等通用能力。AI 平台负责模型接入、Prompt、RAG、Agent 运行、工具调用和评测。两边通过稳定接口协作，身份、审计、配置中心、网关和可观测系统应尽量复用，避免重复建设。

15. 如何统一接入不同的模型 Provider？

参考答案：业务代码不直接调用各家 Provider，而是统一调用内部的模型接口。该接口统一规定消息、模型参数、工具定义、返回内容、错误和用量的格式。每个 Provider 配置一个适配器，负责把统一请求转换成该 Provider 的格式，再把返回结果转换回统一格式。模型网关统一负责模型选择、鉴权、限流、重试、故障切换和调用记录。这样更换或增加 Provider 时，主要修改适配器和配置，不需要改动上层 Agent 逻辑。

16. 追问：不同 Provider 的流式协议、限流、降级和成本统计怎么处理？

参考答案：流式响应先由适配器转换成统一事件，包括内容增量、工具调用、完成和错误，上层只处理这一套事件。限流分别控制租户、Provider 和具体模型的调用量，避免部分请求占满全部资源。主模型超时、被限流或不可用时，模型网关可以切换到能力和数据安全要求都符合条件的备用模型。每次调用都记录模型、Provider、输入输出
Token、价格版本、缓存、重试次数和最终费用，再按任务或租户汇总。

17. 【算法题】 Two Sum 类问题，如何通过一次遍历找到目标组合？题面信息不完整，不还原额外约束。

参考答案：一次遍历维护 value→index 哈希表。对 nums[i] 先查
target-nums[i]，命中即返回两个下标，再写入当前值，能正确处理重复元素。时间 O(n)，空间 O(n)。代码见附录 A。

二面

1. 为什么考虑换工作？目前的工作地点和职级情况如何？

参考答案：先用正向原因说明换工作的目标，再如实给出当前工作城市、可接受地点、现职级和职责范围。不同公司的职级不可直接对齐，可以用负责模块、决策范围和团队影响补充。不要回避地点限制，也不要把薪资作为唯一原因。

2. AI 应用平台最初为什么立项，面向什么用户？

参考答案：建议答法：平台面向【内部开发者/企业客户】，把模型、知识、工具和流程统一成可发布、可治理的 AI 应用。核心解决重复接入、权限审计、长任务可靠性和效果回归。交付物应明确为产品实例、配置或扩展包、接口与运维指标，不能只说“交付一个 Agent”。

3. 追问：现在有哪些真实使用场景，用户实际怎么使用？

参考答案：应按真实情况说明用户是谁、从哪个入口进入、完成什么任务、使用频率如何，以及最终产物进入哪个业务流程。还要区分试用、概念验证、灰度和正式生产，不能只列功能名称。

4. 用户在平台上创建的应用如何发布、嵌入或集成到已有系统？

参考答案：应用先经过版本校验、权限检查、测试和审批，再发布为固定版本。外部系统可通过
API、SDK、网页组件或事件订阅接入。运行时要传递用户身份和租户信息，并支持灰度、回滚、限流和调用审计。

5. 如果从平台研发转向客户现场交付，可以沉淀哪些方法和可复用资产？

参考答案：可沉淀的内容包括需求分类、接入规范、领域数据模型、工具适配器、权限模板、评测集、部署脚本、监控规则和排障手册。客户专有逻辑应留在配置或扩展层，通用部分再进入平台产品。

6. 通用运行内核、业务扩展与服务入口各负责什么？

参考答案：服务入口负责认证、会话、限流、协议转换和流式输出。通用运行内核负责模型与工具循环、状态、预算、检查点、取消和观测。业务扩展负责领域
Skill、Workflow、数据适配、记忆策略和完成校验。三层通过稳定接口连接，业务代码不能直接修改内核。

7. 业务扩展由谁开发？客户最终买到的是产品、服务还是一套定制应用？

参考答案：业务扩展可由平台团队、交付团队或客户研发按统一接口开发，平台团队负责规范、审核和运行保障。客户购买的形态必须说清，可以是云服务、私有部署产品或产品加扩展包。专有代码的归属、升级和运维责任也要明确。

8. 云端部署与私有化部署分别怎么做？

参考答案：部署时通常把入口服务、任务队列、Worker
池和状态存储分开。入口负责接收请求，队列保存待执行任务，Worker 负责实际运行，数据库或对象存储保存配置、进度和结果。公有云部署更便于弹性扩缩。私有化部署则要确保模型、数据、日志和密钥都留在客户规定的边界内。

9. 追问：客户的数据不能出内网时，模型部署和调用链路如何处理？

参考答案：模型、向量库、文件存储、日志和工具执行都应部署在客户内网或客户允许的受控区域。若必须调用外部模型，只发送经过授权、最小化和脱敏的数据，并通过专线或网关审计。无法满足数据边界时应使用本地模型，不能让运行时绕过限制。

10. 工具在平台中怎样抽象？能否讲清一次工具调用的链路？

参考答案：注册表保存稳定 tool_id、版本、JSON 参数格式、权限、超时/重试、幂等性和执行适配器。模型只看到经权限与场景过滤后的短描述。解析后在服务端做参数格式校验、参数规范化、鉴权和风险审批，再执行并返回结构化工具结果（Observation）。用调用正确率、参数合法率和任务完成率评估。

11. Workflow 的工具节点和 ReAct 的 Tool Calling 能否复用同一套执行能力？

参考答案：可以复用同一套工具注册、参数校验、鉴权、超时、幂等和执行适配器。区别只在调用来源：Workflow
由固定节点发起，ReAct 由模型决策发起。两者都必须经过相同的服务端安全检查，并返回统一工具结果。

12. 运行时对 LangGraph 的依赖有多深？

参考答案：应把 LangGraph 限定在状态图和节点调度层。任务存储、租约、权限、工具执行、事件协议、评测和业务扩展通过自有接口实现。这样替换框架时主要重写图适配层和检查点转换，不会影响对外接口与业务能力。

13. 追问：Python 性能是否会成为瓶颈？

参考答案：先区分 I/O 密集与 CPU 密集：FastAPI 的 async 适合非阻塞 I/O，CPU 工作用多进程/独立 Worker。GIL 不妨碍 I/O 并发但限制单进程 Python 字节码多核执行。容量测试要给配置、负载模型、P95/P99、错误率和瓶颈。线上变慢按排队、应用、下游、数据库、模型逐段定位，SQL 用 EXPLAIN ANALYZE 检查扫描、基数估计与索引。

14. 用户身份和角色如何传递到运行时？工具和 Skill 的可见范围、调用权限怎样控制？

参考答案：用户身份和租户信息作为不可伪造的执行上下文贯穿网关、运行时和工具层。执行层按 RBAC/ABAC 对"主体-资源-动作"再次鉴权。存储按租户编号分区并加行级策略，缓存/队列/工作目录带租户命名空间，密钥用短期凭证。可见性过滤不能替代真正鉴权。

15. 日志、Trace 和指标存在哪里？不同人员如何获得查看和排障权限？

参考答案：日志进入集中日志系统，Trace 进入链路追踪系统，指标进入时序监控系统，并通过 run_id 和 trace_id 关联。查看权限按租户、项目、角色和数据敏感级别控制。普通用户只看自己的运行记录，运维和审计人员按授权查看更完整信息，敏感字段统一脱敏。

16. 排障效率的提升具体怎么测量？

参考答案：先统一故障开始、开始处理、完成定位和恢复服务的时间定义。然后统计定位耗时、恢复耗时、一次定位成功率、人工操作次数、人员交接次数和问题再次打开率。耗时既要看中位数，也要看P95，避免少量极慢故障被平均值掩盖。只有故障等级、系统范围和统计周期一致，改动前后的数据才可以比较。

17. 追问：比较基线和统计范围是什么，如何证明是这次改动带来的提升？

参考答案：基线应选择改动前的原有排障流程，或者同期未使用新方案的对照组。统计范围要固定系统、故障类型、严重等级、值班方式和时间窗口。归因时优先使用同一批故障回放、分批上线或对照实验，并排除流量、人员经验和故障难度的影响。如果只有简单的改动前后对比，只能说明两者相关，不能完全证明提升由本次改动造成。

18. 是否考虑过建设统一控制面？当前为什么做或不做？

参考答案：当 Agent、模型、工具、租户和版本数量增加，且发布、权限、成本和观测需要统一治理时，控制面值得建设。规模较小时可先使用配置中心和管理接口，避免过早建设复杂系统。是否建设应由治理成本和重复工作量决定。

19. 你在项目中具体负责哪些部分？

参考答案：明确说明自己负责的模块、做出的关键决策、亲自完成的代码或方案，以及与其他成员的边界。再说明交付结果和量化指标。团队已有能力、开源框架和个人新增能力要分开描述。

20. 行业 Agent 实际解决了什么问题，交付物是什么？

参考答案：应明确业务问题、原流程的成本或风险、Agent 负责的步骤和完成标准。交付物可以是可运行应用、领域Skill、Workflow、工具适配、评测集和运维配置。最终价值要用处理时间、完成率、人工工作量或错误率说明。

21. 托管式 Agent Runtime 是否可行，客户会因为什么选择它？

参考答案：托管服务是可行的，前提是能提供稳定运行、弹性资源、模型与工具接入、状态恢复、权限、观测和成本治理。客户选择它的原因通常是减少基础设施建设和运维，但数据边界、私有化和供应商锁定必须明确。

22. 【算法题】 将两个有序数组原地合并到具有预留空间的第一个数组中。

参考答案：从两个数组末尾向前比较，把较大值写入第一个数组的尾部预留区，避免覆盖未处理元素。最后只需补齐第二个数组的剩余元素。时间 O(m+n)，额外空间 O(1)。见附录 B。

三面

1. AI 应用平台的整体架构怎么设计，采用了哪些技术？

参考答案：可分为控制面（配置、版本、发布、评测）、入口层（鉴权/会话/限流）、运行面（调度、状态机、模型与工具、Checkpoint）、能力层（Skill/MCP/RAG/Memory）和基础设施（队列、存储、沙箱、观测）。请求经入口创建run，运行时逐步执行并持久化事件，最终输出答案/产物和完整审计记录。

2. 为什么选择 LangGraph？

参考答案：选择依据是显式状态图、条件边、Checkpoint/中断恢复、流式事件和可插拔节点，适合长任务与人机协作。把它限定在核心编排层，工具治理、权限、租约、评测和业务扩展由自有层实现，避免深度绑定。同时用同一任务比较自研Loop、LangGraph 与其他框架的复杂度和运行指标。

3. DSL 如何定义，又如何转换为执行图？

参考答案：DSL 是领域专用语言，在这里指用于描述 Agent 执行流程的一套结构化配置。它规定有哪些节点、每个节点需要什么输入、产生什么输出、节点之间怎样连接、什么条件下进入不同分支，以及失败后如何处理。系统读取 DSL 后，先检查格式、参数类型、节点引用和循环依赖是否合法，再把它转换成内部统一的数据结构，最后根据节点之间的依赖关系生成可以实际运行的有向图或状态图。自然语言可以辅助生成 DSL
草稿，但必须经过格式校验、流程检查和试运行后才能正式执行。

4. LangGraph 使用过程中遇到过什么问题？

参考答案：选择一个真实且影响明确的问题，依次说明现象、影响范围、定位过程、根因、修复方法和验证结果。还要说明后续增加了哪些监控、测试或防护，避免问题再次发生。

5. 追问：是否做过性能测试，系统容量如何？

参考答案：先区分 I/O 密集与 CPU 密集：FastAPI 的 async 适合非阻塞 I/O，CPU 工作用多进程/独立 Worker。GIL 不妨碍 I/O 并发但限制单进程 Python 字节码多核执行。容量测试要给配置、负载模型、P95/P99、错误率和瓶颈。线上变慢按排队、应用、下游、数据库、模型逐段定位，SQL 用 EXPLAIN ANALYZE 检查扫描、基数估计与索引。

6. 平台质量怎么衡量，主要看哪些指标？

参考答案：把评测拆成任务完成、答案质量、轨迹合规、延迟/成本和安全。参考标准（Ground
Truth）由真实任务抽样、专家标注和生产失败样本构成，保存期望结果及允许的多种轨迹。LLM Judge
用明确的评分标准、盲测和多 Judge/人工抽检校准，不能只让同类模型自评。发布前回归，线上灰度 A/B 并设回滚阈值。

7. 平台后续如何演进？业务 Agent 是否直接建立在原平台之上？

参考答案：演进重点应放在稳定接口和可替换能力：模型、框架、工具协议和记忆实现都通过适配层接入。内核保持任务状态、权限、恢复、观测和评测的稳定，新的模型或框架先经过兼容性和回归评测，再逐步替换。

8. LangGraph 之外，团队自行实现了哪些能力？

参考答案：通常需要自行实现模型网关、工具注册与鉴权、Skill 加载、租约和幂等、任务恢复、SSE
事件、上下文管理、评测、租户隔离和沙箱。回答时要区分框架原生能力、封装能力和团队新增能力。

9. 运行时怎样划分职责，通用执行层与业务扩展如何分工？

参考答案：服务入口处理认证、协议、会话、限流和
SSE。通用运行内核处理状态机、模型/工具循环、Checkpoint、预算和观测。业务扩展提供领域 Skill、Workflow、Memory 策略、数据适配与完成校验。依赖方向指向稳定接口，业务层不反向修改内核。

10. 【工程编码题】 写出 ReAct Loop 伪代码，并说明如何通过 Hook 接入 Skill、Memory 等能力。

参考答案：循环步骤是：加载状态与预算→注入必要 Skill/Memory→调用模型→解析 Final 或 Tool
Call→鉴权和参数校验→执行工具→记录 工具结果（Observation） 与
Checkpoint→进入下一轮。退出条件必须覆盖完成、轮次/Token 预算、取消和不可恢复错误。见附录 C。

11. Memory 在什么时候检索、更新和保存？

参考答案：任务开始或模型决策前，根据当前目标检索相关记忆。任务过程中只更新工作状态，不应每轮都写长期记忆。任务完成后再从确认过的事实和用户偏好中提取候选，经价值、隐私和可信度检查后保存。发生纠正或冲突时更新版本并保留来源。

12. 工具调用超时怎么处理？

参考答案：先按工具设置不同超时时间，并区分可安全重试与不可安全重试。无副作用调用可按退避策略重试。有副作用调用超时后先查询实际结果，再决定重试、补偿或人工处理。超时要作为结构化错误返回，并保存调用编号和当前任务状态。

13. 页面关闭后，后台任务如何继续执行？

参考答案：前端连接和后台任务必须分离。页面关闭只断开 SSE 或 WebSocket，不取消服务端任务。任务状态、步骤和事件持续写入持久化存储。用户重新打开页面后，客户端用任务编号查询状态，并从最后收到的事件编号继续订阅。只有明确的取消请求才终止任务。

14. 多个 Worker 怎么避免执行冲突？

参考答案：Worker 通过带过期时间的租约认领任务，控制面发放单调递增的
防旧写令牌。所有状态写入和外部资源操作都校验 token/版本并用 CAS（比较并交换） 提交，旧 Worker
即使“复活”也会被拒绝。租约到期后任务才能安全转移。

15. 如何看待 Agent 与 Harness 的后续演进？

参考答案：Agent 会继续提高规划、工具使用和多模态能力，Harness
则会更重视状态持久化、工作空间、权限、评测、成本和跨模型适配。模型能力增强不会消除
Harness，反而要求运行环境提供更稳定的边界和验证。架构上应让模型与框架可替换。

16. 目前的职级是什么？

参考答案：如实给出公司职级、对应职责范围和时间线。晋升用“承担范围扩大 + 关键结果 +
组织影响”说明，绩效与薪酬只报可公开且可核验的信息。若体系不可横向比较，可补充带队规模、决策权和交付复杂度。

二、千问

一面

1. 介绍一下当前工作和比较擅长的方向。

参考答案：按时间顺序说明当前岗位、主要工作和转向 Agent 的阶段，再说明最熟悉的技术方向。重点写清实际负责的模块、投入时间和已经交付的结果，不要把学习、验证和生产经验混在一起。

2. 对工作地点是否有要求，能否接受异地机会？

参考答案：按真实情况给明确边界：首选【城市】，可接受【城市/出差频率/迁移时间】。若有家庭或通勤约束，直接说明可协商范围和最早到岗时间。

3. AI 应用平台与业务 Agent 项目是什么关系？

参考答案：应用平台提供模型、知识库、工具、发布和治理等通用能力，业务 Agent 在其上加入领域流程、业务数据、权限规则和完成标准。能复用的平台能力继续复用。长任务恢复、复杂运行状态或强业务约束不足时，再由独立运行时补充。

4. 你在其中具体承担哪些工作？

参考答案：明确说明自己负责的模块、做出的关键决策、亲自完成的代码或方案，以及与其他成员的边界。再说明交付结果和量化指标。团队已有能力、开源框架和个人新增能力要分开描述。

5. 为什么需要单独建设 Harness，它要解决什么问题？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

6. 企业 Agent 除了 ReAct、Tool Calling 和上下文管理，还需要补哪些能力？

参考答案：还需要身份与权限、工具治理、长任务状态、取消与恢复、上下文预算、长期记忆、沙箱、多租户、观测、评测、成本控制和发布回滚。ReAct 只解决模型与工具怎样循环，不能独立保证生产可靠性。

7. 通用运行内核、业务扩展与服务入口如何划分？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

8. 业务扩展模块具体包含什么？

参考答案：服务入口处理认证、协议、会话、限流和
SSE。通用运行内核处理状态机、模型/工具循环、Checkpoint、预算和观测。业务扩展提供领域 Skill、Workflow、Memory 策略、数据适配与完成校验。依赖方向指向稳定接口，业务层不反向修改内核。

9. 这些模块如何注册、管理版本和升级？

参考答案：模块通过带唯一标识和版本的清单注册，清单包含接口、依赖、权限和兼容范围。发布时做校验和测试，运行任务固定使用已发布版本。升级采用灰度和回滚，进行中的任务不能自动切换到新版本。

10. 运行内核如何与具体业务实现解耦？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

11. Memory 策略应该由业务决定，还是固定在运行内核中？

参考答案：运行内核只定义记忆的读取、写入、删除、版本和权限接口，不应固定业务记忆内容。业务层决定记什么、何时写、怎样冲突处理和保留多久。安全、隐私、审计和租户隔离属于平台强制规则，业务策略不能绕过。

12. 能力声明文件需要描述哪些信息？

参考答案：应包含能力标识、版本、用途、输入输出格式、前置条件、依赖、权限、超时重试、幂等性、资源限制、执行入口、产物类型和兼容范围。声明只描述契约，不应把密钥或敏感配置写入文件。

13. Tool、Skill、Artifact 和 Hook 如何组织？

参考答案：Tool 提供原子动作，Skill 说明何时和怎样组合能力，Artifact 保存可继续编辑的结构化产物，Hook 在模型调用、工具执行和状态变化等固定位置接入扩展逻辑。四者都通过带版本的接口注册，由运行时统一做权限、加载、执行和观测。

14. Workflow 与 ReAct 如何配合？

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

15. 业务扩展到什么程度应当修改内核，边界在哪里？

参考答案：只有当多个业务反复需要同一种生命周期、状态、权限或执行能力，且现有扩展接口无法安全实现时，才考虑修改内核。单一客户规则、领域数据和流程都应留在扩展层。修改内核前必须证明需求具有通用性并评估兼容影响。

16. 目前有多少类业务扩展，是否经过不同场景验证？

参考答案：按真实数据说明扩展类别、已接入数量和覆盖场景，并区分生产、灰度和验证状态。还要说明各类扩展是否复用同一接口、是否有独立评测，以及哪些场景尚未验证。

17. 多人同时开发不同业务能力时，怎样协作和集成？

参考答案：按稳定接口和业务域拆分模块，使用独立版本、测试数据和运行命名空间。合并前执行接口兼容检查、自动测试、代码评审和端到端回归。共享内核只接受通过扩展接口的能力，禁止业务模块直接修改全局状态。

18. 不同业务独立部署，还是共享运行时？

参考答案：部署时通常把入口服务、任务队列、Worker
池和状态存储分开。入口负责接收请求，队列保存待执行任务，Worker 负责实际运行，数据库或对象存储保存配置、进度和结果。公有云部署更便于弹性扩缩。私有化部署则要确保模型、数据、日志和密钥都留在客户规定的边界内。

19. 追问：共享时怎么做隔离？

参考答案：用户身份和租户信息作为不可伪造的执行上下文贯穿网关、运行时和工具层。执行层按 RBAC/ABAC 对"主体-资源-动作"再次鉴权。存储按租户编号分区并加行级策略，缓存/队列/工作目录带租户命名空间，密钥用短期凭证。可见性过滤不能替代真正鉴权。

20. Skill 如何路由和选择？

参考答案：为 Skill 建结构化清单：名称、意图、前置条件、输入输出、权限、成本和版本。先按租户/权限/场景做确定性过滤，再用关键词+Embedding 召回少量候选，必要时让路由模型重排。离线用命中率/误选率，线上用完成率、纠正率和成本评估，并支持显式选择兜底。

21. 用户怎样发现和使用已有 Skill？

参考答案：平台应提供按权限过滤的 Skill 目录、搜索、标签、说明和版本信息。用户可显式选择，也可由运行时根据任务召回候选。启用前展示输入、输出、权限和成本，执行时仍需服务端鉴权。

22. Skill 是代码实现还是配置定义？

参考答案：两者都可以包含。配置描述用途、输入输出、权限、依赖和版本。代码或 Workflow 提供实际执行。纯说明型 Skill 只指导模型，执行型 Skill 必须有受控入口、测试、沙箱和审计。

23. 是否提供沙箱，主要解决什么问题？

参考答案：执行放到短生命周期、无特权的隔离环境：只读基础镜像、独立用户和命名空间、CPU/内存/进程数/时长配额、临时工作目录和默认断网。文件与网络按白名单开放。调度器在沙箱外做鉴权、审计和 Kill，死循环由超时与
资源配额终止。

24. 沙箱带来的延迟与安全收益怎么权衡？

参考答案：高风险代码、文件写入和网络访问应使用强隔离，即使启动成本更高。低风险只读任务可使用预热容器、进程池或较轻隔离。通过容器池、镜像预拉取和资源复用降低延迟，但不能为了性能取消权限和资源限制。

25. Serverless 与临时创建 Kubernetes Pod 的方式有什么差别？

参考答案：部署时通常把入口服务、任务队列、Worker
池和状态存储分开。入口负责接收请求，队列保存待执行任务，Worker 负责实际运行，数据库或对象存储保存配置、进度和结果。公有云部署更便于弹性扩缩。私有化部署则要确保模型、数据、日志和密钥都留在客户规定的边界内。

26. 目前的用户量和并发规模如何？

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

27. 实时交互与后台长任务如何安排 Worker?

参考答案：实时任务使用低延迟队列和短时 Worker，优先保证首响应。后台长任务进入独立队列，由可释放和恢复的 Worker 执行。两类任务使用不同并发、超时和资源配额，避免长任务占满实时资源。状态统一外置，任务可跨 Worker 继续。

28. 你在平台项目中负责哪些模块?

参考答案：明确说明自己负责的模块、做出的关键决策、亲自完成的代码或方案，以及与其他成员的边界。再说明交付结果和量化指标。团队已有能力、开源框架和个人新增能力要分开描述。

29. 平台的实际使用规模怎样，是否已有线上用户?

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

30. 应用如何发布和集成到外部系统?

参考答案：应用先经过版本校验、权限检查、测试和审批，再发布为固定版本。外部系统可通过 API、SDK、网页组件或事件订阅接入。运行时要传递用户身份和租户信息，并支持灰度、回滚、限流和调用审计。

31. 底层执行引擎是什么?

参考答案：说明实际使用的图引擎或自研状态机，以及它负责的最小范围：节点调度、状态传递、条件跳转和检查点。工具执行、权限、任务租约和业务逻辑是否位于引擎之外也要说清楚。

32. 画布 DSL 如何转换为执行图?

参考答案：DSL 先经过语法、参数格式、类型、引用和环路校验，再解析为带版本的 IR，完成默认值、依赖解析和节点绑定，最后编译为 DAG/状态图。自然语言只生成受限 IR 草案，不直接发布最终 DSL。生成后做静态校验、沙箱试跑、测试用例验证和基于错误的有限修复。

33. 是否比较过 AgentScope、pi-agent 等框架，各自有什么特点?

参考答案：不要只列功能。用同一组任务比较状态模型、持久化恢复、多 Agent 协议、工具生态、可观测性、部署复杂度、扩展边界和社区成熟度，再做最小概念验证（PoC）。最终选择取决于团队语言栈、故障模型和交付形态，并保留适配层降低替换成本。

34. ToB 与 ToC Agent 的主要区别是什么?

参考答案：ToB 更重租户隔离、审批审计、私有化、领域流程和 SLA。ToC 更重低延迟、个性化 Memory、海量弹性和单位成本。内核可复用，但身份、权限、记忆授权、资源调度和产品交互策略应分别设计，不能简单复制。

35. 面向 ToC 时，Memory 应该怎样设计?

参考答案：工作记忆只服务当前步骤，短期记忆覆盖会话，长期记忆保存跨会话且稳定的事实/偏好。写入由策略或抽取器产生候选，再经敏感性、价值、置信度和用户授权过滤。检索结合租户过滤、时间衰减与语义相关度，具体存储可按规模选择关系库、全文或向量索引。

36. 工具越来越多，会带来哪些上下文和选择噪声，如何处理？

参考答案：先按权限、业务域、任务阶段和输入类型过滤，再从剩余工具中召回少量候选，只把候选摘要放入上下文。模型选中后再加载完整参数格式。通过工具选择准确率、参数错误率和任务完成率评估。

37. Skill 规则与 Memory 冲突时怎么办？

参考答案：记忆保存事实、偏好、来源、时间、置信度和版本，不做无脑追加。新旧冲突按来源可靠性、时效和用户确认处理：保留新记录，并把旧记录标记为已被替代，同时保留审计。错误内容支持删除和更正，低价值内容按访问频率、过期时间和收益评分淘汰。硬规则始终高于用户记忆。

38. Python 和 Java 分别使用到什么程度？

参考答案：分别说明两种语言的使用年限、最近使用时间、承担过的模块和能够独立完成的工作。Python
可从异步服务、Agent 和数据处理说明，Java
可从微服务、并发、数据库与中间件说明。只写真实熟练度，不用把两种语言说成同一水平。

39. 如果团队使用另一种语言，准备怎样适应？

参考答案：先承认语言切换需要学习成本，再说明会优先掌握团队的工程规范、构建工具、并发模型、依赖管理和测试方式。通过小模块逐步进入主链路，并借助代码评审降低早期风险。

二面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent
工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

2. 最近在读哪些 Agent 框架？

参考答案：如实说明阅读过的项目、跟踪的调用链和完成的验证。重点应覆盖会话创建、上下文组装、工具执行、退出条件、状态保存、恢复和流式事件，并说明自己确认了哪些设计限制。

3. 是否看过 Coding Agent 的实现，重点研究了哪些部分？

参考答案：如实说明阅读过的项目、跟踪的调用链和完成的验证。重点应覆盖会话创建、上下文组装、工具执行、退出条件、状态保存、恢复和流式事件，并说明自己确认了哪些设计限制。

4. 怎样判断一个 Harness 好不好？

参考答案：看它能否稳定提高任务完成率，同时控制权限、恢复、延迟和成本。具体检查上下文管理、工具正确率、长任务恢复、可观测性、评测、扩展接口和部署复杂度。判断必须基于同任务对照和故障测试，不能只看功能数量。

5. 自己的运行时实现了什么，能力边界在哪里？

参考答案：运行时接收任务、身份、配置、上下文和资源限制，负责模型与工具循环、状态保存、恢复、预算和事件输出。输出包括最终答案或产物、结构化状态、执行轨迹和用量指标。它不负责具体业务规则、业务数据定义和模型训练。

6. 通用运行内核、业务扩展和服务入口怎样分层？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

7. Tool、Skill 和 Memory 的接口分别如何设计？

参考答案：工作记忆只服务当前步骤，短期记忆覆盖会话，长期记忆保存跨会话且稳定的事实/偏好。写入由策略或抽取器产生候选，再经敏感性、价值、置信度和用户授权过滤。检索结合租户过滤、时间衰减与语义相关度，具体存储可按规模选择关系库、全文或向量索引。

8. 运行时的输入和输出是什么？

参考答案：运行时接收任务、身份、配置、上下文和资源限制，负责模型与工具循环、状态保存、恢复、预算和事件输出。输出包括最终答案或产物、结构化状态、执行轨迹和用量指标。它不负责具体业务规则、业务数据定义和模型训练。

9. 多个业务并行扩展时，怎样避免互相影响？

参考答案：按稳定接口和业务域拆分模块，使用独立版本、测试数据和运行命名空间。合并前执行接口兼容检查、自动测试、代码评审和端到端回归。共享内核只接受通过扩展接口的能力，禁止业务模块直接修改全局状态。

10. 如何评估自研系统与开源方案的差异？

参考答案：不要只列功能。用同一组任务比较状态模型、持久化恢复、多 Agent 协议、工具生态、可观测性、部署复杂度、扩展边界和社区成熟度，再做最小概念验证（PoC）。最终选择取决于团队语言栈、故障模型和交付形态，并保留适配层降低替换成本。

11. 追问：是否真正用相同任务对比过通用 Agent？

参考答案：应使用相同模型、工具、上下文、任务和预算做对照，只改变 Harness 或运行策略。比较任务完成率、轨迹合规、延迟、成本和人工干预次数。若没有做过严格对照，应直接说明目前只有局部测试，并列出仍需验证的假设。

12. 测试用例、Ground Truth 和 A/B 对比具体怎么做？

参考答案：把评测拆成任务完成、答案质量、轨迹合规、延迟/成本和安全。参考标准（Ground Truth）由真实任务抽样、专家标注和生产失败样本构成，保存期望结果及允许的多种轨迹。LLM Judge 用明确的评分标准、盲测和多 Judge/人工抽检校准，不能只让同类模型自评。发布前回归，线上灰度 A/B 并设回滚阈值。

13. 当前使用哪些模型？

参考答案：按任务类型说明实际使用的对话、Embedding、Rerank 和多模态模型，并给出版本、部署方式和选择依据。还要说明备用模型、数据边界、延迟和成本。只写真实使用过或正式评估过的模型。

14. 是否做过模型后训练？

参考答案：如实区分应用调用、推理部署和训练经验。多数企业 Agent 先用 Prompt、工具、RAG 与评测解决问题。只有错误模式稳定、数据量足且收益可验证时才考虑 SFT/DPO，并建立数据授权、离线评测和灰度回滚。

15. 框架给开发者提供什么接口，开发者需要写哪些代码？

参考答案：框架应提供模型、工具、Skill、Memory、Workflow、Hook、状态存储和事件输出接口。开发者主要实现业务工具、领域流程、数据适配和完成校验，不需要修改主循环。接口必须带版本、权限、输入输出格式和测试约束。

16. 与 AgentScope 相比，有什么不同？

参考答案：不要只列功能。用同一组任务比较状态模型、持久化恢复、多 Agent 协议、工具生态、可观测性、部署复杂度、扩展边界和社区成熟度，再做最小概念验证（PoC）。最终选择取决于团队语言栈、故障模型和交付形态，并保留适配层降低替换成本。

17. 是否了解托管式 Agent 服务？

参考答案：托管服务是可行的，前提是能提供稳定运行、弹性资源、模型与工具接入、状态恢复、权限、观测和成本治理。客户选择它的原因通常是减少基础设施建设和运维，但数据边界、私有化和供应商锁定必须明确。

18. 目前有多少真实用户，实际使用情况如何？

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

19. 开发中遇到的最大挑战是什么？

参考答案：选择一个真实且影响明确的问题，依次说明现象、影响范围、定位过程、根因、修复方法和验证结果。还要说明后续增加了哪些监控、测试或防护，避免问题再次发生。

20. 较重的 Workflow 如何与 ReAct 主循环结合？

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

21. Sub-agent 如何定义和启动？

参考答案：Sub-agent 用结构化契约定义目标、输入输出、工具、权限、预算和结束条件。主 Agent 或调度器创建子任务并分配独立状态与租约，子任务完成后通过任务交接结果返回。启动前要校验是否真的需要独立上下文或权限。

22. 追问：子 Agent 是提前声明的，还是运行过程中动态创建的？

参考答案：核心角色和高权限 Agent 应提前声明并审核。运行过程中可以动态创建受限子任务，但只能从已批准的能力模板中选择工具、权限和预算，不能让模型生成任意新权限。两种方式都要限制深度和总任务数。

23. 为什么考虑离开当前公司？

参考答案：用正向原因回答：当前工作已经形成了哪些积累，下一阶段希望扩大哪类责任或技术深度，以及目标岗位为什么更匹配。不要评价原公司的人和管理，也不要只把薪资作为原因。

24. 为什么选择这个方向和这家公司？

参考答案：用正向动机回答：现岗位让我积累了【能力】，下一步希望在更大规模、更完整责任边界下深耕 Agent 工程化。目标岗位在【业务/技术/团队】上与此匹配。不要抱怨原公司，也不要只说薪资。补充未来 2-3 年希望承担的结果。

25. 如何看待当前公司的优势与不足？

参考答案：优势应从业务、团队或工程能力中选择自己真正了解的部分。不足要描述为客观限制，并说明它对工作产生的影响以及自己采取过的改进。避免情绪化评价或涉及保密信息。

26. 后续职业规划是什么？

参考答案：用正向动机回答：现岗位让我积累了【能力】，下一步希望在更大规模、更完整责任边界下深耕 Agent 工程化。目标岗位在【业务/技术/团队】上与此匹配。不要抱怨原公司，也不要只说薪资。补充未来 2-3 年希望承担的结果。

三面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

2. 入职当前公司时是什么级别，现在是什么级别？

参考答案：如实说明入职级别、当前级别和对应时间。重点补充职责范围怎样变化、是否承担关键设计或团队协作责任，以及变化由哪些可核验结果支持。不同公司的职级不能直接横向等同。

3. 是否有晋升经历？如有，最近一次是什么时候？

参考答案：如实给出公司职级、对应职责范围和时间线。晋升用“承担范围扩大 + 关键结果 + 组织影响”说明，绩效与薪酬只报可公开且可核验的信息。若体系不可横向比较，可补充带队规模、决策权和交付复杂度。

4. 追问：如果有晋升经历，主要凭借哪些工作和贡献？

参考答案：如实给出公司职级、对应职责范围和时间线。晋升用“承担范围扩大 + 关键结果 + 组织影响”说明，绩效与薪酬只报可公开且可核验的信息。若体系不可横向比较，可补充带队规模、决策权和交付复杂度。

5. 如果有晋升和转向 AI 研发的经历，两者的时间先后关系是什么？

参考答案：如实给出公司职级、对应职责范围和时间线。晋升用“承担范围扩大 + 关键结果 + 组织影响”说明，绩效与薪酬只报可公开且可核验的信息。若体系不可横向比较，可补充带队规模、决策权和交付复杂度。

6. 介绍一下当前业务 Agent 项目。

参考答案：按“问题—目标—方案—个人职责—结果”组织。先说明项目解决什么问题和为什么值得做，再讲核心架构与关键难点，最后给出上线范围、效果、稳定性或交付周期。选择能体现岗位能力且个人贡献清楚的项目。

7. FDE 在实际工作中意味着什么？

参考答案：FDE 通常指深入客户场景、同时承担需求澄清、方案设计、集成实施和结果验证的工程角色。重点不是临时定制，而是把现场问题转化为可复用的产品能力，并对最终业务结果负责。

8. 项目团队有哪些角色，怎样分工？

参考答案：按真实团队说明产品、业务、算法、平台、后端、前端、测试、交付和运维的职责。重点说明需求、架构、代码、评测、发布和线上问题分别由谁负责，以及自己处于哪个协作位置。

9. 是否所有研发人员都参与运行内核开发？

参考答案：通常不需要。少量平台人员维护运行内核和稳定接口，业务研发通过 Skill、Workflow、工具和数据适配扩展能力。这样能减少内核变更和回归风险，同时让业务团队独立交付。

10. 业务 Agent 是否建立在原有应用平台之上？

参考答案：应用平台提供模型、知识库、工具、发布和治理等通用能力，业务 Agent 在其上加入领域流程、业务数据、权限规则和完成标准。能复用的平台能力继续复用。长任务恢复、复杂运行状态或强业务约束不足时，再由独立运行时补充。

11. 你具体负责哪部分？

参考答案：明确说明自己负责的模块、做出的关键决策、亲自完成的代码或方案，以及与其他成员的边界。再说明交付结果和量化指标。团队已有能力、开源框架和个人新增能力要分开描述。

12. 架构方案由谁决定，你参与了哪些设计决策？

参考答案：先说明最终决策人和评审机制，再列出自己负责评估和推动的设计项。回答应包含候选方案、主要约束、取舍依据和最终责任边界，避免把团队决策全部说成个人决定。

13. 为什么采用运行内核、业务扩展和服务入口的分层方式？

参考答案：三层的变化速度和责任不同。服务入口处理协议与用户访问，运行内核保证通用执行和可靠性，业务扩展承载领域差异。分层后可以独立升级、测试和复用，并防止业务代码侵入核心执行流程。

14. 运行内核内部包含哪些模块？

参考答案：核心模块通常包括任务状态机、模型调用、工具执行、上下文预算、检查点与恢复、取消与超时、租约、事件输出、观测和扩展接口。权限、租户和资源限制贯穿各模块。具体业务流程与数据适配不应放入内核。

15. Workflow 和 ReAct 如何结合？

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

16. 除此之外，还有哪些比较有特点的设计？

参考答案：选择已经落地且能说明价值的设计，重点讲它解决的问题、所在层次、与常规方案的差异、带来的结果和适用限制。不要罗列功能名称，也不要把仍在设想中的方案说成已经实现。

17. 为什么把 Memory 实现放在运行内核之外？

参考答案：不同业务对记忆内容、写入条件、保留时间和冲突处理的要求不同，写死在内核会造成强耦合。内核只提供统一接口和安全边界，具体记忆实现由业务扩展注入。这样可以替换存储与策略而不修改主循环。

18. 记忆接口如何定义？

参考答案：至少提供检索、候选写入、确认保存、更新、删除和审计接口。请求包含租户、用户、任务、记忆类型和权限上下文，结果返回内容、来源、时间、置信度和版本。接口与具体向量库或数据库解耦。

19. 追问：具体实现由谁提供，又怎样注入运行时？

参考答案：业务团队通过统一接口提供具体实现，并在发布时注册实现标识、版本、依赖和权限。运行时只依赖接口，通过配置和依赖注入加载对应实现。任务开始后固定版本，避免运行过程中被替换。

20. 上下文工程做了哪些工作？

参考答案：把预算分给系统/策略、当前任务、近期对话、检索证据和工具结果，并预留输出空间。达到 70%-80% 阈值时先删重复与低价值日志，再对旧历史和长工具结果做结构化摘要。必须保留目标、约束、关键事实、未完成项、来源引用和工具副作用记录，原文留在外部存储以便按需回取。

21. Token 预算如何分配，什么时候触发压缩？

参考答案：把预算分给系统/策略、当前任务、近期对话、检索证据和工具结果，并预留输出空间。达到 70%-80% 阈值时先删重复与低价值日志，再对旧历史和长工具结果做结构化摘要。必须保留目标、约束、关键事实、未完成项、来源引用和工具副作用记录，原文留在外部存储以便按需回取。

22. 工具结果和历史消息分别怎么压缩？

参考答案：把预算分给系统/策略、当前任务、近期对话、检索证据和工具结果，并预留输出空间。达到 70%-80% 阈值时先删重复与低价值日志，再对旧历史和长工具结果做结构化摘要。必须保留目标、约束、关键事实、未完成项、来源引用和工具副作用记录，原文留在外部存储以便按需回取。

23. 这些设计中，哪些已经做过实际验证？

参考答案：按生产使用、灰度验证、离线测试和方案推演四个层级分别说明。每项设计都要给出测试范围、指标、时间和限制。没有真实验证的内容应明确标注，不能用代码完成代替效果验证。

24. 平时怎样追踪 Agent 领域的变化？

参考答案：固定关注官方文档、版本说明、核心仓库、技术论文和可靠的工程实践。对重要变化先阅读接口与测试，再用小型任务验证行为、性能和兼容性，最后判断是否影响现有系统。只说明真实阅读和验证过的内容。

25. 最近观察到哪些值得关注的架构趋势？

参考答案：可从长任务持久化、统一工具协议、渐进式能力加载、工作空间与 Artifact、上下文压缩、计算机操作和多模型路由说明。回答重点是这些变化会影响现有架构的哪一层，以及是否经过实际验证。

26. 下一阶段的职业发展考虑是什么？

参考答案：用正向动机回答：现岗位让我积累了【能力】，下一步希望在更大规模、更完整责任边界下深耕 Agent 工程化。目标岗位在【业务/技术/团队】上与此匹配。不要抱怨原公司，也不要只说薪资。补充未来 2-3 年希望承担的结果。

27. 工作地点上有哪些考虑？

参考答案：按真实情况给明确边界：首选【城市】，可接受【城市/出差频率/迁移时间】。若有家庭或通勤约束，直接说明可协商范围和最早到岗时间。

三、淘天

一面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

2. 单 Agent、多 Agent 和 Workflow 分别适合什么场景？

参考答案：单 Agent 适合工具数量可控、上下文能够统一管理的开放任务。多 Agent 适合职责、上下文或权限需要隔离，并且存在并行收益的任务。Workflow 适合步骤稳定、审计严格和失败代价高的流程。应先选择最简单且能满足要求的方案。

3. 它们的核心区别是什么？

参考答案：单 Agent 由一个上下文和一个决策循环完成任务，结构最简单。多 Agent 把任务交给多个独立角色，适合上下文、权限或职责需要隔离的情况。Workflow 的步骤和流转规则由系统预先确定，稳定性和可审计性更强，但自主调整能力较弱。

4. 任务复杂，就一定应该用多 Agent 吗？

参考答案：只有在职责可分解、上下文可隔离、可并行或需要不同权限/模型时才引入多 Agent。否则单 Agent + 工具更简单。Manager 生成任务契约，任务交接（Handoff）传目标、输入、约束、证据、预算和期望输出。用最大深度、调用图去环、租约和独立状态空间避免循环与冲突。

5. 追问：什么条件下才值得引入多 Agent？

参考答案：只有在职责可分解、上下文可隔离、可并行或需要不同权限/模型时才引入多 Agent。否则单 Agent + 工具更简单。Manager 生成任务契约，任务交接（Handoff）传目标、输入、约束、证据、预算和期望输出。用最大深度、调用图去环、租约和独立状态空间避免循环与冲突。

6. 为什么将 Workflow 封装为 Skill？

参考答案：封装后，Agent 可以把一段经过验证的固定流程当作一个受控能力调用，而不需要理解内部每个节点。这样能复用权限、版本、输入输出和观测机制，并减少上下文长度。代价是自由度下降和维护一个额外能力契约。

7. 一次用户请求从进入到结束，生命周期是什么样的？

参考答案：请求进入后先完成鉴权、限流和会话绑定，再创建任务并加载配置、身份、上下文和检查点。运行时调用模型，执行工具或流程节点，并持续保存状态和输出事件。结束前检查任务是否真正完成，随后保存答案或产物、记录指标，并释放资源。失败、取消和等待人工输入都应成为明确的任务状态。

8. 运行内核、业务扩展和服务入口分别负责什么？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

9. 一次完整任务与其中一轮模型推理、工具调用是什么关系？

参考答案：完整任务是从接收请求到最终完成的整个运行过程，可以包含多轮模型推理和多次工具调用。一轮推理只负责根据当前状态作出下一步决定，工具调用是该决定产生的动作。任务级状态负责预算、取消、恢复和最终完成判定，单轮状态只记录本轮输入、输出和工具结果。

10. Skill 如何被选中，并进入上下文？

参考答案：先按租户、权限、任务阶段和前置条件过滤，再从剩余 Skill 中召回和排序。模型选中后只加载该 Skill 当前需要的说明、工具和资源，不把全部内容一次放入上下文。执行前仍要服务端鉴权。

11. 工具调用什么时候并行，什么时候串行？

参考答案：相互独立、没有共享写入、没有前后依赖的工具调用可以并行。后一步依赖前一步结果、多个调用会修改同一资源，或调用顺序影响业务结果时必须串行。并行前还要检查限流、资源配额和失败处理，结果汇总时要保留每个调用的状态。

12. 为什么选择 LangGraph？

参考答案：选择依据是显式状态图、条件边、Checkpoint/中断恢复、流式事件和可插拔节点，适合长任务与人机协作。把它限定在核心编排层，工具治理、权限、租约、评测和业务扩展由自有层实现，避免深度绑定。同时用同一任务比较自研 Loop、LangGraph 与其他框架的复杂度和运行指标。

13. Agent 效果从哪些维度评估？

参考答案：至少分为任务完成度、答案或产物质量、轨迹合规、安全、延迟、稳定性和成本。每个维度使用独立指标，不能用单一成功率概括。还要按场景和难度分组，避免平均值掩盖关键失败。

14. 测试用例和参考执行轨迹怎么构建？

参考答案：用例来自真实任务、边界条件和线上失败记录。每条用例应包含输入、约束、必要证据、完成条件和允许的结果范围。参考轨迹由业务与工程人员共同确认，只规定必须经过和禁止发生的动作，不强制唯一执行路径。

15. LLM-as-a-Judge 具体评价哪些内容？

参考答案：评价任务是否完成、答案是否正确完整、是否忠于证据、格式是否符合要求，以及轨迹是否满足规则。评分标准要拆成独立维度，并要求给出依据。延迟、成本和安全事件应由系统指标评价，不交给 Judge 猜测。

16. 怎样把评测接入发布和回归流程？

参考答案：每次修改模型、Prompt、工具、检索或流程后自动运行固定回归集，并与当前线上版本比较。未达到质量、安全、延迟和成本阈值时禁止发布。上线后抽样真实任务进入评测集，新增失败记录后再进行版本化回归。

17. 如果上线后效果下降，怎么定位和回滚？

参考答案：按数据→预处理→Embedding/Reranker 版本→索引构建/增量同步→查询改写→过滤条件→服务缓存的顺序排查。先复现同一批查询，核对文档数、向量维度、索引时间和配置哈希，再做组件旁路对照，快速定位从哪一阶段开始偏离。

18. 如何防范 Prompt Injection?

参考答案：把网页/文档视为不可信数据，与系统指令分层标记，禁止其直接改变权限和工具策略。高风险动作由确定性策略引擎二次鉴权。再配合工具白名单、参数约束、最小权限、敏感信息脱敏和人工确认，并用注入样本做回归测试。

19. 沙箱承担哪些安全职责?

参考答案：执行放到短生命周期、无特权的隔离环境：只读基础镜像、独立用户和命名空间、CPU/内存/进程数/时长配额、临时工作目录和默认断网。文件与网络按白名单开放。调度器在沙箱外做鉴权、审计和 Kill，死循环由超时与
资源配额终止。

20. Agent 需要访问敏感数据时，怎样控制权限?

参考答案：用户身份和租户信息作为不可伪造的执行上下文贯穿网关、运行时和工具层。执行层按 RBAC/ABAC 对"主体-资源-动作"再次鉴权。存储按租户编号分区并加行级策略，缓存/队列/工作目录带租户命名空间，密钥用短期凭证。可见性过滤不能替代真正鉴权。

21. 上下文接近模型窗口上限时怎么办?

参考答案：把预算分给系统/策略、当前任务、近期对话、检索证据和工具结果，并预留输出空间。达到 70%-80% 阈值时先删重复与低价值日志，再对旧历史和长工具结果做结构化摘要。必须保留目标、约束、关键事实、未完成项、来源引用和工具副作用记录，原文留在外部存储以便按需回取。

22. 平时如何保持上下文干净，避免无关信息不断积累?

参考答案：把预算分给系统/策略、当前任务、近期对话、检索证据和工具结果，并预留输出空间。达到 70%-80% 阈值时先删重复与低价值日志，再对旧历史和长工具结果做结构化摘要。必须保留目标、约束、关键事实、未完成项、来源引用和工具副作用记录，原文留在外部存储以便按需回取。

23. 工具返回过长，或连续调用工具产生大量记录时，怎么处理?

参考答案：先让工具返回结构化摘要和结果引用，只把当前决策所需字段放入上下文。完整结果保存在外部存储，通过结果编号按需读取。裁剪时必须保留错误、关键数值、来源、未完成项和外部副作用，不能只按字符数截断。

24. 追问：压缩时哪些信息必须保留?

参考答案：不要把强约束只写进 Prompt。把必经步骤编译成固定流程和规则检查，声明 必调工具和必交付产物
和前后置条件。结束前由独立 独立校验器 检查证据。未通过应标记为
未完成或失败，进入补偿、重试或人工确认，不能计作成功。

25. 业务场景中的长期记忆如何组织?

参考答案：工作记忆只服务当前步骤，短期记忆覆盖会话，长期记忆保存跨会话且稳定的事实/偏好。写入由策略或抽取器产生候选，再经敏感性、价值、置信度和用户授权过滤。检索结合租户过滤、时间衰减与语义相关度，具体存储可按规模选择关系库、全文或向量索引。

26. 新旧记忆冲突时怎么处理？

参考答案：记忆保存事实、偏好、来源、时间、置信度和版本，不做无脑追加。新旧冲突按来源可靠性、时效和用户确认处理：保留新记录，并把旧记录标记为已被替代，同时保留审计。错误内容支持删除和更正，低价值内容按访问频率、过期时间和收益评分淘汰。硬规则始终高于用户记忆。

27. 【工程编码题】 实现一个简化的 Plan-and-Execute Agent：查询两个城市的天气，并计算温度差。

参考答案：Planner 生成有类型、依赖和输出名的步骤。Executor 按 DAG 执行 weather(cityA)、weather(cityB) 和 calculate_diff，并把结果写入环境。Summarizer
只读取结构化结果生成结论。每步保存状态，可重试但副作用工具需幂等。见附录 D。

28. Planner、Executor 和 Summarizer 各负责什么？

参考答案：计划用结构化 Step 表示：id、type、deps、inputs、结果名称、policy、status。后续步骤只按 结果名称 引用前序结果。Planner 负责分解与重规划，Executor 负责确定性执行和状态落库，Summarizer
只基于已验证结果生成答案，避免三者职责混杂。

29. 步骤怎样表示，后续步骤如何引用前序结果？

参考答案：计划用结构化 Step 表示：id、type、deps、inputs、结果名称、policy、status。后续步骤只按 结果名称 引用前序结果。Planner 负责分解与重规划，Executor 负责确定性执行和状态落库，Summarizer
只基于已验证结果生成答案，避免三者职责混杂。

30. 怎样区分调用工具的步骤和调用模型的步骤？

参考答案：能用明确接口和确定规则完成的动作应定义为工具步骤，输入输出必须结构化。需要理解自然语言、生成内容、判断意图或制定计划的步骤才交给模型。关键业务计算、权限判断和数据写入不应依赖模型自由生成。

31. 工具注册表如何设计？

参考答案：注册表保存稳定 tool_id、版本、JSON 参数格式、权限、超时/重试、幂等性和执行适配器。模型只看到经权限与场景过滤后的短描述。解析后在服务端做参数格式校验、参数规范化、鉴权和风险审批，再执行并返回结构化工具结果（Observation）。用调用正确率、参数合法率和任务完成率评估。

32. 目前其他求职流程进展如何？

参考答案：如实说明当前流程所处阶段和预计决策时间，同时表达对本岗位的兴趣。不要虚构
Offer，也不要用无法核实的信息施压。

33. 对工作地点有什么考虑？

参考答案：按真实情况给明确边界：首选【城市】，可接受【城市/出差频率/迁移时间】。若有家庭或通勤约束，直接说明可协商范围和最早到岗时间。

34. 为什么考虑换工作？

参考答案：用正向动机回答：现岗位让我积累了【能力】，下一步希望在更大规模、更完整责任边界下深耕 Agent 工程化。目标岗位在【业务/技术/团队】上与此匹配。不要抱怨原公司，也不要只说薪资。补充未来 2-3
年希望承担的结果。

二面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

2. AI 应用平台最初要解决什么问题，为什么建设这个平台？

参考答案：建议答法：平台面向【内部开发者/企业客户】，把模型、知识、工具和流程统一成可发布、可治理的 AI 应用。核心解决重复接入、权限审计、长任务可靠性和效果回归。交付物应明确为产品实例、配置或扩展包、接口与运维指标，不能只说“交付一个 Agent”。

3. 与 Dify、Coze、Palantir 等方案相比，借鉴了什么，又有什么不同？

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

4. 是否阅读过相关开源项目的源码？

参考答案：固定关注官方文档、版本说明、核心仓库、技术论文和可靠的工程实践。对重要变化先阅读接口与测试，再用小型任务验证行为、性能和兼容性，最后判断是否影响现有系统。只说明真实阅读和验证过的内容。

5. Workflow 节点非常多时，怎样处理复杂度？

参考答案：先按业务阶段拆成子 Workflow，并明确每个子流程的输入输出和错误边界。公共步骤抽成可复用节点，复杂分支使用子图和命名空间。发布前做静态检查、可达性分析和端到端测试，避免一个巨型流程难以维护。

6. 是否支持通过自然语言生成 Workflow？

参考答案：DSL 先经过语法、参数格式、类型、引用和环路校验，再解析为带版本的 IR，完成默认值、依赖解析和节点绑定，最后编译为 DAG/状态图。自然语言只生成受限 IR 草案，不直接发布最终 DSL。生成后做静态校验、沙箱试跑、测试用例验证和基于错误的有限修复。

7. 为什么不让模型直接生成最终 DSL？

参考答案：DSL 先经过语法、参数格式、类型、引用和环路校验，再解析为带版本的 IR，完成默认值、依赖解析和节点绑定，最后编译为 DAG/状态图。自然语言只生成受限 IR 草案，不直接发布最终 DSL。生成后做静态校验、沙箱试跑、测试用例验证和基于错误的有限修复。

8. 生成后怎样校验节点、参数和连线？

参考答案：先用 DSL 的参数格式校验节点类型、必填字段和参数范围，再检查端口类型、引用是否存在、连线方向、环路和不可达节点。随后做权限、资源和版本校验。只有静态检查和沙箱试运行都通过，才能进入发布流程。

9. 能否自动执行测试，并根据错误修复生成结果？

参考答案：可以，但修复必须受限。系统先在沙箱运行固定测试，根据结构化错误只修改相关节点或参数，并限制修复次数。每次修改都重新执行完整校验和测试。超过次数或涉及权限与业务语义时转人工处理。

10. 工具数量很多时，如何组织和选择？

参考答案：注册表保存稳定 tool_id、版本、JSON 参数格式、权限、超时/重试、幂等性和执行适配器。模型只看到经权限与场景过滤后的短描述。解析后在服务端做参数格式校验、参数规范化、鉴权和风险审批，再执行并返回结构化工具结果（Observation）。用调用正确率、参数合法率和任务完成率评估。

11. 如果是面向 ToC 的 Agent，工具信息如何进入上下文？

参考答案：先按用户权限、设备能力、地区、任务意图和当前阶段过滤工具，再只注入少量候选摘要。模型选择后加载完整说明。个人隐私和危险能力默认不可见，服务端执行前仍需鉴权。

12. 如何通过作用域和渐进式披露限制当前可见能力？

参考答案：先根据租户、身份、场景和当前阶段筛掉无权或无关能力，只向模型展示少量能力摘要。模型选中后再加载完整说明、参数格式和资源。执行前仍由服务端做鉴权，能力可见和允许执行必须分开。

13. 工具描述和历史消息过长时如何压缩？

参考答案：把预算分给系统/策略、当前任务、近期对话、检索证据和工具结果，并预留输出空间。达到 70%-80% 阈值时先删重复与低价值日志，再对旧历史和长工具结果做结构化摘要。必须保留目标、约束、关键事实、未完成项、来源引用和工具副作用记录，原文留在外部存储以便按需回取。

14. 完整的 RAG 链路怎么设计？

参考答案：完整链路是采集与权限校验→解析/OCR→结构化与切分→去重和元数据→Embedding/全文索引→查询理解→权限过滤→混合召回→融合/重排→上下文组装→带引用生成→评测与反馈。跨 Chunk
信息用父子块、窗口扩展、实体/表格关联或多跳检索，权限过滤必须在召回阶段落实。

15. 不同文件类型分别如何处理？

参考答案：先按文件类型选择解析器，并保留标题、页码、表格、图片和层级结构。纯文本按段落处理，PDF 和 PPT 按版面与阅读顺序处理，Word 按标题层级处理，Excel
按工作表、表头和行处理。解析结果使用统一文档格式进入切分和索引。

16. PPL 切分是怎么做的？

参考答案：固定切分稳定便宜，适合结构弱文本。层级切分保留标题/章节关系。语义或 PPL
切分在相邻句条件概率突变处找边界。阈值应在标注集上按文档类型调优，而非全局固定。同时评估
Recall、完整性、Chunk 数、延迟和成本，并为表格/代码使用专门规则。

17. BM25、向量检索、RRF、MMR 和 Rerank 如何组合？

参考答案：BM25 擅长精确词，向量召回语义近邻。两路各取候选后用 RRF 融合，再由 Reranker 精排。MMR 在结果高度重复、需要覆盖多方面时使用，会牺牲部分相关性。代码库常优先
grep/glob/符号索引，长文档可用层级索引。选择应由评测、时延和成本共同决定。

18. 如何评估检索和回答效果？

参考答案：检索看 Recall@K、MRR、NDCG 和权限过滤正确率，回答看忠实性、完整性、引用正确率和人工通过率。两层分开评估，才能判断问题发生在召回、排序还是生成。

19. 如果生成答案和评价答案都依赖同类模型，怎样看待评测偏差？

参考答案：同类模型可能偏好相似表达和推理方式，也可能共同忽略同一类错误。应使用明确评分标准、不同模型交叉评审、人工盲测和一致性统计，并保留无法判定的样本。Judge 只能辅助，不能替代人工校准。

20. 模型生成的评测集可信吗，怎样验证？

参考答案：只能作为候选数据。生成后要检查任务是否真实、答案是否正确、难度是否覆盖、是否泄漏提示词，并由领域人员抽检或复核。最终评测集还要包含真实用户任务和线上失败记录。

21. 在已有平台之外，为什么还需要独立的业务 Harness？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

22. 运行内核、业务扩展与服务入口怎样分工？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

23. 这套运行时能否迁移到其他行业？

参考答案：可复用的是任务生命周期、模型与工具接口、状态保存、权限、观测和评测机制。行业差异放在 Skill、Workflow、数据适配、记忆策略和完成校验中。只要扩展协议稳定，就不应因新增行业而修改核心循环。

24. 不同行业的业务指标不同，通用能力和业务评价怎么分开？

参考答案：通用层评估运行健康、工具正确率、延迟、成本、安全和恢复。业务层评估任务完成、业务规则、产物质量和实际价值。平台提供统一评测框架，各行业提供自己的完成标准和标注数据。

25. 页面关闭或服务重启后，任务如何恢复？

参考答案：以 task_id/run_id 保存不可变输入、当前节点、步骤状态、上下文摘要、工具调用记录、等待事件和版本号。每个状态跃迁在同一事务中写入数据库。恢复时从最后一个已提交边界继续，并以租约/版本校验防止双执行。外部副作用还要靠幂等键、状态查询或补偿，Checkpoint 本身不保证外部操作只执行一次。

26. 如何认领任务，避免重复执行？

参考答案：Worker 通过带过期时间的租约认领任务，控制面发放单调递增的防旧写令牌。所有状态写入和外部资源操作都校验 token/版本并用 CAS（比较并交换） 提交，旧 Worker 即使"复活"也会被拒绝。租约到期后任务才能安全转移。

27. 追问：如果工具会产生外部副作用，怎样处理重复调用？

参考答案：为每次业务操作生成稳定 幂等键，并在工具侧建立唯一约束和结果表。超时且结果未知时先按 key 查询外部系统，不盲目重试。无法查询时把状态置为 UNKNOWN，走对账、补偿或人工确认。Harness 负责传播 key 和策略，业务工具负责真正的原子去重。

28. 从业务模板演进到通用运行时，需要做哪些调整？

参考答案：服务入口处理认证、协议、会话、限流和
SSE。通用运行内核处理状态机、模型/工具循环、Checkpoint、预算和观测。业务扩展提供领域 Skill、Workflow、Memory 策略、数据适配与完成校验。依赖方向指向稳定接口，业务层不反向修改内核。

29. 这套设计迁移到 ToC 场景，会遇到什么问题？

参考答案：ToB 更重租户隔离、审批审计、私有化、领域流程和 SLA。ToC 更重低延迟、个性化 Memory、海量弹性和单位成本。内核可复用，但身份、权限、记忆授权、资源调度和产品交互策略应分别设计，不能简单复制。

30. 为什么考虑离开当前公司？

参考答案：用正向原因回答：当前工作已经形成了哪些积累，下一阶段希望扩大哪类责任或技术深度，以及目标岗位为什么更匹配。不要评价原公司的人和管理，也不要只把薪资作为原因。

31. 其他求职流程进展如何？

参考答案：如实说明当前流程所处阶段和预计决策时间，同时表达对本岗位的兴趣。不要虚构
Offer，也不要用无法核实的信息施压。

四、小红书

方向1一面

1. 平台的定位是什么，核心能力有哪些？

参考答案：建议答法：平台面向【内部开发者/企业客户】，把模型、知识、工具和流程统一成可发布、可治理的 AI 应用。核心解决重复接入、权限审计、长任务可靠性和效果回归。交付物应明确为产品实例、配置或扩展包、接口与运维指标，不能只说“交付一个 Agent”。

2. 客户实际怎样使用这个平台？

参考答案：应按真实情况说明用户是谁、从哪个入口进入、完成什么任务、使用频率如何，以及最终产物进入哪个业务流程。还要区分试用、概念验证、灰度和正式生产，不能只列功能名称。

3. 同时支持 Workflow 和自主 Agent 吗？应用如何发布？

参考答案：可以共用模型、工具、权限、状态和观测能力。Workflow 负责固定、可审计的流程，Agent 负责需要动态决策的局部。应用发布时固定配置与版本，经过测试和审批后再通过 API、SDK 或页面入口提供服务。

4. 平台与现有产品相比，有什么差异化？

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

5. 为什么用 LangGraph，而不是沿用原有后端语言自行实现？

参考答案：选择依据是显式状态图、条件边、Checkpoint/中断恢复、流式事件和可插拔节点，适合长任务与人机协作。把它限定在核心编排层，工具治理、权限、租约、评测和业务扩展由自有层实现，避免深度绑定。同时用同一任务比较自研 Loop、LangGraph 与其他框架的复杂度和运行指标。

6. 多 Agent 采用什么协作模式？

参考答案：常见模式包括主管分派、对等协作和固定流水线。选择取决于任务是否可分解、是否需要并行、结果怎样汇总和权限是否不同。无论哪种模式，都要有任务契约、所有权、结束条件和循环限制。

7. Planner 如何选择和路由到具体 Agent？

参考答案：计划用结构化 Step 表示：id、type、deps、inputs、结果名称、policy、status。后续步骤只按 结果名称 引用前序结果。Planner 负责分解与重规划，Executor 负责确定性执行和状态落库，Summarizer 只基于已验证结果生成答案，避免三者职责混杂。

8. 多个 Agent 能力相似时，会不会选错？

参考答案：会出现误选。先用权限、前置条件、领域和输入格式做硬过滤，再根据能力说明和历史效果排序。低置信度时应请求澄清或使用明确的兜底路由，并通过路由准确率、改派率和最终完成率持续评估。

9. 追问：路由是否正确，怎样评估？

参考答案：离线使用标注任务计算路由准确率、Top-K 命中率和误选率，线上观察改派率、人工纠正率、任务完成率和成本。能力相似或标注有歧义的任务要单独统计，不能只看总体准确率。

10. Agent 之间怎样传递上下文？

参考答案：不要直接复制全部对话。任务交接应传目标、约束、已确认事实、证据引用、已完成步骤、未完成项、预算和期望输出。敏感数据按接收方权限过滤，完整历史保留在共享存储中按需读取。

11. 如果并行执行并写入共享状态，怎么处理冲突？

参考答案：并行分支只写各自的局部状态，汇总节点再合并。必须共享写入时使用版本号和 CAS（比较并交换）检查，发生冲突后按字段合并规则重试。对不可合并的外部操作应串行执行或使用唯一约束。

12. 检索效果用什么指标衡量？

参考答案：检索层用 Recall@K 看相关证据是否被召回，MRR 看首个相关结果位置，NDCG 适合多级相关性和整体排序。回答层再看忠实性、完整性、引用正确率与人工胜率。数据集要覆盖真实查询、难例和时间切片，并报告置信区间与失败样本。

13. 评测数据集怎样构建，预期召回结果由谁确定？

参考答案：用例来自真实任务、边界条件和线上失败记录。每条用例应包含输入、约束、必要证据、完成条件和允许的结果范围。参考轨迹由业务与工程人员共同确认，只规定必须经过和禁止发生的动作，不强制唯一执行路径。

14. 为什么从单一检索演进到混合检索，再加入重排？

参考答案：BM25 擅长精确词，向量召回语义近邻。两路各取候选后用 RRF 融合，再由 Reranker 精排。MMR 在结果高度重复、需要覆盖多方面时使用，会牺牲部分相关性。代码库常优先 grep/glob/符号索引，长文档可用层级索引。选择应由评测、时延和成本共同决定。

15. Query Rewrite 具体做了什么？

参考答案：改写负责消歧、补全指代、抽取实体和生成多路检索式，但必须保留原查询并限制新增事实。离线比较改写前后的 Recall/NDCG，线上看完成率与零结果率。对专有名词被改坏、查询过度扩展等失败例，使用规则保护和回退原查询。

16. 追问：提升怎么测出来的，有没有失败案例？

参考答案：先固定评测对象、样本、指标和统计周期，再比较改动前后的结果。需要同时报告绝对值、变化幅度和样本量，并通过同一批数据回放、分批上线或对照组排除流量、人员经验和任务难度变化的影响。

17. 多模态文档如何解析和切分，对检索有什么影响？

参考答案：先保留文档结构而非只抽纯文本：PDF/PPT 按页、标题、版面和阅读顺序解析，图片走 OCR/视觉模型并关联页码。Excel 按 Sheet、表头、主键和行块处理，大表优先导入结构化存储后用 SQL/聚合检索。表格、图片和正文共享 doc_id/section_id 以支持联合召回。

18. PPL 切分的原理是什么？

参考答案：固定切分稳定便宜，适合结构弱文本。层级切分保留标题/章节关系。语义或 PPL 切分在相邻句条件概率突变处找边界。阈值应在标注集上按文档类型调优，而非全局固定。同时评估 Recall、完整性、Chunk 数、延迟和成本，并为表格/代码使用专门规则。

19. 用什么模型计算，成本如何？

参考答案：说明实际使用的模型类型、部署方式、调用频率和单位文档成本。成本由文档长度、切分候选数、模型价格和是否离线批处理决定。应先用较轻模型完成初筛，只把边界不确定的内容交给高成本模型。

20. 切分阈值是固定的还是动态的？

参考答案：不建议所有文档共用一个固定阈值。阈值应按文档类型、句子长度和模型分布在标注集上确定，并允许在一定范围内动态调整。上线后同时观察召回率、块完整性、块数量、延迟和成本。

21. 长对话中出现偏离当前任务的消息，怎样处理边界？

参考答案：运行时应维护当前任务目标、范围和未完成项。新消息先判断是补充当前任务、修改任务还是开启新任务。与当前目标无关的内容不写入活动上下文，可单独建立任务或只保留在会话历史。任务切换必须得到明确确认。

22. 短期上下文与长期记忆如何区分？

参考答案：工作记忆只服务当前步骤，短期记忆覆盖会话，长期记忆保存跨会话且稳定的事实/偏好。写入由策略或抽取器产生候选，再经敏感性、价值、置信度和用户授权过滤。检索结合租户过滤、时间衰减与语义相关度，具体存储可按规模选择关系库、全文或向量索引。

23. 长期记忆怎样更新？

参考答案：记忆保存事实、偏好、来源、时间、置信度和版本，不做无脑追加。新旧冲突按来源可靠性、时效和用户确认处理：保留新记录，并把旧记录标记为已被替代，同时保留审计。错误内容支持删除和更正，低价值内容按访问频率、过期时间和收益评分淘汰。硬规则始终高于用户记忆。

24. Memory 与固定 Workflow 的关系是什么？

参考答案：Workflow 决定必须执行的步骤和规则，Memory 只提供历史事实与偏好，不能改变强制流程。Workflow 在明确节点读取或写入记忆，写入内容仍需校验和授权。发生冲突时以当前规则和显式用户输入为准。

25. 是否支持 Skill？Memory 与 Skill 的规则冲突时怎么办？

参考答案：记忆保存事实、偏好、来源、时间、置信度和版本，不做无脑追加。新旧冲突按来源可靠性、时效和用户确认处理：保留新记录，并把旧记录标记为已被替代，同时保留审计。错误内容支持删除和更正，低价值内容按访问频率、过期时间和收益评分淘汰。硬规则始终高于用户记忆。

方向1二面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

2. 平台的整体架构和核心模块是什么？

参考答案：可分为控制面（配置、版本、发布、评测）、入口层（鉴权/会话/限流）、运行面（调度、状态机、模型与工具、Checkpoint）、能力层（Skill/MCP/RAG/Memory）和基础设施（队列、存储、沙箱、观测）。请求经入口创建 run，运行时逐步执行并持久化事件，最终输出答案/产物和完整审计记录。

3. 画布 DSL 如何转换成 LangGraph 图？

参考答案：DSL 先经过语法、参数格式、类型、引用和环路校验，再解析为带版本的 IR，完成默认值、依赖解析和节点绑定，最后编译为 DAG/状态图。自然语言只生成受限 IR 草案，不直接发布最终 DSL。生成后做静态校验、沙箱试跑、测试用例验证和基于错误的有限修复。

4. Workflow 与 ReAct 如何复用底层能力？

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

5. 与 OpenClaw 这类 Agent 相比，有什么区别？

参考答案：比较时应分开看模型能力和 Harness 能力。重点比较工具协议、上下文管理、计划与执行方式、代码与终端权限、检查点、人工交互、扩展机制和安全边界。不能只根据一次回答效果判断产品差异。

6. 长期记忆和个人工作空间怎样设计？

参考答案：长期记忆保存跨会话的稳定事实、偏好和来源。个人工作空间保存文件、产物和任务状态。两者都按用户和租户隔离，但使用不同的数据模型、权限和保留策略。记忆可引用工作空间内容，不能复制未授权原文。

7. 业务数据平台中的数据、逻辑和动作如何接入 Agent？

参考答案：数据通过受权限控制的查询接口或 RAG 接入，业务逻辑通过服务或 Workflow 接入，写操作通过有明确参数和审计的工具接入。所有调用携带用户和租户身份，读取与写入分别授权，模型不能直接连接底层数据库。

8. 业务本体由平台统一建设，还是由具体项目提供？

参考答案：平台负责本体规范、标识、版本、查询接口和治理工具，具体项目负责领域实体、关系和业务含义。共性部分可以沉淀到平台，但领域定义必须由业务方确认，避免平台替业务作错误解释。

9. 知识图谱怎样查询，查询结果如何用于任务？

参考答案：先把自然语言转成受限制的查询意图，再由查询服务生成并校验图查询语句。查询结果转成带来源的结构化实体和关系，作为工具结果交给 Agent。权限、查询深度和返回数量必须在服务端限制。

10. RAG 是固定执行链路，还是作为 Tool 让模型决定是否调用？

参考答案：高频且必须基于知识回答的场景适合固定检索链路，保证每次都取证。只有部分请求需要知识库时，可把 RAG 作为工具由模型选择。无论哪种方式，权限过滤、引用和结果长度控制都必须固定执行。

11. 工具返回内容很长时，如何裁剪？

参考答案：先让工具返回结构化摘要和结果引用，只把当前决策所需字段放入上下文。完整结果保存在外部存储，通过结果编号按需读取。裁剪时必须保留错误、关键数值、来源、未完成项和外部副作用，不能只按字符数截断。

12. 使用了哪些文档切分策略，各自适合什么情况？

参考答案：固定切分稳定便宜，适合结构弱文本。层级切分保留标题/章节关系。语义或 PPL 切分在相邻句条件概率突变处找边界。阈值应在标注集上按文档类型调优，而非全局固定。同时评估 Recall、完整性、Chunk 数、延迟和成本，并为表格/代码使用专门规则。

13. PDF、PPT、Excel 分别如何处理？

参考答案：先保留文档结构而非只抽纯文本：PDF/PPT 按页、标题、版面和阅读顺序解析，图片走 OCR/视觉模型并关联页码。Excel 按 Sheet、表头、主键和行块处理，大表优先导入结构化存储后用 SQL/聚合检索。表格、图片和正文共享 doc_id/section_id 以支持联合召回。

14. 追问：图片和表格怎样解析？

参考答案：先保留文档结构而非只抽纯文本：PDF/PPT 按页、标题、版面和阅读顺序解析，图片走 OCR/视觉模型并关联页码。Excel 按 Sheet、表头、主键和行块处理，大表优先导入结构化存储后用 SQL/聚合检索。表格、图片和正文共享 doc_id/section_id 以支持联合召回。

15. 切分粒度与召回效果之间是什么关系？

参考答案：固定切分稳定便宜，适合结构弱文本。层级切分保留标题/章节关系。语义或 PPL 切分在相邻句条件概率突变处找边界。阈值应在标注集上按文档类型调优，而非全局固定。同时评估 Recall、完整性、Chunk 数、延迟和成本，并为表格/代码使用专门规则。

16. 混合检索和重排具体怎么组合？

参考答案：BM25 擅长精确词，向量召回语义近邻。两路各取候选后用 RRF 融合，再由 Reranker 精排。MMR 在结果高度重复、需要覆盖多方面时使用，会牺牲部分相关性。代码库常优先 grep/glob/符号索引，长文档可用层级索引。选择应由评测、时延和成本共同决定。

17. 为什么没有采用 MMR？

参考答案：如果候选重复度不高、重排模型已经能控制多样性，或更关注最高相关性与延迟，就可以不使用 MMR。决定应基于重复率、Recall、NDCG、回答质量和额外成本，而不是因为组件数量少。

18. Workflow 作为 Skill，究竟是什么意思？

参考答案：Skill 包含声明、说明、资源与可执行入口。运行时只把必要说明渐进式注入上下文。Workflow Skill 以受版本管理的确定性流程执行：ReAct 挂起并传入结构化参数，子流程返回结果/错误/产物后主循环恢复。脚本和代码必须经过签名、权限与沙箱策略。

19. Skill、Tool、Workflow 与通用运行内核怎样解耦？

参考答案：Skill 包含声明、说明、资源与可执行入口。运行时只把必要说明渐进式注入上下文。Workflow Skill 以受版本管理的确定性流程执行：ReAct
挂起并传入结构化参数，子流程返回结果/错误/产物后主循环恢复。脚本和代码必须经过签名、权限与沙箱策略。

20. 当前职级和过去的绩效情况如何？

参考答案：如实给出公司职级、对应职责范围和时间线。晋升用"承担范围扩大 + 关键结果 + 组织影响"说明，绩效与薪酬只报可公开且可核验的信息。若体系不可横向比较，可补充带队规模、决策权和交付复杂度。

方向1三面

1. 平台主要解决什么问题，核心指标是什么？

参考答案：先明确平台降低的是接入成本、交付周期、运行风险还是质量波动。核心指标应覆盖应用交付周期、任务完成率、质量通过率、P95 延迟、单位任务成本、故障恢复和实际活跃使用。指标必须对应平台目标。

2. 主要部署在公有云还是客户私有环境？

参考答案：部署时通常把入口服务、任务队列、Worker 池和状态存储分开。入口负责接收请求，队列保存待执行任务，Worker 负责实际运行，数据库或对象存储保存配置、进度和结果。公有云部署更便于弹性扩缩。私有化部署则要确保模型、数据、日志和密钥都留在客户规定的边界内。

3. 当前客户、用户以及 Agent / Workflow 的使用规模如何？

参考答案：分别报告客户数、活跃用户数、已发布 Agent 和 Workflow 数、日任务量、峰值并发和统计周期。还要区分概念验证、灰度和生产，避免把创建数量当成真实使用量。

4. 整体架构怎么设计？

参考答案：可分为控制面（配置、版本、发布、评测）、入口层（鉴权/会话/限流）、运行面（调度、状态机、模型与工具、Checkpoint）、能力层（Skill/MCP/RAG/Memory）和基础设施（队列、存储、沙箱、观测）。请求经入口创建 run，运行时逐步执行并持久化事件，最终输出答案/产物和完整审计记录。

5. 为什么选 LangGraph，与其他框架比较过吗？

参考答案：选择重点是显式状态图、条件分支、检查点、中断恢复和流式事件。比较时使用同一任务评估状态模型、恢复、扩展接口、观测、部署和维护成本。若未完成严格对比，应说明已验证范围和仍未确认的部分。

6. DSL 从加载、编译到执行，完整过程是什么？

参考答案：DSL 先经过语法、参数格式、类型、引用和环路校验，再解析为带版本的 IR，完成默认值、依赖解析和节点绑定，最后编译为 DAG/状态图。自然语言只生成受限 IR 草案，不直接发布最终 DSL。生成后做静态校验、沙箱试跑、测试用例验证和基于错误的有限修复。

7. 节点类型如何组织，不同能力怎么接入？

参考答案：节点统一包含类型、版本、输入输出格式、配置、权限、超时重试、状态和执行器。常见类型包括模型、工具、条件、并行、子流程、人工等待、代码、检索和结果校验。节点定义与运行状态必须分离，多个任务才能安全复用同一张图。

8. 与 Dify、Coze 相比有什么不同？

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

9. Workflow 与 ReAct 怎样共用执行能力？

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

10. Agent 能否作为一个 Workflow 节点？

参考答案：可以。该节点应有明确输入输出、工具和权限范围、预算、超时、检查点和完成条件。Workflow 负责节点前后的确定性控制，Agent 只在节点内部自主决策。失败后按节点策略重试、降级或转人工。

11. Worker、Token 和模型资源如何管理？

参考答案：Worker 按任务类型和资源需求分池，使用队列、并发配额和自动扩缩容。模型 Token 按租户、任务和步骤设置预算，并记录实际用量。模型并发通过网关限流和路由，避免单一租户或长任务占满资源。

12. 项目最大的挑战是什么，你具体贡献了什么？

参考答案：明确说明自己负责的模块、做出的关键决策、亲自完成的代码或方案，以及与其他成员的边界。再说明交付结果和量化指标。团队已有能力、开源框架和个人新增能力要分开描述。

13. 接入 API、MCP、知识库和业务数据分别需要多少开发工作？

参考答案：MCP 提供统一的能力发现、参数格式、调用与传输约定，降低不同 Host 与工具服务的适配成本。Skill 则描述何时用、怎样组合、业务规则和产物要求，两者互补。MCP 的治理价值并非独有，鉴权、审计、限流仍需平台实现。简单稳定 API 可直接接 OpenAPI，跨客户端复用时 MCP 更有价值。

14. 为什么引入 Kafka，同步的是什么数据？

参考答案：Kafka 适合解耦生产与消费、承受流量峰值、保留可重放事件并支持多个下游。应说明同步的是业务事件、状态变化还是数据更新，并明确分区键、顺序要求、幂等消费、积压监控和保留周期。

15. 运行时如何跨客户复用？

参考答案：可复用的是任务生命周期、模型与工具接口、状态保存、权限、观测和评测机制。行业差异放在 Skill、Workflow、数据适配、记忆策略和完成校验中。只要扩展协议稳定，就不应因新增行业而修改核心循环。

16. 业务扩展模块里包含哪些内容？

参考答案：服务入口处理认证、协议、会话、限流和
SSE。通用运行内核处理状态机、模型/工具循环、Checkpoint、预算和观测。业务扩展提供领域 Skill、Workflow、Memory 策略、数据适配与完成校验。依赖方向指向稳定接口，业务层不反向修改内核。

17. 上下文如何分层，各层分别放什么？

参考答案：可分为系统规则、身份与权限、当前任务目标、近期对话、长期记忆、检索证据、工具结果和工作状态。稳定且高优先级的信息放前层，体积大的原文和历史结果放外部存储，通过引用按需加载。

18. 什么时候触发压缩，怎样保留关键内容？

参考答案：把预算分给系统/策略、当前任务、近期对话、检索证据和工具结果，并预留输出空间。达到 70%-80% 阈值时先删重复与低价值日志，再对旧历史和长工具结果做结构化摘要。必须保留目标、约束、关键事实、未完成项、来源引用和工具副作用记录，原文留在外部存储以便按需回取。

19. 当前能力处于 PoC 还是生产使用阶段？

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

20. PoC 验证和最终交付分别由谁负责？

参考答案：概念验证由产品、业务和研发共同确认场景、数据和成功标准，研发完成技术验证。最终交付需要产品确认范围，研发保证功能与性能，测试负责验收，交付和运维负责部署与运行。责任应在项目开始时明确。

21. 长期记忆如何分类、提取和更新？

参考答案：记忆保存事实、偏好、来源、时间、置信度和版本，不做无脑追加。新旧冲突按来源可靠性、时效和用户确认处理：保留新记录，并把旧记录标记为已被替代，同时保留审计。错误内容支持删除和更正，低价值内容按访问频率、过期时间和收益评分淘汰。硬规则始终高于用户记忆。

22. 多 Agent 如何进行委派、隔离和任务交接？

参考答案：用户身份和租户信息作为不可伪造的执行上下文贯穿网关、运行时和工具层。执行层按 RBAC/ABAC 对"主体-资源-动作"再次鉴权。存储按租户编号分区并加行级策略，缓存/队列/工作目录带租户命名空间，密钥用短期凭证。可见性过滤不能替代真正鉴权。

23. 当前职级与入职时相比有什么变化？

参考答案：如实给出公司职级、对应职责范围和时间线。晋升用"承担范围扩大 + 关键结果 + 组织影响"说明，绩效与薪酬只报可公开且可核验的信息。若体系不可横向比较，可补充带队规模、决策权和交付复杂度。

24. 为什么考虑换工作，又为什么选择这个方向？

参考答案：用正向动机回答：现岗位让我积累了【能力】，下一步希望在更大规模、更完整责任边界下深耕 Agent 工程化。目标岗位在【业务/技术/团队】上与此匹配。不要抱怨原公司，也不要只说薪资。补充未来 2-3 年希望承担的结果。

方向2一面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

2. 平台是基于 LangGraph 建设的吗？

参考答案：应明确回答使用范围。可以说明 LangGraph 只承担核心状态图和节点调度，平台的发布、权限、模型网关、工具执行、任务租约、评测和多租户能力由其他模块承担。不能把整个平台简单说成基于一个框架。

3. 在框架之上，自己补充了哪些能力？

参考答案：通常需要自行实现模型网关、工具注册与鉴权、Skill 加载、租约和幂等、任务恢复、SSE 事件、上下文管理、评测、租户隔离和沙箱。回答时要区分框架原生能力、封装能力和团队新增能力。

4. 节点和 DSL 怎样转换为可执行图？

参考答案：DSL 先经过语法、参数格式、类型、引用和环路校验，再解析为带版本的 IR，完成默认值、依赖解析和节点绑定，最后编译为 DAG/状态图。自然语言只生成受限 IR 草案，不直接发布最终 DSL。生成后做静态校验、沙箱试跑、测试用例验证和基于错误的有限修复。

5. 文件处理、RAG 和评测能力分别做了哪些扩展？

参考答案：完整链路是采集与权限校验→解析/OCR→结构化与切分→去重和元数据→Embedding/全文索引→查询理解→权限过滤→混合召回→融合/重排→上下文组装→带引用生成→评测与反馈。跨 Chunk 信息用父子块、窗口扩展、实体/表格关联或多跳检索，权限过滤必须在召回阶段落实。

6. 为什么不直接使用 Dify？

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

7. 平台的定位和能力边界与已有产品有什么不同？

参考答案：建议答法：平台面向【内部开发者/企业客户】，把模型、知识、工具和流程统一成可发布、可治理的 AI 应用。核心解决重复接入、权限审计、长任务可靠性和效果回归。交付物应明确为产品实例、配置或扩展包、接口与运维指标，不能只说“交付一个 Agent”。

8. 目前有哪些真实业务在使用？

参考答案：应按真实情况说明用户是谁、从哪个入口进入、完成什么任务、使用频率如何，以及最终产物进入哪个业务流程。还要区分试用、概念验证、灰度和正式生产，不能只列功能名称。

9. 平台面向 ToB 还是 ToC？

参考答案：ToB 更重租户隔离、审批审计、私有化、领域流程和 SLA。ToC 更重低延迟、个性化 Memory、海量弹性和单位成本。内核可复用，但身份、权限、记忆授权、资源调度和产品交互策略应分别设计，不能简单复制。

10. Agent 生态变化很快，这些变化对平台有什么影响？

参考答案：演进重点应放在稳定接口和可替换能力：模型、框架、工具协议和记忆实现都通过适配层接入。内核保持任务状态、权限、恢复、观测和评测的稳定，新的模型或框架先经过兼容性和回归评测，再逐步替换。

11. 平台更偏用于配置和管理的管理台，还是直接承接在线业务流量的运行台？

参考答案：控制面负责配置、版本、权限、发布、评测和资源策略，不处理高频任务执行。运行面负责接收请求、调度 Worker、调用模型和工具、保存状态并输出事件。两者通过带版本的配置和发布记录连接，运行面不能直接读取未发布草稿。

12. 配置控制面与运行执行面如何划分？

参考答案：控制面负责配置、版本、权限、发布、评测和资源策略，不处理高频任务执行。运行面负责接收请求、调度 Worker、调用模型和工具、保存状态并输出事件。两者通过带版本的配置和发布记录连接，运行面不能直接读取未发布草稿。

13. 开放自治 Agent 与固定 Workflow 应怎样取舍？

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

14. 哪些具体业务适合采用 Agent？

参考答案：适合目标明确但执行路径需要根据上下文动态决定、需要调用多种工具、允许一定探索且结果可以校验的任务。规则固定、低延迟、强事务或错误代价极高的流程更适合普通服务或 Workflow。

15. 公司内部是否已有相关 SDK 或托管产品，为什么仍需要建设这套能力？

参考答案：先判断现有能力是否满足数据边界、权限、长任务恢复、领域流程、部署方式和成本要求。只有存在持续且无法通过扩展解决的缺口时才建设新系统。新能力还必须说明与现有 SDK 或托管产品的分工，避免重复建设。

16. 如果模型跳过权限判断或审批节点，仅靠 Skill 描述能否保证强约束流程？

参考答案：不要把强约束只写进 Prompt。把必经步骤编译成固定流程和规则检查，声明 必调工具和必交付产物 和前后置条件。结束前由独立 独立校验器 检查证据。未通过应标记为 未完成或失败，进入补偿、重试或人工确认，不能计作成功。

17. 必需的工具没有调用、必需的产物没有生成时，怎样判断任务是否完成？

参考答案：不要把强约束只写进 Prompt。把必经步骤编译成固定流程和规则检查，声明 必调工具和必交付产物 和前后置条件。结束前由独立 独立校验器 检查证据。未通过应标记为 未完成或失败，进入补偿、重试或人工确认，不能计作成功。

18. 追问：运行结束时校验不通过，后续怎么处理，是否计为成功？

参考答案：不要把强约束只写进 Prompt。把必经步骤编译成固定流程和规则检查，声明 必调工具和必交付产物 和前后置条件。结束前由独立 独立校验器 检查证据。未通过应标记为 未完成或失败，进入补偿、重试或人工确认，不能计作成功。

19. 对工作地点是否有要求？

参考答案：按真实情况给明确边界：首选【城市】，可接受【城市/出差频率/迁移时间】。若有家庭或通勤约束，直接说明可协商范围和最早到岗时间。

方向2二面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

2. 挑一个最有代表性的项目介绍一下，为什么选择它？

参考答案：按"问题—目标—方案—个人职责—结果"组织。先说明项目解决什么问题和为什么值得做，再讲核心架构与关键难点，最后给出上线范围、效果、稳定性或交付周期。选择能体现岗位能力且个人贡献清楚的项目。

3. 运行内核、业务扩展和服务入口如何分层？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

4. Durable Checkpoint 具体保存了什么？

参考答案：以 task_id/run_id 保存不可变输入、当前节点、步骤状态、上下文摘要、工具调用记录、等待事件和版本号。每个状态跃迁在同一事务中写入数据库。恢复时从最后一个已提交边界继续，并以租约/版本校验防止双执行。外部副作用还要靠幂等键、状态查询或补偿，Checkpoint 本身不保证外部操作只执行一次。

5. 这部分是自行实现的，还是直接使用 LangGraph 的快照？

参考答案：LangGraph 快照可以保存图内状态，但生产恢复还需要任务所有权、租约、外部工具记录、等待事件、业务版本和审计信息。可以复用快照保存图状态，同时由自有持久化层管理任务级元数据和恢复规则。

6. 为什么只用 LangGraph 承载核心循环，而不是把全部运行机制交给它？

参考答案：应把 LangGraph 限定在状态图和节点调度层。任务存储、租约、权限、工具执行、事件协议、评测和业务扩展通过自有接口实现。这样替换框架时主要重写图适配层和检查点转换，不会影响对外接口与业务能力。

7. 工具调用怎样保证幂等？

参考答案：为每次业务操作生成稳定 幂等键，并在工具侧建立唯一约束和结果表。超时且结果未知时先按 key 查询外部系统，不盲目重试。无法查询时把状态置为 UNKNOWN，走对账、补偿或人工确认。Harness 负责传播 key 和策略，业务工具负责真正的原子去重。

8. 追问：调用时连接断了，但不知道外部操作是否成功，这种情况怎么办？

参考答案：为每次业务操作生成稳定 幂等键，并在工具侧建立唯一约束和结果表。超时且结果未知时先按 key 查询外部系统，不盲目重试。无法查询时把状态置为 UNKNOWN，走对账、补偿或人工确认。Harness 负责传播 key 和策略，业务工具负责真正的原子去重。

9. 幂等和可靠性由 Harness 保证，还是由具体业务工具保证？

参考答案：为每次业务操作生成稳定 幂等键，并在工具侧建立唯一约束和结果表。超时且结果未知时先按 key 查询外部系统，不盲目重试。无法查询时把状态置为 UNKNOWN，走对账、补偿或人工确认。Harness 负责传播 key 和策略，业务工具负责真正的原子去重。

10. 运行租约和 Fencing Token 分别解决什么问题？

参考答案：Worker 通过带过期时间的租约认领任务，控制面发放单调递增的防旧写令牌。所有状态写入和外部资源操作都校验 token/版本并用 CAS（比较并交换） 提交，旧 Worker 即使"复活"也会被拒绝。租约到期后任务才能安全转移。

11. Token 为什么需要单调递增，由谁发放？

参考答案：这里的 Token 是防旧写令牌，不是模型Token。每次任务所有权变化时由统一存储通过原子操作发放更大的编号。资源和状态写入必须携带并校验该编号，旧 Worker 即使恢复也会因编号较小而被拒绝。

12. Worker 写状态时怎样校验 Token，是否使用 CAS？

参考答案：Worker 通过带过期时间的租约认领任务，控制面发放单调递增的防旧写令牌。所有状态写入和外部资源操作都校验 token/版本并用 CAS（比较并交换） 提交，旧 Worker 即使"复活"也会被拒绝。租约到期后任务才能安全转移。

13. 平台的节点抽象是什么样的？

参考答案：节点统一包含类型、版本、输入输出格式、配置、权限、超时重试、状态和执行器。常见类型包括模型、工具、条件、并行、子流程、人工等待、代码、检索和结果校验。节点定义与运行状态必须分离，多个任务才能安全复用同一张图。

14. 举一个比较复杂的节点，说明它如何执行。

参考答案：回答时选择自己真实实现过的复杂节点，说明输入校验、权限检查、执行步骤、状态保存、超时重试、输出格式和错误处理。重点是节点如何与运行时交互，而不是只描述内部业务逻辑。

15. 业务数据、逻辑和动作如何接入 Workflow 或 Agent？

参考答案：业务数据通过带权限的查询接口或 RAG 接入，业务逻辑封装为服务或Workflow，写操作封装为经过审计的工具。Workflow 可确定性调用这些能力，Agent 只能从授权候选中选择。模型不能直接连接底层数据库。

16. 多个任务共享同一张执行图时，运行状态怎样隔离？

参考答案：用户身份和租户信息作为不可伪造的执行上下文贯穿网关、运行时和工具层。执行层按 RBAC/ABAC 对"主体-资源-动作"再次鉴权。存储按租户编号分区并加行级策略，缓存/队列/工作目录带租户命名空间，密钥用短期凭证。可见性过滤不能替代真正鉴权。

17. 动态 DSL 每次编译为图，会不会影响性能？

参考答案：会有开销，但通常可通过按 DSL 版本缓存编译结果解决。发布时完成语法、类型、依赖和权限校验，运行时直接加载不可变执行图。只有配置版本变化才重新编译，并记录编译耗时和缓存命中率。

18. Agent 如何持续做评测？

参考答案：每次修改模型、Prompt、工具、检索或流程后自动运行固定回归集，并与当前线上版本比较。未达到质量、安全、延迟和成本阈值时禁止发布。上线后抽样真实任务进入评测集，新增失败记录后再进行版本化回归。

19. 测试用例、参考轨迹、任务完成度和 Judge 怎么配合？

参考答案：把评测拆成任务完成、答案质量、轨迹合规、延迟/成本和安全。参考标准（Ground Truth）由真实任务抽样、专家标注和生产失败样本构成，保存期望结果及允许的多种轨迹。LLM Judge 用明确的评分标准、盲测和多 Judge/人工抽检校准，不能只让同类模型自评。发布前回归，线上灰度 A/B 并设回滚阈值。

20. 如果主 Agent 跳过了必须执行的规则检查，却直接完成发布，怎么发现和阻止？

参考答案：不要把强约束只写进 Prompt。把必经步骤编译成固定流程和规则检查，声明 必调工具和必交付产物 和前后置条件。结束前由独立 独立校验器 检查证据。未通过应标记为 未完成或失败，进入补偿、重试或人工确认，不能计作成功。

21. 如何声明必须调用的工具、必须生成的产物？

参考答案：不要把强约束只写进 Prompt。把必经步骤编译成固定流程和规则检查，声明 必调工具和必交付产物 和前后置条件。结束前由独立 独立校验器 检查证据。未通过应标记为 未完成或失败，进入补偿、重试或人工确认，不能计作成功。

22. 最后的校验没通过时怎么处理？

参考答案：不能把任务标记为成功。先判断缺失的是可自动补做的步骤、可重试的工具，还是需要用户确认的信息。系统可有限次数重新规划或补执行，仍未通过则返回未完成状态、原因和可恢复位置。

23. 将 Workflow 封装为 Skill，会不会限制 Agent 自由度，又带来什么成本？

参考答案：Skill 包含声明、说明、资源与可执行入口。运行时只把必要说明渐进式注入上下文。Workflow Skill 以受版本管理的确定性流程执行：ReAct 挂起并传入结构化参数，子流程返回结果/错误/产物后主循环恢复。脚本和代码必须经过签名、权限与沙箱策略。

24. 大量运行中任务需要迁移时，怎样处理？

参考答案：先停止向旧 Worker 分配新任务，再让安全点上的任务保存检查点并释放租约。无法及时停下的任务等待租约过期，由新 Worker 按版本接管。迁移过程要记录任务清单、状态和校验结果，并支持分批执行和回滚。

25. 追问：如何避免任务丢失或一直卡住？

参考答案：任务状态必须持久化，队列投递和状态写入要可对账。为 RUNNING、WAITING 和 RETRYING 状态设置超时扫描，租约过期后重新认领，超过重试上限进入人工处理。监控任务年龄、租约、队列积压和等待事件是否到达。

26. 如果同时有独立复现和团队项目，如何区分各自覆盖的能力与个人贡献？

参考答案：把内容分成三部分：独立复现验证了哪些机制，团队项目交付了哪些生产能力，个人在团队项目中负责哪些决策和实现。分别说明代码来源、运行环境、数据规模和验证结果，不能把复现结果当成团队生产成果。

27. 如何比较 LangGraph、AgentScope、Google ADK 等框架？

参考答案：不要只列功能。用同一组任务比较状态模型、持久化恢复、多 Agent 协议、工具生态、可观测性、部署复杂度、扩展边界和社区成熟度，再做最小概念验证（PoC）。最终选择取决于团队语言栈、故障模型和交付形态，并保留适配层降低替换成本。

五、美团

一面

1. 选一个最有代表性的项目，说明解决的问题、业务价值和个人职责。

参考答案：明确说明自己负责的模块、做出的关键决策、亲自完成的代码或方案，以及与其他成员的边界。再说明交付结果和量化指标。团队已有能力、开源框架和个人新增能力要分开描述。

2. 为什么划分为通用运行内核、业务扩展和服务入口？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

3. 业务扩展模块与 Skill 是什么关系？

参考答案：Tool 是原子能力，Skill 是“何时以及怎样使用一组能力”的可复用方法，Workflow
是确定性的步骤编排。把变化快、强业务语义的 Skill
放在扩展层，内核只负责发现、加载、权限、执行与观测接口，以实现版本化和不改内核扩展。

4. 除了声明文件或 YAML 配置，还包含哪些真正的运行逻辑？

参考答案：业务扩展还应包含工具适配、Workflow、数据转换、权限规则、完成校验、错误处理、测试和观测逻辑。声明文件只负责描述入口、版本、依赖和配置，真正的执行逻辑必须在受版本管理和测试的代码或流程中。

5. 用代码实现的 Workflow，怎样装配进运行时？

参考答案：Workflow 通过统一接口声明标识、版本、输入输出、依赖、权限和执行入口，发布后注册到能力目录。运行时按固定版本加载，在独立状态空间执行，并通过标准事件、检查点和结果格式与主循环连接。

6. Tool 和 Skill 数量增加后，上下文怎么控制？

参考答案：把预算分给系统/策略、当前任务、近期对话、检索证据和工具结果，并预留输出空间。达到 70%-80% 阈值时先删重复与低价值日志，再对旧历史和长工具结果做结构化摘要。必须保留目标、约束、关键事实、未完成项、来源引用和工具副作用记录，原文留在外部存储以便按需回取。

7. 不同业务域的 Sub-agent，能力和权限怎样划分？

参考答案：用户身份和租户信息作为不可伪造的执行上下文贯穿网关、运行时和工具层。执行层按 RBAC/ABAC 对“主体-资源-动作”再次鉴权。存储按租户编号分区并加行级策略，缓存/队列/工作目录带租户命名空间，密钥用短期凭证。可见性过滤不能替代真正鉴权。

8. 渐进式披露具体怎样实现？

参考答案：先根据租户、身份、场景和当前阶段筛掉无权或无关能力，只向模型展示少量能力摘要。模型选中后再加载完整说明、参数格式和资源。执行前仍由服务端做鉴权，能力可见和允许执行必须分开。

9. 这些设计与标准 ReAct Loop 或现有开源 Agent 有什么不同？

参考答案：核心循环是 Thought/Decision→Action→工具结果（Observation）→更新状态，直到 Final 或触发预算/取消/错误边界。生产实现还必须加入工具鉴权、参数校验、超时、幂等、Checkpoint、上下文预算和可观测事件。循环迟迟不结束时用最大轮次、重复调用检测和无进展判定收敛。

10. 你主要负责运行稳定性，还是也负责最终输出质量？

参考答案：同一模型和参数通常近似稳定，但服务版本、硬件、浮点计算、批处理和预处理变化都可能造成漂移。固定模型版本与归一化流程，重复请求统计向量余弦差/排序一致性，并对 Reranker 做 JSON 参数格式、分数范围、候选 ID 白名单和回退排序校验。

11. Workflow 什么时候接管执行，执行后如何归还控制权？

参考答案：当命中固定业务规则、模型明确选择对应 Skill，或进入必须执行的阶段时，主循环保存状态并把控制权交给 Workflow。Workflow 完成、失败或等待后返回结构化结果与状态，主循环再决定继续推理、补救还是结束。

12. 触发条件是固定规则，还是由模型决定？

参考答案：两种方式可以并存。权限、审批、强业务条件和明确入口使用固定规则。语义意图和开放任务可由模型判断。模型判断必须返回置信度并受候选范围限制，低置信度时要求澄清或使用规则兜底。

13. 自然语言输入、页面操作和显式选择 Skill，是否走同一条链路？

参考答案：为 Skill 建结构化清单：名称、意图、前置条件、输入输出、权限、成本和版本。先按租户/权限/场景做确定性过滤，再用关键词+Embedding 召回少量候选，必要时让路由模型重排。离线用命中率/误选率，线上用完成率、纠正率和成本评估，并支持显式选择兜底。

14. 必须执行某个 Skill 时，怎样保证模型不会选错或跳过？

参考答案：不要把强约束只写进 Prompt。把必经步骤编译成固定流程和规则检查，声明 必调工具和必交付产物 和前后置条件。结束前由独立 独立校验器 检查证据。未通过应标记为 未完成或失败，进入补偿、重试或人工确认，不能计作成功。

15. 没有调用必需工具时，系统能否发现和补救？

参考答案：系统必须能发现。任务定义应声明必调工具、必需证据和必交付产物，结束前由独立校验器检查轨迹。缺失时可有限次数补执行或重新规划，仍未满足则标记未完成，不能返回成功。

16. 举一个真实 Bad Case，说明原因、修复过程和结果。

参考答案：选择一个真实且影响明确的问题，依次说明现象、影响范围、定位过程、根因、修复方法和验证结果。还要说明后续增加了哪些监控、测试或防护，避免问题再次发生。

17. 追问：除了修改 Prompt，还采取了什么工程措施？

参考答案：还应从工具参数格式、权限、路由、完成校验、上下文裁剪、超时重试、幂等、监控和回归测试处理。Prompt 只能影响模型倾向，不能替代确定性的安全和业务规则。

18. 跨多轮对话使用同一个 Skill，会不会反复注入相同内容？

参考答案：Tool 是原子能力，Skill 是“何时以及怎样使用一组能力”的可复用方法，Workflow 是确定性的步骤编排。把变化快、强业务语义的 Skill 放在扩展层，内核只负责发现、加载、权限、执行与观测接口，以实现版本化和不改内核扩展。

19. 哪些消息会持久化到历史记录里？

参考答案：保存用户可见消息、最终答案、关键工具结果摘要、产物引用和影响恢复的状态。临时推理内容、重复日志、密钥、敏感原文和仅用于当前步骤的大结果不应直接进入对话历史。审计记录与用户历史应分开存储。

20. 追问：不希望保存的内容在哪一步过滤，数据模型怎样区分？

参考答案：在写入持久化存储前统一过滤，而不是在读取时临时隐藏。数据模型应标记消息类型、可见范围、保留期限、敏感级别和是否可用于模型上下文。敏感字段先脱敏，再进入日志或审计系统。

21. 创建一个结果与修改已有结果，是否分别处理？

参考答案：应分别处理。创建需要生成新的产物编号、结构和版本。修改必须指定目标产物与基础版本，计算差异并做并发检查。修改失败不能覆盖原版本，必要时支持撤销和版本回退。

22. 用户提出超出能力范围的请求时，系统怎么回应？

参考答案：系统应明确说明无法完成的部分、原因和现有能力边界，并停止高风险或无依据的操作。可以返回仍能完成的部分和需要补充的信息，但不能假装已经完成。

23. 普通文本回答与结构化 Artifact 如何区分？

参考答案：文本回答是一次会话输出，主要用于解释和确认。Artifact 是有类型、标识、版本、字段格式和生命周期的持久化产物，可继续编辑、下载或被其他流程引用。两者应使用不同的数据模型和更新接口。

24. 如果用户要求修改 Artifact 中不支持的字段或布局，怎么办？

参考答案：先根据 Artifact 的能力格式检查请求。支持的部分可以修改，不支持的部分要明确说明限制，并返回可接受的字段或布局范围。不能让模型直接写入未定义字段，也不能静默丢弃用户要求。

25. 多租户隔离在哪些层实现？

参考答案：用户身份和租户信息作为不可伪造的执行上下文贯穿网关、运行时和工具层。执行层按 RBAC/ABAC 对“主体-资源-动作”再次鉴权。存储按租户编号分区并加行级策略，缓存/队列/工作目录带租户命名空间，密钥用短期凭证。可见性过滤不能替代真正鉴权。

26. 沙箱解决什么问题？

参考答案：执行放到短生命周期、无特权的隔离环境：只读基础镜像、独立用户和命名空间、CPU/内存/进程数/时长配额、临时工作目录和默认断网。文件与网络按白名单开放。调度器在沙箱外做鉴权、审计和 Kill，死循环由超时与资源配额终止。

27. 工具幂等与分布式锁是什么关系？

参考答案：为每次业务操作生成稳定 幂等键，并在工具侧建立唯一约束和结果表。超时且结果未知时先按 key 查询外部系统，不盲目重试。无法查询时把状态置为 UNKNOWN，走对账、补偿或人工确认。Harness 负责传播 key 和策略，业务工具负责真正的原子去重。

28. 超时、重试、降级和错误返回怎样处理？

参考答案：先按错误类型决定是否重试：参数和权限错误不重试，网络和限流错误可退避重试，副作用未知先查询状态。超过上限后按策略切换备用模型或工具，无法降级时返回统一错误码、可恢复状态和调用编号。

29. 每次任务是否都有独立工作空间？

参考答案：每个任务至少应有独立的逻辑工作空间和访问令牌。高风险代码或文件任务使用独立容器或临时目录。低风险只读任务可共享底层存储，但路径、权限、缓存和清理必须按任务与租户隔离。

30. Agent 从一个业务域访问另一个业务域时，权限怎样控制？

参考答案：用户身份和租户信息作为不可伪造的执行上下文贯穿网关、运行时和工具层。执行层按 RBAC/ABAC 对"主体-资源-动作"再次鉴权。存储按租户编号分区并加行级策略，缓存/队列/工作目录带租户命名空间，密钥用短期凭证。可见性过滤不能替代真正鉴权。

31. 当前实际使用规模如何？

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

32. 为什么不直接用通用 Agent，加行业专家和同一套 MCP 工具？

参考答案：MCP 提供统一的能力发现、参数格式、调用与传输约定，降低不同 Host 与工具服务的适配成本。Skill 则描述何时用、怎样组合、业务规则和产物要求，两者互补。MCP 的治理价值并非独有，鉴权、审计、限流仍需平台实现。简单稳定 API 可直接接 OpenAPI，跨客户端复用时 MCP 更有价值。

33. 追问：在相同模型、工具和任务条件下，自研方案到底好在哪里？

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

34. 如果通用 Agent 效果更好，客户为什么还要为自研方案付费？

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

35. 怎样避免最终变成高成本的定制开发？

参考答案：先限定产品边界，再把需求分为通用能力、可配置差异和客户专有逻辑。通用能力进入平台，可配置差异进入模板或插件，专有逻辑由项目层维护。交付前评估复用率、维护责任和升级成本，不能让客户代码进入核心内核。

36. 探索型任务和稳定的生产任务，是否应该采用不同方案？

参考答案：应采用不同方案。探索型任务可以保留较高自主性和较宽工具范围，但需要预算和安全边界。稳定生产任务应使用 Workflow、强校验、固定权限和明确回滚。两者可以共享工具和运行基础设施，但控制方式不同。

37. 产品或解决方案角色如何参与定义价值，而不是只由研发判断？

参考答案：产品或解决方案人员负责确认目标用户、业务流程、成功标准和付费价值，研发负责可行性、成本、风险和指标实现。立项、评测和验收都应由双方共同完成，不能只用技术指标判断价值。

六、拼多多

一面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

2. 项目是否已经落地，实际使用规模如何？

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

3. 怎样区分 Agent Harness 与 Agent Framework？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

4. 运行内核包含哪些主要模块？

参考答案：核心模块通常包括任务状态机、模型调用、工具执行、上下文预算、检查点与恢复、取消与超时、租约、事件输出、观测和扩展接口。权限、租户和资源限制贯穿各模块。具体业务流程与数据适配不应放入内核。

5. 项目主要面向 ToB 还是 ToC？

参考答案：ToB 更重租户隔离、审批审计、私有化、领域流程和 SLA。ToC 更重低延迟、个性化 Memory、海量弹性和单位成本。内核可复用，但身份、权限、记忆授权、资源调度和产品交互策略应分别设计，不能简单复制。

6. 通用运行层与业务能力如何分层？

参考答案：服务入口处理认证、协议、会话、限流和 SSE。通用运行内核处理状态机、模型/工具循环、Checkpoint、预算和观测。业务扩展提供领域 Skill、Workflow、Memory 策略、数据适配与完成校验。依赖方向指向稳定接口，业务层不反向修改内核。

7. 多个任务共享 Worker 时，状态怎么管理？

参考答案：Worker 不保存任务私有状态，只在执行期间持有带版本的状态副本。任务状态按 run_id 持久化到外部存储，写入时校验租约和版本。任务切换时清理临时上下文、文件句柄和凭证，防止状态串扰。

8. 为什么同一时刻只激活一个 Skill？

参考答案：为 Skill 建结构化清单：名称、意图、前置条件、输入输出、权限、成本和版本。先按租户/权限/场景做确定性过滤，再用关键词+Embedding 召回少量候选，必要时让路由模型重排。离线用命中率/误选率，线上用完成率、纠正率和成本评估，并支持显式选择兜底。

9. Workflow 与 ReAct 如何衔接？

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

10. 不同行业的业务差异怎样处理，能否不修改内核？

参考答案：可复用的是任务生命周期、模型与工具接口、状态保存、权限、观测和评测机制。行业差异放在 Skill、Workflow、数据适配、记忆策略和完成校验中。只要扩展协议稳定，就不应因新增行业而修改核心循环。

11. 共用工作空间会不会导致文件或数据串到其他任务？

参考答案：每个任务至少应有独立的逻辑工作空间和访问令牌。高风险代码或文件任务使用独立容器或临时目录。低风险只读任务可共享底层存储，但路径、权限、缓存和清理必须按任务与租户隔离。

12. 沙箱如何设计？

参考答案：执行放到短生命周期、无特权的隔离环境：只读基础镜像、独立用户和命名空间、CPU/内存/进程数/时长配额、临时工作目录和默认断网。文件与网络按白名单开放。调度器在沙箱外做鉴权、审计和 Kill，死循环由超时与资源配额终止。

13. Docker、Kubernetes 或 Serverless 分别适合什么情况？

参考答案：部署时通常把入口服务、任务队列、Worker 池和状态存储分开。入口负责接收请求，队列保存待执行任务，Worker 负责实际运行，数据库或对象存储保存配置、进度和结果。公有云部署更便于弹性扩缩。私有化部署则要确保模型、数据、日志和密钥都留在客户规定的边界内。

14. 文件、网络和系统权限如何限制？

参考答案：用户身份和租户信息作为不可伪造的执行上下文贯穿网关、运行时和工具层。执行层按 RBAC/ABAC 对"主体-资源-动作"再次鉴权。存储按租户编号分区并加行级策略，缓存/队列/工作目录带租户命名空间，密钥用短期凭证。可见性过滤不能替代真正鉴权。

15. Skill 选择正确与否，如何评估？

参考答案：为 Skill 建结构化清单：名称、意图、前置条件、输入输出、权限、成本和版本。先按租户/权限/场景做确定性过滤，再用关键词+Embedding 召回少量候选，必要时让路由模型重排。离线用命中率/误选率，线上用完成率、纠正率和成本评估，并支持显式选择兜底。

16. 主 Agent、专用 Agent 和 Sub-agent 如何配置？

参考答案：主 Agent 负责目标、分解和汇总。专用 Agent 绑定稳定领域能力。Sub-agent 是一次任务中的受限子任务。三者分别配置工具、权限、模型、预算、上下文和输出格式，并限制调用深度与并发数。

17. Memory 由谁决定写入，什么时候写？

参考答案：工作记忆只服务当前步骤，短期记忆覆盖会话，长期记忆保存跨会话且稳定的事实/偏好。写入由策略或抽取器产生候选，再经敏感性、价值、置信度和用户授权过滤。检索结合租户过滤、时间衰减与语义相关度，具体存储可按规模选择关系库、全文或向量索引。

18. 记忆存在哪里，怎样向量化和检索？

参考答案：工作记忆只服务当前步骤，短期记忆覆盖会话，长期记忆保存跨会话且稳定的事实/偏好。写入由策略或抽取器产生候选，再经敏感性、价值、置信度和用户授权过滤。检索结合租户过滤、时间衰减与语义相关度，具体存储可按规模选择关系库、全文或向量索引。

19. 记忆只是不断追加吗，怎样更新和删除？

参考答案：记忆保存事实、偏好、来源、时间、置信度和版本，不做无脑追加。新旧冲突按来源可靠性、时效和用户确认处理：保留新记录，并把旧记录标记为已被替代，同时保留审计。错误内容支持删除和更正，低价值内容按访问频率、过期时间和收益评分淘汰。硬规则始终高于用户记忆。

20. 新旧记忆发生冲突怎么办？

参考答案：记忆保存事实、偏好、来源、时间、置信度和版本，不做无脑追加。新旧冲突按来源可靠性、时效和用户确认处理：保留新记录，并把旧记录标记为已被替代，同时保留审计。错误内容支持删除和更正，低价值内容按访问频率、过期时间和收益评分淘汰。硬规则始终高于用户记忆。

21. 低价值记忆怎样遗忘？

参考答案：记忆保存事实、偏好、来源、时间、置信度和版本，不做无脑追加。新旧冲突按来源可靠性、时效和用户确认处理：保留新记录，并把旧记录标记为已被替代，同时保留审计。错误内容支持删除和更正，低价值内容按访问频率、过期时间和收益评分淘汰。硬规则始终高于用户记忆。

22. 追问：如果存进去的是错误记忆，怎样纠正？

参考答案：记忆保存事实、偏好、来源、时间、置信度和版本，不做无脑追加。新旧冲突按来源可靠性、时效和用户确认处理：保留新记录，并把旧记录标记为已被替代，同时保留审计。错误内容支持删除和更正，低价值内容按访问频率、过期时间和收益评分淘汰。硬规则始终高于用户记忆。

23. 工作记忆、短期记忆和长期记忆如何区分？

参考答案：工作记忆只服务当前步骤，短期记忆覆盖会话，长期记忆保存跨会话且稳定的事实/偏好。写入由策略或抽取器产生候选，再经敏感性、价值、置信度和用户授权过滤。检索结合租户过滤、时间衰减与语义相关度，具体存储可按规模选择关系库、全文或向量索引。

24. 多 Agent 相互交接时，会不会出现循环调用？

参考答案：会。系统应记录调用图和任务来源，禁止把任务交回已在当前路径中的
Agent，并限制最大深度、总次数和预算。每次交接还要检查是否产生新信息，无进展时立即停止或回到主 Agent。

25. Handoff 应传哪些结构化信息？

参考答案：应传目标、输入、约束、已确认事实、证据引用、已完成步骤、未完成项、权限、预算、截止时间和期望输出格式。不要复制全部对话，敏感内容按接收方权限过滤。

26. 进程重启后，Checkpoint 如何恢复？

参考答案：以 task_id/run_id 保存不可变输入、当前节点、步骤状态、上下文摘要、工具调用记录、等待事件和版本号。每个状态跃迁在同一事务中写入数据库。恢复时从最后一个已提交边界继续，并以租约/版本校验防止双执行。外部副作用还要靠幂等键、状态查询或补偿，Checkpoint 本身不保证外部操作只执行一次。

27. 追问：父任务和子任务存在依赖时，怎样恢复它们的关系和执行顺序？

参考答案：父子任务关系、依赖边、输入输出引用和各自状态必须持久化。恢复时先重建任务图，只调度依赖已经完成的子任务。父任务等待所有必需结果后才能继续。每个子任务仍需独立租约、版本和幂等键。

28. 工具已经产生副作用，恢复后如何避免重复操作？

参考答案：调用前生成稳定幂等键，并在工具侧用唯一约束保存操作状态与结果。恢复时先按幂等键查询，成功则复用结果，执行中则等待或对账，失败才按策略重试。无法查询的外部操作进入人工确认或补偿。

29. 完整 RAG 链路如何设计？

参考答案：完整链路是采集与权限校验→解析/OCR→结构化与切分→去重和元数据→Embedding/全文索引→查询理解→权限过滤→混合召回→融合/重排→上下文组装→带引用生成→评测与反馈。跨 Chunk
信息用父子块、窗口扩展、实体/表格关联或多跳检索，权限过滤必须在召回阶段落实。

30. 如果记忆规模很小，还有必要使用向量库和 Reranker 吗？

参考答案：通常不需要向量库和 Reranker。数据量小时可用关系库、全文搜索、标签和时间排序，结构更简单且便于修改删除。只有语义查询需求明显、规模增长或召回质量不足时再引入向量检索。

31. 怎样看待 grep、glob、PageIndex 等不同检索方式？

参考答案：BM25 擅长精确词，向量召回语义近邻。两路各取候选后用 RRF 融合，再由 Reranker 精排。MMR
在结果高度重复、需要覆盖多方面时使用，会牺牲部分相关性。代码库常优先
grep/glob/符号索引，长文档可用层级索引。选择应由评测、时延和成本共同决定。

32. 网页内容中带有 Prompt Injection，怎么办？

参考答案：把网页/文档视为不可信数据，与系统指令分层标记，禁止其直接改变权限和工具策略。高风险动作由确定性策略引擎二次鉴权。再配合工具白名单、参数约束、最小权限、敏感信息脱敏和人工确认，并用注入样本做回归测试。

33. 信任边界、最小权限和人工确认应如何配合？

参考答案：在高风险、低置信度或需要业务决策的节点生成等待点编号，持久化候选动作、上下文摘要、允许的响应类型和过期策略。用户输入必须携带
任务编号和等待点编号，服务端校验身份与当前版本后写入事件并恢复。修改方向应使受影响步骤失效后重规划。

34. Thread.sleep(0) 有什么作用？

参考答案：Thread.sleep(0) 不会真正睡眠，通常只是给调度器一次重新调度的机会，其效果依赖 JVM
与操作系统，不能用于线程同步、让出 CPU 的可靠保证或等待条件。正确做法是锁、条件变量、队列、Future
或明确的调度原语。

35. 【算法题】 给一个含重复元素的数组，重新排列，使相邻元素不相同。

参考答案：可行条件是最大频次 maxCount ≤ $\lceil n/2 \rceil$ 。用最大堆每次取两个频次最高且不同的元素依次放回，或先把最高频元素放偶数位再填其余元素。时间 O(n log k)。见附录 E。

36. 【算法题·追问】 什么情况下无解，如何判断？

参考答案：可行条件是最大频次 maxCount ≤ $\lceil n/2 \rceil$ 。用最大堆每次取两个频次最高且不同的元素依次放回，或先把最高频元素放偶数位再填其余元素。时间 O(n log k)。见附录 E。

37. 平时有什么兴趣，最近读了什么书？

参考答案：选择真实、熟悉、能继续讨论的内容。说明最近阅读的【书/源码/论文】、关注的问题、得到的结论和适用限制。兴趣部分只需说明一项长期坚持的活动。

七、腾讯

一面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

2. 项目有哪些模块，一次请求的执行流程是什么？

参考答案：可分为控制面（配置、版本、发布、评测）、入口层（鉴权/会话/限流）、运行面（调度、状态机、模型与工具、Checkpoint）、能力层（Skill/MCP/RAG/Memory）和基础设施（队列、存储、沙箱、观测）。请求经入口创建 run，运行时逐步执行并持久化事件，最终输出答案/产物和完整审计记录。

3. 系统怎样部署，最终由谁使用？

参考答案：说明入口服务、运行时 Worker、队列、状态存储、模型网关和沙箱分别部署在哪里，以及如何扩缩容。再说明直接用户、业务系统和运维人员各自通过什么入口使用。部署结构必须与用户规模和数据边界一致。

4. 上下文预算如何分配？

参考答案：把预算分给系统/策略、当前任务、近期对话、检索证据和工具结果，并预留输出空间。达到 70%-80% 阈值时先删重复与低价值日志，再对旧历史和长工具结果做结构化摘要。必须保留目标、约束、关键事实、未完成项、来源引用和工具副作用记录，原文留在外部存储以便按需回取。

5. 历史消息和工具结果分别如何压缩？

参考答案：把预算分给系统/策略、当前任务、近期对话、检索证据和工具结果，并预留输出空间。达到 70%-80% 阈值时先删重复与低价值日志，再对旧历史和长工具结果做结构化摘要。必须保留目标、约束、关键事实、未完成项、来源引用和工具副作用记录，原文留在外部存储以便按需回取。

6. 压缩丢掉的信息后续又需要了，怎么找回来？

参考答案：把预算分给系统/策略、当前任务、近期对话、检索证据和工具结果，并预留输出空间。达到 70%-80% 阈值时先删重复与低价值日志，再对旧历史和长工具结果做结构化摘要。必须保留目标、约束、关键事实、未完成项、来源引用和工具副作用记录，原文留在外部存储以便按需回取。

7. 在什么时候触发压缩？同步做还是异步做？

参考答案：把预算分给系统/策略、当前任务、近期对话、检索证据和工具结果，并预留输出空间。达到 70%-80% 阈值时先删重复与低价值日志，再对旧历史和长工具结果做结构化摘要。必须保留目标、约束、关键事实、未完成项、来源引用和工具副作用记录，原文留在外部存储以便按需回取。

8. 是否支持多模态输入，整体链路如何处理？

参考答案：入口先识别文本、图片、音频和文件类型，完成病毒检查、权限校验和内容解析。不同模态由对应模型或解析器处理，再统一成带来源和位置的结构化内容进入上下文。原文件保存在外部存储，模型只接收当前任务需要的部分。

9. 视觉模型与文本模型什么时候切换，由谁决定？

参考答案：定义统一 Chat/Embedding/Rerank 接口，把消息、工具
参数格式、流式事件、错误码和用量归一化。适配器处理各 Provider 差异。网关按能力、数据边界、延迟、成本和健康度路由，配合限流、熔断、降级与成本标签。模型选择应版本化并可灰度回滚。

10. PDF、Word 等文档怎么处理？

参考答案：先保留文档结构而非只抽纯文本：PDF/PPT 按页、标题、版面和阅读顺序解析，图片走
OCR/视觉模型并关联页码。Excel 按 Sheet、表头、主键和行块处理，大表优先导入结构化存储后用
SQL/聚合检索。表格、图片和正文共享 doc_id/section_id 以支持联合召回。

11. 历史数据与实时信息如何结合，更新怎样生效？

参考答案：采集链路应有设备身份、序列号/事件时间、消息队列、幂等落库和数据质量告警。原始数据不可变，指标定义带版本。窗口按业务语义选择滚动/滑动并处理乱序水位。重算从原始数据按版本回放。Redis
可做低延迟临时状态，但不宜作为唯一的可审计历史存储。

12. 用户手工编排 Workflow，与自然语言生成计划有什么区别？

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用
ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给
Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

13. DSL 如何转换为执行图？

参考答案：DSL 先经过语法、参数格式、类型、引用和环路校验，再解析为带版本的
IR，完成默认值、依赖解析和节点绑定，最后编译为 DAG/状态图。自然语言只生成受限 IR 草案，不直接发布最终
DSL。生成后做静态校验、沙箱试跑、测试用例验证和基于错误的有限修复。

14. 视频生成等耗时较长的工具，怎样放进 Agent 执行流程？

参考答案：把外部长任务建模为 等待外部结果状态：提交后保存外部任务编号、恢复节点和截止时间并释放
Worker。优先用带签名的回调/事件触发，轮询作为兜底。收到结果后按 task_id + 等待点编号
幂等唤醒，超时走重试、取消或人工处理。

15. 计划保存在哪里？步骤失败、中断后怎样继续？

参考答案：以 task_id/run_id 保存不可变输入、当前节点、步骤状态、上下文摘要、工具调用记录、等待事件和版本号。每个状态跃迁在同一事务中写入数据库。恢复时从最后一个已提交边界继续，并以租约/版本校验防止双执行。外部副作用还要靠幂等键、状态查询或补偿，Checkpoint 本身不保证外部操作只执行一次。

16. 异步结果通过轮询还是回调拿到？

参考答案：把外部长任务建模为 等待外部结果状态：提交后保存外部任务编号、恢复节点和截止时间并释放
Worker。优先用带签名的回调/事件触发，轮询作为兜底。收到结果后按 task_id + 等待点编号
幂等唤醒，超时走重试、取消或人工处理。

17. 追问：拿到结果后，怎样恢复原来的任务？

参考答案：提交外部任务时保存本地任务编号、外部任务编号、等待点和恢复节点。结果到达后先校验签名和当前版本，再以等待点编号幂等写入结果并唤醒任务。若任务已取消、超时或已处理，该结果只记录不再触发执行。

18. Human-in-the-loop 在什么时候介入？

参考答案：在高风险、低置信度或需要业务决策的节点生成等待点编号，持久化候选动作、上下文摘要、允许的响应类型和过期策略。用户输入必须携带
任务编号和等待点编号，服务端校验身份与当前版本后写入事件并恢复。修改方向应使受影响步骤失效后重规划。

19. 用户补充输入、修改方向和等待确认，分别怎么处理？

参考答案：在高风险、低置信度或需要业务决策的节点生成等待点编号，持久化候选动作、上下文摘要、允许的响应类型和过期策略。用户输入必须携带
任务编号和等待点编号，服务端校验身份与当前版本后写入事件并恢复。修改方向应使受影响步骤失效后重规划。

20. 恢复执行时，如何确定输入对应哪个任务、哪个等待点？

参考答案：以 task_id/run_id 保存不可变输入、当前节点、步骤状态、上下文摘要、工具调用记录、等待事件和版本号。每个状态跃迁在同一事务中写入数据库。恢复时从最后一个已提交边界继续，并以租约/版本校验防止双执行。外部副作用还要靠幂等键、状态查询或补偿，Checkpoint 本身不保证外部操作只执行一次。

21. RAG 链路如何设计？

参考答案：完整链路是采集与权限校验→解析/OCR→结构化与切分→去重和元数据→Embedding/全文索引→查询理解→权限过滤→混合召回→融合/重排→上下文组装→带引用生成→评测与反馈。跨 Chunk
信息用父子块、窗口扩展、实体/表格关联或多跳检索，权限过滤必须在召回阶段落实。

22. Excel 或大型表格怎样解析和检索？

参考答案：先保留文档结构而非只抽纯文本：PDF/PPT 按页、标题、版面和阅读顺序解析，图片走
OCR/视觉模型并关联页码。Excel 按 Sheet、表头、主键和行块处理，大表优先导入结构化存储后用
SQL/聚合检索。表格、图片和正文共享 doc_id/section_id 以支持联合召回。

23. 【算法题】 二叉树的层序遍历。题型来自面试后的补充回忆，完整题面及作答过程未能确认。

参考答案：用队列保存待访问节点。每轮先记录队列长度，弹出这一层的节点并压入左右孩子，从而得到二维分层结果。时间 O(n)，队列空间 O(w)。见附录 F。

八、蚂蚁

一面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

2. 如果参与过 Java 微服务项目，主要负责哪些模块？

参考答案：按真实经历说明服务拆分、接口设计、数据库、缓存、队列、并发、监控和部署中实际负责的部分。只描述亲自做过的模块，并说明规模、故障处理和结果。

3. 以实时通信系统为例，底层采用 SIP、VoIP 还是其他协议，应如何选择？

参考答案：协议选择看终端兼容、媒体能力、NAT 穿透、合规和运维成本。SIP 负责会话信令，RTP/相关协议承载媒体。底层维护连接与协议状态，上层维护业务会话和坐席分配。事件带会话 ID、序号与时间戳，经幂等消费和对账修正状态。

4. 通信控制层与上层业务服务分别负责什么？

参考答案：协议选择看终端兼容、媒体能力、NAT 穿透、合规和运维成本。SIP 负责会话信令，RTP/相关协议承载媒体。底层维护连接与协议状态，上层维护业务会话和坐席分配。事件带会话 ID、序号与时间戳，经幂等消费和对账修正状态。

5. 系统的坐席和并发规模如何？

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

6. 连接状态由底层通信组件还是上层调度服务维护？

参考答案：底层组件维护网络连接和协议状态，上层服务维护业务会话、坐席分配和任务状态。两层通过带序号的状态事件同步。上层不能凭接口返回猜测连接状态，底层也不应直接决定业务流程。

7. 怎样保证通话状态同步和准确？

参考答案：协议选择看终端兼容、媒体能力、NAT 穿透、合规和运维成本。SIP 负责会话信令，RTP/相关协议承载媒体。底层维护连接与协议状态，上层维护业务会话和坐席分配。事件带会话 ID、序号与时间戳，经幂等消费和对账修正状态。

8. 状态事件通过消息队列还是回调传递？

参考答案：跨服务、需要重放和削峰的状态事件更适合消息队列。低延迟且调用关系简单的通知可用回调。关键状态仍应落库，队列或回调只负责传递事件，并通过事件编号、幂等消费和对账保证一致。

9. 如果参与过数据处理平台，平台解决什么问题，个人负责哪部分？

参考答案：建议答法：平台面向【内部开发者/企业客户】，把模型、知识、工具和流程统一成可发布、可治理的 AI 应用。核心解决重复接入、权限审计、长任务可靠性和效果回归。交付物应明确为产品实例、配置或扩展包、接口与运维指标，不能只说“交付一个 Agent”。

10. 以设备数据为例，怎样设计上报和存储链路？

参考答案：采集链路应有设备身份、序列号/事件时间、消息队列、幂等落库和数据质量告警。原始数据不可变，指标定义带版本。窗口按业务语义选择滚动/滑动并处理乱序水位。重算从原始数据按版本回放。Redis
可做低延迟临时状态，但不宜作为唯一的可审计历史存储。

11. 使用什么时序数据库？

参考答案：采集链路应有设备身份、序列号/事件时间、消息队列、幂等落库和数据质量告警。原始数据不可变，指标定义带版本。窗口按业务语义选择滚动/滑动并处理乱序水位。重算从原始数据按版本回放。Redis
可做低延迟临时状态，但不宜作为唯一的可审计历史存储。

12. 如果数据上报丢失，怎样发现和定位？

参考答案：采集链路应有设备身份、序列号/事件时间、消息队列、幂等落库和数据质量告警。原始数据不可变，指标定义带版本。窗口按业务语义选择滚动/滑动并处理乱序水位。重算从原始数据按版本回放。Redis
可做低延迟临时状态，但不宜作为唯一的可审计历史存储。

13. 用户怎样配置指标计算逻辑？

参考答案：通过受限 DSL 或可视化配置定义数据源、字段、过滤条件、窗口、聚合方式和输出单位。发布前做类型、权限和资源校验，并生成版本。生产执行使用已发布版本，不能直接运行任意用户代码。

14. 流式数据的时间窗口如何实现？

参考答案：事件使用单调 事件编号 持久化，客户端记录 最后收到的事件编号，重连后先补发缺口再切到实时流。客户端按 id 去重，服务端保留心跳和可控的事件窗口。首个 Token 延迟（TTFT）要拆成排队、上下文准备、模型首 Token
和网络传输，分别统计 P50/P95。

15. 追问：为什么没有采用滑动窗口？

参考答案：采集链路应有设备身份、序列号/事件时间、消息队列、幂等落库和数据质量告警。原始数据不可变，指标定义带版本。窗口按业务语义选择滚动/滑动并处理乱序水位。重算从原始数据按版本回放。Redis
可做低延迟临时状态，但不宜作为唯一的可审计历史存储。

16. 为什么需要中间汇总计算层？

参考答案：采集链路应有设备身份、序列号/事件时间、消息队列、幂等落库和数据质量告警。原始数据不可变，指标定义带版本。窗口按业务语义选择滚动/滑动并处理乱序水位。重算从原始数据按版本回放。Redis
可做低延迟临时状态，但不宜作为唯一的可审计历史存储。

17. 指标定义发生变化后，历史数据怎么重算？

参考答案：指标定义必须版本化，原始数据保持不可变。重算任务指定数据范围和目标版本，从原始数据或可验证中间结果重新计算，写入新的结果分区。完成核对后再切换读取版本，并保留回滚能力。

18. 追问：如果重新设计指标版本与重算机制，会怎样改进？

参考答案：把指标定义、依赖、代码和参数统一版本化，保存生效时间和数据范围。重算由独立任务编排，支持分片、幂等、进度、失败恢复和结果校验。新旧结果并存，验证通过后再原子切换。

19. 是否考虑过使用 Redis 承载中间计算？

参考答案：采集链路应有设备身份、序列号/事件时间、消息队列、幂等落库和数据质量告警。原始数据不可变，指标定义带版本。窗口按业务语义选择滚动/滑动并处理乱序水位。重算从原始数据按版本回放。Redis
可做低延迟临时状态，但不宜作为唯一的可审计历史存储。

20. 为什么选择 LangGraph？

参考答案：选择依据是显式状态图、条件边、Checkpoint/中断恢复、流式事件和可插拔节点，适合长任务与人机协作。把它限定在核心编排层，工具治理、权限、租约、评测和业务扩展由自有层实现，避免深度绑定。同时用同一任务比较自研 Loop、LangGraph 与其他框架的复杂度和运行指标。

21. LangGraph 在平台中具体承担什么职责？

参考答案：选择依据是显式状态图、条件边、Checkpoint/中断恢复、流式事件和可插拔节点，适合长任务与人机协作。把它限定在核心编排层，工具治理、权限、租约、评测和业务扩展由自有层实现，避免深度绑定。同时用同一任务比较自研 Loop、LangGraph 与其他框架的复杂度和运行指标。

22. 为什么不用封装程度更高的 Agent 框架？

参考答案：封装越高，上手越快，但状态、恢复、工具执行和观测更难按业务要求调整。需要强权限、私有化、长任务和领域流程时，选择较低层框架更容易控制边界。前提是团队愿意承担更多工程建设和维护成本。

23. 从文件上传到问答，知识库的完整处理链路是什么？

参考答案：完整链路是采集与权限校验→解析/OCR→结构化与切分→去重和元数据→Embedding/全文索引→查询理解→权限过滤→混合召回→融合/重排→上下文组装→带引用生成→评测与反馈。跨 Chunk
信息用父子块、窗口扩展、实体/表格关联或多跳检索，权限过滤必须在召回阶段落实。

24. 超大型文件或十几万行的 Excel 如何处理？

参考答案：先保留文档结构而非只抽纯文本：PDF/PPT 按页、标题、版面和阅读顺序解析，图片走
OCR/视觉模型并关联页码。Excel 按 Sheet、表头、主键和行块处理，大表优先导入结构化存储后用
SQL/聚合检索。表格、图片和正文共享 doc_id/section_id 以支持联合召回。

25. 关联信息在文档中相距很远，怎样保证一起召回？

参考答案：完整链路是采集与权限校验→解析/OCR→结构化与切分→去重和元数据→Embedding/全文索引→查询理解→权限过滤→混合召回→融合/重排→上下文组装→带引用生成→评测与反馈。跨 Chunk
信息用父子块、窗口扩展、实体/表格关联或多跳检索，权限过滤必须在召回阶段落实。

26. Metadata Filter 在检索中怎么使用？

参考答案：在召回前根据租户、用户权限、文档类型、时间、业务域和状态过滤候选，防止无权内容进入后续排序。过滤字段来自受信任元数据，不能由模型随意生成。过滤过严或字段缺失要有监控和回退。

27. PPL 语义边界检测如何实现？

参考答案：固定切分稳定便宜，适合结构弱文本。层级切分保留标题/章节关系。语义或 PPL 切分在相邻句条件概率突变处找边界。阈值应在标注集上按文档类型调优，而非全局固定。同时评估 Recall、完整性、Chunk 数、延迟和成本，并为表格/代码使用专门规则。

28. 追问：这种切分方案有哪些问题？

参考答案：语义或 PPL 切分依赖模型和阈值，结果可能不稳定，成本也高于规则切分。标题、表格、代码和跨段引用容易被错误拆开，过细会丢失完整语义，过粗又会增加检索噪声。因此要保留文档结构，并设置规则回退。

29. 如果 Recall@5 突然大幅下降，怎么排查？

参考答案：检索层用 Recall@K 看相关证据是否被召回，MRR 看首个相关结果位置，NDCG 适合多级相关性和整体排序。回答层再看忠实性、完整性、引用正确率与人工胜率。数据集要覆盖真实查询、难例和时间切片，并报告置信区间与失败样本。

30. 追问：用户、场景和配置都没有变化，应优先怀疑什么？

参考答案：优先检查模型或服务版本漂移、索引是否重建失败、增量数据是否缺失、Embedding 是否变化、缓存是否异常，以及上游预处理结果是否改变。先核对版本哈希、索引时间和文档数量，再用固定查询复现。

31. Embedding 是否“幂等”，同一输入能否得到稳定输出？

参考答案：为每次业务操作生成稳定 幂等键，并在工具侧建立唯一约束和结果表。超时且结果未知时先按 key 查询外部系统，不盲目重试。无法查询时把状态置为 UNKNOWN，走对账、补偿或人工确认。Harness 负责传播 key 和策略，业务工具负责真正的原子去重。

32. 追问：怎样验证输出的稳定性？

参考答案：同一模型和参数通常近似稳定，但服务版本、硬件、浮点计算、批处理和预处理变化都可能造成漂移。固定模型版本与归一化流程，重复请求统计向量余弦差/排序一致性，并对 Reranker 做 JSON 参数格式、分数范围、候选 ID 白名单和回退排序校验。

33. 使用什么 Reranker？

参考答案：应直接说明实际使用的模型、版本、部署方式、输入长度和选择依据。选择时比较排序效果、语言覆盖、延迟、吞吐、成本和私有化要求，并准备无 Reranker 或轻量排序的降级方案。

34. Reranker 的输出是否稳定？

参考答案：通常近似稳定，但模型版本、服务实现、浮点计算、批处理和输入截断都可能造成变化。固定版本和预处理后，重复运行同一候选集，统计排序一致性和分数波动，并对输出格式和候选编号做严格校验。

35. 追问：如何保证模型排序结果符合格式和行为约束？

参考答案：同一模型和参数通常近似稳定，但服务版本、硬件、浮点计算、批处理和预处理变化都可能造成漂移。固定模型版本与归一化流程，重复请求统计向量余弦差/排序一致性，并对 Reranker 做 JSON 参数格式、分数范围、候选 ID 白名单和回退排序校验。

36. Agent 执行链路如何做到可观测？

参考答案：每次请求绑定 trace_id/run_id/租户编号，Trace 记录模型、工具、节点和等待时段，Metric 统计成功率、完成率、P50/P95 延迟、Token/成本、重试和队列积压，Log 记录结构化错误但脱敏。可观测解决“发生了什么”，评测再把轨迹与任务质量标签连接起来。

37. 追问：有了可观测性，怎样进一步做到可评测？

参考答案：每次请求绑定 trace_id/run_id/租户编号，Trace 记录模型、工具、节点和等待时段，Metric 统计成功率、完成率、P50/P95 延迟、Token/成本、重试和队列积压，Log 记录结构化错误但脱敏。可观测解决“发生了什么”，评测再把轨迹与任务质量标签连接起来。

38. 最终输出与工具调用轨迹分别怎样评价？

参考答案：最终输出看正确性、完整性、忠实性、格式和业务完成度。工具轨迹看必需步骤是否执行、工具和参数是否正确、是否出现越权或重复副作用。两者分别计分，因为结果正确不代表过程合规，过程合规也不代表结果正确。

39. 为什么独立建设 Agent Harness？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

40. 通用运行内核、服务入口与业务扩展怎样分工？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

41. 如何约束业务实现，避免侵入运行内核？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

42. Codex 与 Claude Code 有什么区别？

参考答案：比较时应分开看模型能力和 Harness 能力。重点比较工具协议、上下文管理、计划与执行方式、代码与终端权限、检查点、人工交互、扩展机制和安全边界。不能只根据一次回答效果判断产品差异。

43. 追问：行为差异主要来自模型，还是 Harness？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

二面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

2. 业务 Agent Harness 是当前的重点工作吗？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

3. 项目团队的开发人员和其他角色怎样配置？

参考答案：按真实团队说明产品、业务、算法、平台、后端、前端、测试、交付和运维人员数量及职责。重点解释需求、架构、代码、评测、发布和线上支持分别由谁负责。

4. 主要采用 AI 辅助编码，还是传统人工编码？

参考答案：按真实比例说明。AI 可以用于代码草稿、测试和重构，但接口设计、安全、核心逻辑和最终合并由工程师负责。所有 AI 生成代码都应经过评审、静态检查和测试，不能按生成量衡量贡献。

5. 你在项目中具体负责什么？

参考答案：明确说明自己负责的模块、做出的关键决策、亲自完成的代码或方案，以及与其他成员的边界。再说明交付结果和量化指标。团队已有能力、开源框架和个人新增能力要分开描述。

6. 多人同时使用 AI Coding 时，怎样拆分任务、控制协作和集成风险？

参考答案：按稳定接口和业务域拆分模块，使用独立版本、测试数据和运行命名空间。合并前执行接口兼容检查、自动测试、代码评审和端到端回归。共享内核只接受通过扩展接口的能力，禁止业务模块直接修改全局状态。

7. 业务扩展模块是否按业务域划分？

参考答案：服务入口处理认证、协议、会话、限流和 SSE。通用运行内核处理状态机、模型/工具循环、Checkpoint、预算和观测。业务扩展提供领域 Skill、Workflow、Memory 策略、数据适配与完成校验。依赖方向指向稳定接口，业务层不反向修改内核。

8. 运行内核由多少人维护？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

9. 你在团队中的职级和角色是什么？

参考答案：如实给出公司职级、对应职责范围和时间线。晋升用"承担范围扩大 + 关键结果 + 组织影响"说明，绩效与薪酬只报可公开且可核验的信息。若体系不可横向比较，可补充带队规模、决策权和交付复杂度。

10. 不同用工类型的人员怎样分工？

参考答案：按责任和风险分工，不按身份简单分配。核心架构、权限、安全和生产发布由长期负责团队掌握。边界清晰的模块、测试和交付工作可由其他成员承担。所有代码使用同一评审、测试和权限标准。

11. 运行内核、业务扩展和服务入口是独立 REST 服务，还是装配在同一个服务内？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

12. 运行内核从零实现，还是基于开源项目？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

13. LangGraph 在其中承担什么职责？

参考答案：选择依据是显式状态图、条件边、Checkpoint/中断恢复、流式事件和可插拔节点，适合长任务与人机协作。把它限定在核心编排层，工具治理、权限、租约、评测和业务扩展由自有层实现，避免深度绑定。同时用同一任务比较自研 Loop、LangGraph 与其他框架的复杂度和运行指标。

14. 一轮 ReAct 包含哪些节点？

参考答案：核心循环是 Thought/Decision→Action→工具结果（Observation）→更新状态，直到 Final 或触发预算/取消/错误边界。生产实现还必须加入工具鉴权、参数校验、超时、幂等、Checkpoint、上下文预算和可观测事件。循环迟迟不结束时用最大轮次、重复调用检测和无进展判定收敛。

15. Observation 里记录哪些信息？

参考答案：记录工具名称与版本、调用编号、参数摘要、开始结束时间、状态、结构化结果、错误类型、重试次数、外部副作用、结果引用和可见范围。敏感字段必须脱敏，过长结果只保存引用。

16. 为什么可以把这套运行内核看作最小 Harness？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

17. 业务扩展模块具体包含什么？

参考答案：服务入口处理认证、协议、会话、限流和SSE。通用运行内核处理状态机、模型/工具循环、Checkpoint、预算和观测。业务扩展提供领域 Skill、Workflow、Memory 策略、数据适配与完成校验。依赖方向指向稳定接口，业务层不反向修改内核。

18. 追问：它与 Skill 是什么关系？

参考答案：Tool 是原子能力，Skill 是“何时以及怎样使用一组能力”的可复用方法，Workflow 是确定性的步骤编排。把变化快、强业务语义的 Skill 放在扩展层，内核只负责发现、加载、权限、执行与观测接口，以实现版本化和不改内核扩展。

19. MCP、外部业务系统适配和权限约束放在哪一层？

参考答案：MCP 协议与外部系统适配放在能力接入层，负责把外部接口转换为统一工具。身份传递、权限策略和审计由平台治理层定义，并在工具执行层强制校验。业务扩展只声明需要的权限，不能自行绕过。

20. 描述式 Skill 和 Workflow Skill 有什么区别？

参考答案：描述式 Skill 主要向模型提供规则、步骤和资源，没有独立执行状态。Workflow Skill 是可执行流程，拥有节点、状态、重试、等待和结果。前者更灵活，后者更稳定、可审计。

21. 确定性 Workflow 如何挂起并恢复 ReAct？

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

22. 目前经过多少真实客户的验证？

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

23. 为什么考虑换工作？

参考答案：用正向动机回答：现岗位让我积累了【能力】，下一步希望在更大规模、更完整责任边界下深耕 Agent 工程化。目标岗位在【业务/技术/团队】上与此匹配。不要抱怨原公司，也不要只说薪资。补充未来 2-3 年希望承担的结果。

24. 目前的职级、绩效和薪酬结构如何？

参考答案：如实给出公司职级、对应职责范围和时间线。晋升用“承担范围扩大 + 关键结果 + 组织影响”说明，绩效与薪酬只报可公开且可核验的信息。若体系不可横向比较，可补充带队规模、决策权和交付复杂度。

25. 应用平台与 Dify、Coze、Palantir 分别是什么关系？

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

26. 下层业务数据、逻辑和动作怎样被应用平台使用？

参考答案：数据通过受权限控制的查询接口或 RAG 接入，业务逻辑通过服务或 Workflow 接入，写操作通过有明确参数和审计的工具接入。所有调用携带用户和租户身份，读取与写入分别授权，模型不能直接连接底层数据库。

27. 平台当前有多少客户和线上用户？

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

28. 如果没有深度业务共建，平台怎样形成超越通用低代码产品的竞争力？

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

29. 低代码产品本来是为了降低门槛，为什么仍需要研发人员陪同客户落地？

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

30. 什么情况下需要从画布平台转向 Agent Harness？

参考答案：当任务路径难以预先画出、需要动态选工具、长期运行、持续交互或个人工作空间时，才需要 Agent Harness。规则固定、可视化配置足够的业务继续使用画布 Workflow。两者可以共享工具、权限和发布能力。

31. 一次客户交付之后，哪些东西能够成为长期资产？

参考答案：可沉淀的内容包括需求分类、接入规范、领域数据模型、工具适配器、权限模板、评测集、部署脚本、监控规则和排障手册。客户专有逻辑应留在配置或扩展层，通用部分再进入平台产品。

32. 原平台的 Workflow 和 Agent 能否迁移到新运行时？

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

33. 最复杂的 Workflow 大概有多大？

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

34. 平台支持哪些节点类型？

参考答案：节点统一包含类型、版本、输入输出格式、配置、权限、超时重试、状态和执行器。常见类型包括模型、工具、条件、并行、子流程、人工等待、代码、检索和结果校验。节点定义与运行状态必须分离，多个任务才能安全复用同一张图。

35. 迁移后，原有 Workflow 能力怎样继续承载？

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

36. 你说的 Workflow Skill，究竟是描述文件，还是一段可执行代码？

参考答案：Skill 包含声明、说明、资源与可执行入口。运行时只把必要说明渐进式注入上下文。Workflow Skill 以受版本管理的确定性流程执行：ReAct 挂起并传入结构化参数，子流程返回结果/错误/产物后主循环恢复。脚本和代码必须经过签名、权限与沙箱策略。

37. 追问：业务逻辑是不是仍然需要研发人员编写代码？

参考答案：需要。稳定、复杂或高风险的业务逻辑仍应由研发通过工具、服务或 Workflow 实现。低代码和 Agent 主要降低配置、组合和调用成本，不能替代对正确性、权限和维护负责的工程实现。

38. 运行内核以外，还有哪些能够稳定复用的公共资产？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

九、百度

一面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

2. 是否了解百度千帆 AppBuilder?

参考答案：应先说明了解程度，再从应用编排、模型与知识接入、工具、发布和企业治理几个方面概括。若没有深入使用，应明确说明只做过资料阅读或基础验证，不要把了解产品说成生产经验。

3. 是否了解 Coze、Dify 等 AI 应用开发平台?

参考答案：按真实使用深度说明。可以从画布编排、模型与知识库接入、工具、发布、运行时和企业治理比较，但要区分阅读资料、实际试用和生产使用。不了解的实现细节不要推测。

4. 当前平台的产品定位和技术选型是什么?

参考答案：先说明服务对象、核心问题和交付形态，再说明运行框架、服务语言、存储、队列、模型网关和部署方式。每项技术选择都要对应业务约束，不能只列技术名称。

5. 为什么使用 LangGraph，平台自行实现了哪些部分?

参考答案：LangGraph 用于状态图、条件分支和节点调度。平台自行实现模型网关、工具注册与鉴权、Skill、上下文、任务租约、持久化恢复、SSE、多租户、沙箱、观测和评测。回答时应明确每部分的责任边界。

6. 画布式 Workflow 编排中最大的难点是什么?

参考答案：DSL 先经过语法、参数格式、类型、引用和环路校验，再解析为带版本的 IR，完成默认值、依赖解析和节点绑定，最后编译为 DAG/状态图。自然语言只生成受限 IR 草案，不直接发布最终 DSL。生成后做静态校验、沙箱试跑、测试用例验证和基于错误的有限修复。

7. 大型复杂流程如何通过子 Workflow 拆分?

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

8. 主流程与子流程怎样传递上下文和结果?

参考答案：把预算分给系统/策略、当前任务、近期对话、检索证据和工具结果，并预留输出空间。达到 70%-80% 阈值时先删重复与低价值日志，再对旧历史和长工具结果做结构化摘要。必须保留目标、约束、关键事实、未完成项、来源引用和工具副作用记录，原文留在外部存储以便按需回取。

9. 追问：子流程出现中断或异常，主流程如何处理？

参考答案：子流程应把状态、错误类型和已产生的副作用返回主流程。主流程根据策略选择重试、跳过、补偿、降级或终止，并保存子流程检查点。不能把异常吞掉后继续标记整个流程成功。

10. 代码节点怎样保证执行安全？

参考答案：执行放到短生命周期、无特权的隔离环境：只读基础镜像、独立用户和命名空间、CPU/内存/进程数/时长配额、临时工作目录和默认断网。文件与网络按白名单开放。调度器在沙箱外做鉴权、审计和 Kill，死循环由超时与资源配额终止。

11. Docker 或 Kubernetes 提供了哪些隔离机制？

参考答案：Docker 提供进程、文件系统、网络、用户和资源命名空间隔离，并可通过只读镜像、能力裁剪和 seccomp 限制系统调用。Kubernetes 在此基础上提供
Pod、命名空间、网络策略、资源配额、服务账号和安全策略。两者都不能替代应用层鉴权。

12. 从文件上传到生成回答，完整 RAG 链路是什么？

参考答案：完整链路是采集与权限校验→解析/OCR→结构化与切分→去重和元数据→Embedding/全文索引→查询理解→权限过滤→混合召回→融合/重排→上下文组装→带引用生成→评测与反馈。跨 Chunk
信息用父子块、窗口扩展、实体/表格关联或多跳检索，权限过滤必须在召回阶段落实。

13. Excel 等结构化文档如何解析和切分？

参考答案：先保留文档结构而非只抽纯文本：PDF/PPT 按页、标题、版面和阅读顺序解析，图片走 OCR/视觉模型并关联页码。Excel 按 Sheet、表头、主键和行块处理，大表优先导入结构化存储后用 SQL/聚合检索。表格、图片和正文共享 doc_id/section_id 以支持联合召回。

14. 固定切分、层级切分和语义切分如何选择？

参考答案：固定切分稳定便宜，适合结构弱文本。层级切分保留标题/章节关系。语义或 PPL 切分在相邻句条件概率突变处找边界。阈值应在标注集上按文档类型调优，而非全局固定。同时评估 Recall、完整性、Chunk 数、延迟和成本，并为表格/代码使用专门规则。

15. PPL 语义切分的原理和依据是什么？

参考答案：固定切分稳定便宜，适合结构弱文本。层级切分保留标题/章节关系。语义或 PPL 切分在相邻句条件概率突变处找边界。阈值应在标注集上按文档类型调优，而非全局固定。同时评估 Recall、完整性、Chunk 数、延迟和成本，并为表格/代码使用专门规则。

16. 追问：怎么通过评测证明这种切分有效？

参考答案：保持文档、查询、Embedding、召回和重排配置不变，只替换切分方式。比较 Recall@K、完整证据命中率、回答正确率、块数量、延迟和成本，并对差异做人工复核。只有在多个指标和关键场景上稳定提升，才能认为有效。

17. BM25、向量检索、RRF 和 Rerank 如何组合？

参考答案：BM25 擅长精确词，向量召回语义近邻。两路各取候选后用 RRF 融合，再由 Reranker 精排。MMR 在结果高度重复、需要覆盖多方面时使用，会牺牲部分相关性。代码库常优先 grep/glob/符号索引，长文档可用层级索引。选择应由评测、时延和成本共同决定。

18. 答案需要的信息跨多个 Chunk 时，怎样召回？

参考答案：固定切分稳定便宜，适合结构弱文本。层级切分保留标题/章节关系。语义或 PPL 切分在相邻句条件概率突变处找边界。阈值应在标注集上按文档类型调优，而非全局固定。同时评估 Recall、完整性、Chunk 数、延迟和成本，并为表格/代码使用专门规则。

19. 怎样看待 Coding Agent 使用 grep 或文件遍历，而不是向量 RAG？

参考答案：代码具有精确符号、路径和字符串，grep、glob 与符号索引可快速得到可解释结果，并保持最新，不需要额外向量索引。向量检索更适合语义模糊查询。实际系统可先做精确搜索，再在需要时补充语义检索。

20. PageIndex 一类层级索引适合什么场景？

参考答案：适合章节结构清晰、篇幅长、查询需要先定位目录再深入正文的文档。它能保留父子关系和全局位置，但构建与查询更复杂。结构弱、文档短或高频实时更新时，普通全文和向量索引可能更合适。

21. 流式节点与非流式节点如何衔接，并统一展示？

参考答案：事件使用单调 事件编号 持久化，客户端记录 最后收到的事件编号，重连后先补发缺口再切到实时流。客户端按 id 去重，服务端保留心跳和可控的事件窗口。首个 Token 延迟（TTFT）要拆成排队、上下文准备、模型首 Token 和网络传输，分别统计 P50/P95。

22. 追问：是否研究过 Dify 对这两类节点的抽象？

参考答案：按真实阅读结果说明流式输出怎样表示、节点结果怎样统一成事件、下游节点何时开始执行，以及错误与取消怎样传播。若未读到具体实现，应直接说明只了解公开接口，不推测内部机制。

23. 平台的客户量、调用量和并发规模怎样？

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

24. 平台如何部署，Pod 和 Worker 如何组织？

参考答案：部署时通常把入口服务、任务队列、Worker 池和状态存储分开。入口负责接收请求，队列保存待执行任务，Worker 负责实际运行，数据库或对象存储保存配置、进度和结果。公有云部署更便于弹性扩缩。私有化部署则要确保模型、数据、日志和密钥都留在客户规定的边界内。

25. Python GIL 对多核并发有什么影响？

参考答案：GIL 使一个 CPython 进程中的多个线程不能同时执行 Python 字节码，因此 CPU 密集任务难以利用多核。I/O 等待时线程可以释放 GIL，所以网络和文件 I/O 仍可并发。CPU 密集任务应使用多进程、原生扩展或独立计算服务。

26. FastAPI 的并发依靠协程、线程还是进程？

参考答案：async 接口主要依靠事件循环和协程处理非阻塞 I/O。普通 def 接口通常由线程池执行，避免阻塞事件循环。多个服务进程由 Uvicorn 或进程管理器启动，用于多核和故障隔离。三者可以同时存在。

27. 线上接口延迟突然升高，怎么排查？

参考答案：先确认是全局还是局部、从何时开始，再按网关排队、应用线程或协程、模型与工具下游、数据库、网络和资源使用逐段定位。对比 P50/P95、队列长度、CPU、内存、连接池和下游延迟，并核对最近发布与配置变化。

28. PostgreSQL 慢查询怎么定位，EXPLAIN 和索引如何使用？

参考答案：先从慢查询日志和统计视图找到高耗时 SQL，再用 EXPLAIN ANALYZE 查看实际执行计划、扫描行数、连接方式、排序和临时文件。索引应服务真实过滤、连接和排序条件，同时检查基数估计、统计信息、锁等待和连接池。

29. 【口述算法题】 怎样实现固定容量的 LRU Cache？

参考答案：哈希表负责快速找到数据，双向链表负责记录最近使用顺序。get 和 put 都把刚访问的节点移到链表头部。超过容量时删除尾部节点，并从哈希表同步删除。两种操作的平均时间复杂度都是 O(1)，代码见附录 G。

30. 【口述系统设计题】 如果自己实现线程池，需要哪些核心组件，执行流程是什么？

参考答案：核心组件包括有界任务队列、Worker 集合、拒绝策略、生命周期状态、锁/条件变量和监控。submit 入队并唤醒 Worker。Worker 循环取任务、执行、捕获异常并更新指标。shutdown 停止接收并决定排空或取消。见附录 H。

31. 追问：任务进入队列之后，由谁驱动 Worker 取出并执行？

参考答案：Worker 自己运行取任务循环，通过条件变量、阻塞队列或消息队列消费接口等待任务。提交线程只负责入队和唤醒，调度器负责扩缩容与生命周期。Worker 取到任务后更新状态，执行完成再继续获取下一项。

32. 如果项目涉及 Ray，会用它承担哪些工作？

参考答案：Ray 可承担分布式任务调度、并行数据处理、模型推理和有状态 Actor。是否使用取决于任务并行度、数据规模和容错需求。普通异步 I/O 服务不必为了分布式而引入 Ray。

33. Ray Head 与 Worker 如何解耦？

参考答案：Head 负责控制面、资源信息和调度，Worker 负责执行任务。业务任务定义、输入、检查点和结果位置应保存在外部系统，不能只依赖 Head 内存。通过稳定任务编号和外部状态存储，Head 重启后可以重新发现 Worker 并重建调度。

34. Head 故障后，任务调度、结果汇聚和后续任务怎样继续？

参考答案：先区分 Ray 控制面是否启用高可用和外部持久化。任务定义、对象引用和业务检查点不能只存在 Head 内存中。Head 恢复后应重新发现 Worker，并由业务层根据任务状态重建调度和结果汇聚。无法恢复的任务按幂等规则重跑。

二面

1. 研发经历是否都与 Agent 有关？如果涉及方向转型，是在什么阶段？

参考答案：按时间顺序说明当前岗位、主要工作和转向 Agent 的阶段，再说明最熟悉的技术方向。重点写清实际负责的模块、投入时间和已经交付的结果，不要把学习、验证和生产经验混在一起。

2. AI 应用平台具体是什么，客户是谁？

参考答案：先说明平台向谁提供服务，再说明它把模型、知识库、工具和流程组合成可发布、可治理的 AI 应用。客户可以是内部业务团队或外部企业，终端用户与付费客户要区分。最后说明交付形态和核心指标。

3. 部署在哪里，目前规模多大？

参考答案：部署时通常把入口服务、任务队列、Worker 池和状态存储分开。入口负责接收请求，队列保存待执行任务，Worker 负责实际运行，数据库或对象存储保存配置、进度和结果。公有云部署更便于弹性扩缩。私有化部署则要确保模型、数据、日志和密钥都留在客户规定的边界内。

4. 你在平台中具体承担哪些工作？

参考答案：明确列出自己负责的模块、设计决策、代码范围、上线责任和结果指标。说明与产品、算法、后端、交付和运维的分工，并区分团队已有能力、开源能力和个人新增内容。

5. 为什么选择 LangGraph，它在系统中负责什么？

参考答案：选择依据是显式状态图、条件边、Checkpoint/中断恢复、流式事件和可插拔节点，适合长任务与人机协作。把它限定在核心编排层，工具治理、权限、租约、评测和业务扩展由自有层实现，避免深度绑定。同时用同一任务比较自研 Loop、LangGraph 与其他框架的复杂度和运行指标。

6. 工具分层最终解决谁的问题？模型看到的不都是工具吗？

参考答案：服务入口处理认证、协议、会话、限流和 SSE。通用运行内核处理状态机、模型/工具循环、Checkpoint、预算和观测。业务扩展提供领域 Skill、Workflow、Memory 策略、数据适配与完成校验。依赖方向指向稳定接口，业务层不反向修改内核。

7. 用户创建的 AI 应用如何评价，有哪些具体指标和数据？

参考答案：把评测拆成任务完成、答案质量、轨迹合规、延迟/成本和安全。参考标准（Ground Truth）由真实任务抽样、专家标注和生产失败样本构成，保存期望结果及允许的多种轨迹。LLM Judge 用明确的评分标准、盲测和多 Judge/人工抽检校准，不能只让同类模型自评。发布前回归，线上灰度 A/B 并设回滚阈值。

8. P95 TTFT 统计什么阶段？首响应较长时应如何分析原因？

参考答案：首个 Token 延迟（TTFT）
从服务端收到请求开始，到客户端收到第一个可见内容结束。应拆分排队、鉴权、上下文准备、模型请求、模型首 Token 和网络传输。P95 较高时先看哪一段长尾增加，再检查队列、上下文长度、模型负载和网络。

9. 画布平台、Agent Harness 和客户交付是什么关系？

参考答案：画布平台负责配置、编排、测试和发布。Agent Harness 负责线上执行、状态、工具、恢复和观测。客户交付是在两者之上加入业务数据、权限、扩展和部署配置。三者共享能力，但产品入口、运行责任和交付边界不同。

10. 客户最终购买什么，拿到什么交付物？

参考答案：建议答法：平台面向【内部开发者/企业客户】，把模型、知识、工具和流程统一成可发布、可治理的 AI 应用。核心解决重复接入、权限审计、长任务可靠性和效果回归。交付物应明确为产品实例、配置或扩展包、接口与运维指标，不能只说“交付一个 Agent”。

11. 客户数据在这条链路中如何处理？

参考答案：先按数据级别和用途授权，再在客户边界或指定区域完成存储与处理。传输与静态存储都要加密，模型调用前做最小化和脱敏，检索按用户与租户过滤。日志不得保存敏感原文，并支持审计、保留期限和删除。

12. 平台的执行引擎与独立 Agent 运行内核，本质区别是什么？

参考答案：执行引擎只负责节点调度、状态传递和条件跳转。Agent 运行内核还负责模型与工具循环、上下文、权限、预算、恢复、事件、评测和多租户。前者是调度组件，后者是完整运行环境。

13. Skill 有什么作用？

参考答案：Tool 是原子能力，Skill 是“何时以及怎样使用一组能力”的可复用方法，Workflow 是确定性的步骤编排。把变化快、强业务语义的 Skill 放在扩展层，内核只负责发现、加载、权限、执行与观测接口，以实现版本化和不改内核扩展。

14. 追问：为什么还要把 Workflow 包装成 Skill？

参考答案：封装后，Agent 可以把一段经过验证的固定流程当作一个受控能力调用，而不需要理解内部每个节点。这样能复用权限、版本、输入输出和观测机制，并减少上下文长度。代价是自由度下降和维护一个额外能力契约。

15. 企业 Agent 为什么不像 Codex、Claude Code 那样保持更开放的自主性？

参考答案：企业任务通常有更严格的权限、审批、数据边界、审计和稳定性要求，因此不能把所有步骤交给模型自由决定。可以在低风险、可校验的局部保留自主性，关键写入和发布仍由 Workflow、规则和人工确认控制。

16. App 关闭再打开后，怎样继续看到原任务的流式输出？

参考答案：事件使用单调 事件编号 持久化，客户端记录 最后收到的事件编号，重连后先补发缺口再切到实时流。客户端按 id 去重，服务端保留心跳和可控的事件窗口。首个 Token 延迟（TTFT）要拆成排队、上下文准备、模型首 Token 和网络传输，分别统计 P50/P95。

17. 追问：SSE 重连怎样做到不漏字、不重复，协议具体怎么设计？

参考答案：事件使用单调 事件编号 持久化，客户端记录 最后收到的事件编号，重连后先补发缺口再切到实时流。客户端按 id 去重，服务端保留心跳和可控的事件窗口。首个 Token 延迟（TTFT）要拆成排队、上下文准备、模型首 Token 和网络传输，分别统计 P50/P95。

18. Workflow-as-Skill 中是否包含外部 I/O？

参考答案：可以包含，但外部 I/O 必须通过统一工具层执行，不能在流程代码中绕过鉴权和审计。调用要设置超时、幂等、重试和结果查询，并记录外部副作用。

19. 追问：这样一个 Workflow 与普通 API 服务有什么区别？

参考答案：API 服务通常完成一次明确调用并返回结果。Workflow
保存多步骤状态、依赖、分支、重试、等待和恢复，可以跨越较长时间并产生多个中间结果。Workflow 可调用
API，但不等同于单个 API。

20. Human-in-the-loop 在哪里中断，如何恢复？

参考答案：在高风险、低置信度或需要业务决策的节点生成等待点编号，持久化候选动作、上下文摘要、允许的响应类型和过期策略。用户输入必须携带
任务编号和等待点编号，服务端校验身份与当前版本后写入事件并恢复。修改方向应使受影响步骤失效后重规划。

21. Redis 中的状态过期以后，任务还能继续吗？

参考答案：如果 Redis
是唯一状态源，过期后任务无法可靠恢复，因此不能这样设计。任务关键状态应保存在持久化数据库或对象存储，Redis
只做缓存、锁或短期事件窗口。缓存丢失后可从持久化状态重建。

22. 上下文工程具体做了哪些事情？

参考答案：把预算分给系统/策略、当前任务、近期对话、检索证据和工具结果，并预留输出空间。达到 70%-80% 阈值时先删重复与低价值日志，再对旧历史和长工具结果做结构化摘要。必须保留目标、约束、关键事实、未完成项、来源引用和工具副作用记录，原文留在外部存储以便按需回取。

23. 项目是否包含 Memory，什么应该记、什么不应该记？

参考答案：工作记忆只服务当前步骤，短期记忆覆盖会话，长期记忆保存跨会话且稳定的事实/偏好。写入由策略或抽取器产生候选，再经敏感性、价值、置信度和用户授权过滤。检索结合租户过滤、时间衰减与语义相关度，具体存储可按规模选择关系库、全文或向量索引。

24. “支持同步、异步”具体指什么？

参考答案：同步指请求保持连接并等待任务完成，适合短任务。异步指请求先返回任务编号，后台继续执行，客户端通过查询、回调或事件流获取进度和结果。两种方式应共用同一套任务状态机，不需要维护两套 ReAct 循环。

25. 追问：是否存在两套不同的 ReAct Loop？

参考答案：通常不需要两套。同步和异步只是等待结果与输出方式不同，核心 ReAct
状态机应保持一致。短任务在连接内持续输出，长任务把状态持久化后由后台 Worker
继续，完成后再通过查询或事件返回。

26. 现在是否还独立写代码，能否现场完成一道简单 Coding？本轮反问后还有现场编码。题目通过屏幕展示，现有信息不足以还原完整题面、输入输出和最终代码，因此不补写具体算法题。

参考答案：如实说明最近独立编码的频率、负责的模块和使用的语言，并明确可以完成现场编码。题面信息不完整时只说明解题流程、边界和复杂度，不补写无法确认的代码。

三面

1. 介绍一下当前工作，重点讲技术设计、遇到的问题和解决方式。

参考答案：按“问题—目标—方案—个人职责—结果”组织。先说明项目解决什么问题和为什么值得做，再讲核心架构与关键难点，最后给出上线范围、效果、稳定性或交付周期。选择能体现岗位能力且个人贡献清楚的项目。

2. Harness 怎么设计，运行过程是什么，如何提升效果？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

3. 通用运行内核、业务扩展和服务入口怎样分工？

参考答案：Framework 提供图、Agent 或消息等编程抽象。Harness 是把模型变成可用产品的整套运行环境，包括上下文、工具/Skill、权限、状态、恢复、观测、评测和交付。最小内核只负责生命周期与扩展协议，业务策略通过端口注入，服务入口负责鉴权、会话、流式协议和限流。

4. 实际交付给客户的是中间件，还是云上的产品？

参考答案：必须按真实交付形态回答，并说明客户得到的是云服务、私有部署软件、运行时组件还是配置与扩展包。还要说明部署位置、运维责任、升级方式、数据边界和服务指标，不能只用“平台”概括。

5. ReAct、Skill 和 Browser Use 等能力运行在哪里？

参考答案：ReAct 循环运行在通用运行内核中，Skill 的说明由运行时加载，Skill 代码和 Browser Use 在受控工具执行层或沙箱中运行。业务服务通过接口提供数据和动作。不同能力必须共享身份、权限、状态和审计信息。

6. 为什么多个 Agent 共享 Worker？

参考答案：部署时通常把入口服务、任务队列、Worker 池和状态存储分开。入口负责接收请求，队列保存待执行任务，Worker 负责实际运行，数据库或对象存储保存配置、进度和结果。公有云部署更便于弹性扩缩。私有化部署则要确保模型、数据、日志和密钥都留在客户规定的边界内。

7. 不同用户和业务数据怎样隔离？

参考答案：用户身份和租户信息作为不可伪造的执行上下文贯穿网关、运行时和工具层。执行层按 RBAC/ABAC 对“主体-资源-动作”再次鉴权。存储按租户编号分区并加行级策略，缓存/队列/工作目录带租户命名空间，密钥用短期凭证。可见性过滤不能替代真正鉴权。

8. 是否提供代码和 Bash 能力，代码在哪里执行？

参考答案：执行放到短生命周期、无特权的隔离环境：只读基础镜像、独立用户和命名空间、CPU/内存/进程数/时长配额、临时工作目录和默认断网。文件与网络按白名单开放。调度器在沙箱外做鉴权、审计和 Kill，死循环由超时与资源配额终止。

9. 为什么使用 LangGraph，而不是直接实现一个 Loop？

参考答案：简单 Loop 容易实现，但条件分支、中断恢复、人工等待、流式事件和状态持久化会很快变复杂。LangGraph 提供显式状态图和节点边界，便于恢复与观测。若任务简单且不需要这些能力，自研 Loop 反而更轻。

10. 运行图包含哪些节点？

参考答案：节点统一包含类型、版本、输入输出格式、配置、权限、超时重试、状态和执行器。常见类型包括模型、工具、条件、并行、子流程、人工等待、代码、检索和结果校验。节点定义与运行状态必须分离，多个任务才能安全复用同一张图。

11. 追问：LangGraph 能否替换，替换会影响哪些部分？

参考答案：应把 LangGraph 限定在状态图和节点调度层。任务存储、租约、权限、工具执行、事件协议、评测和业务扩展通过自有接口实现。这样替换框架时主要重写图适配层和检查点转换，不会影响对外接口与业务能力。

12. Skill 中的代码怎样运行？

参考答案：Skill 包含声明、说明、资源与可执行入口。运行时只把必要说明渐进式注入上下文。Workflow Skill
以受版本管理的确定性流程执行：ReAct
挂起并传入结构化参数，子流程返回结果/错误/产物后主循环恢复。脚本和代码必须经过签名、权限与沙箱策略。

13. Workflow 形式的 Skill 如何定义和执行？

参考答案：Skill 包含声明、说明、资源与可执行入口。运行时只把必要说明渐进式注入上下文。Workflow Skill
以受版本管理的确定性流程执行：ReAct
挂起并传入结构化参数，子流程返回结果/错误/产物后主循环恢复。脚本和代码必须经过签名、权限与沙箱策略。

14. LangGraph Checkpoint 与自行建设的持久化恢复机制，有什么区别？

参考答案：LangGraph Checkpoint 主要保存图状态和节点进度。自有恢复机制还要管理任务租约、外部调用、等待事件、用户取消、业务版本、事件流和审计。可以复用图检查点，但任务级可靠性不能只依赖它。

15. 如何评估性能和稳定性？

参考答案：性能看吞吐、并发、首响应、总延迟、队列等待、Token 和资源成本，并报告 P50、P95 和 P99。稳定性看任务成功率、错误率、重试率、恢复成功率、积压和长时间运行结果。测试要固定模型、负载和部署配置，并加入超时、重启和下游故障。

16. 如果一次工具调用失败，但最后任务成功了，成功率应该怎么算？

参考答案：分开统计步骤健康度与任务结果：工具调用成功率 反映基础设施，任务完成率 反映最终完成，答案质量通过率 反映答案/产物。一次工具失败后经替代路径成功，任务可算成功，但该工具调用仍计失败并记录恢复次数和额外成本。

17. Agent 的 Ground Truth 和测试用例如何构建？

参考答案：把评测拆成任务完成、答案质量、轨迹合规、延迟/成本和安全。参考标准（Ground Truth）由真实任务抽样、专家标注和生产失败样本构成，保存期望结果及允许的多种轨迹。LLM Judge
用明确的评分标准、盲测和多 Judge/人工抽检校准，不能只让同类模型自评。发布前回归，线上灰度 A/B 并设回滚阈值。

18. 追问：怎样判断任务真正完成？

参考答案：不要把强约束只写进 Prompt。把必经步骤编译成固定流程和规则检查，声明 必调工具和必交付产物
和前后置条件。结束前由独立 独立校验器 检查证据。未通过应标记为
未完成或失败，进入补偿、重试或人工确认，不能计作成功。

19. 耗时、性能和 Token 消耗目前是什么水平？

参考答案：只报告真实测量数据，并区分任务类型、模型、环境和统计周期。耗时至少给首个可见响应、总时长和 P95。性能给并发、吞吐和错误率。Token 给输入、输出和单任务成本。没有测量的数据不要估算成线上结果。

20. 追问：如何判断首个可见响应时间是否过长，统计的究竟是哪段时间？

参考答案：事件使用单调 事件编号 持久化，客户端记录 最后收到的事件编号，重连后先补发缺口再切到实时流。客户端按 id 去重，服务端保留心跳和可控的事件窗口。首个 Token 延迟（TTFT）要拆成排队、上下文准备、模型首 Token 和网络传输，分别统计 P50/P95。

21. 实际客户、终端用户和并发规模如何？

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

22. BM25、Embedding、RRF、Reranker 与 MMR 怎么组合？

参考答案：BM25 擅长精确词，向量召回语义近邻。两路各取候选后用 RRF 融合，再由 Reranker 精排。MMR 在结果高度重复、需要覆盖多方面时使用，会牺牲部分相关性。代码库常优先 grep/glob/符号索引，长文档可用层级索引。选择应由评测、时延和成本共同决定。

23. 如果要继续提高召回率，最难的部分是什么？

参考答案：最难的是建立可靠的参考数据并处理需要跨段、跨文档或隐含关系的问题。继续调参数的收益通常有限，更关键的是文档结构、查询理解、权限过滤和多跳召回。每项改动都要用固定数据集确认是否真正提高有效证据命中。

24. RAG 评测是否亲自做过，用过哪些指标？

参考答案：检索层用 Recall@K 看相关证据是否被召回，MRR 看首个相关结果位置，NDCG 适合多级相关性和整体排序。回答层再看忠实性、完整性、引用正确率与人工胜率。数据集要覆盖真实查询、难例和时间切片，并报告置信区间与失败样本。

25. Recall@K、MRR、NDCG 分别适合衡量什么？

参考答案：检索层用 Recall@K 看相关证据是否被召回，MRR 看首个相关结果位置，NDCG 适合多级相关性和整体排序。回答层再看忠实性、完整性、引用正确率与人工胜率。数据集要覆盖真实查询、难例和时间切片，并报告置信区间与失败样本。

26. 是否关注 Codex、Claude Code、WorkBuddy 等新方案？

参考答案：可从长任务持久化、统一工具协议、渐进式能力加载、工作空间与 Artifact、上下文压缩、计算机操作和多模型路由说明。回答重点是这些变化会影响现有架构的哪一层，以及是否经过实际验证。

27. 是否阅读源码，最近看到了哪些设计变化？

参考答案：如实说明阅读过的项目、跟踪的调用链和完成的验证。重点应覆盖会话创建、上下文组装、工具执行、退出条件、状态保存、恢复和流式事件，并说明自己确认了哪些设计限制。

28. 你提到的新 Harness 有什么优势？这种优势是绝对的吗？

参考答案：优势可能来自更稳定的任务恢复、更清晰的权限、更适合业务的上下文与 Skill、以及更完整的评测和观测。优势不是绝对的。在短任务、通用工具和低治理要求下，成熟通用 Harness 可能更简单。结论必须用同任务对照验证。

29. 为什么现在考虑换工作？

参考答案：用正向原因回答：当前工作已经形成了哪些积累，下一阶段希望扩大哪类责任或技术深度，以及目标岗位为什么更匹配。不要评价原公司的人和管理，也不要只把薪资作为原因。

十、深信服

一面

1. 请做一下自我介绍。

参考答案：回答顺序：先说明【X 年】后端或 AI 应用研发经验，再说明近【X 年】负责的 Agent 工作、核心职责和量化结果，最后说明这些能力与岗位要求的关系。全程控制在两分钟内，只保留与岗位相关的信息。

2. 最近的工作主要都与 Agent 有关吗？

参考答案：按时间顺序说明当前岗位、主要工作和转向 Agent 的阶段，再说明最熟悉的技术方向。重点写清实际负责的模块、投入时间和已经交付的结果，不要把学习、验证和生产经验混在一起。

3. 具体涉及平台研发、框架开发还是业务应用落地？

参考答案：按实际投入比例说明三类工作的范围。平台研发关注发布与治理，框架开发关注运行机制和扩展接口，业务落地关注领域数据、流程与效果。说明主要职责和次要参与内容，避免把所有方向都说成核心经验。

4. 选一个项目，讲讲怎样快速落地 Agent 应用。

参考答案：按"问题—目标—方案—个人职责—结果"组织。先说明项目解决什么问题和为什么值得做，再讲核心架构与关键难点，最后给出上线范围、效果、稳定性或交付周期。选择能体现岗位能力且个人贡献清楚的项目。

5. 刚才业务场景讲得比较多，能否进一步介绍 Agent 架构设计？

参考答案：从入口、运行时、模型与工具、状态存储、业务扩展、权限、观测和评测几个层次说明。再按一次请求讲清状态怎样流转、工具怎样执行、任务怎样结束和失败怎样恢复。架构说明应与实际部署保持一致。

6. 框架更偏固定 Workflow，还是自主规划 Agent？

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

7. 在 Agent 的具体设计中，你承担了哪些工作？

参考答案：明确说明自己负责的模块、做出的关键决策、亲自完成的代码或方案，以及与其他成员的边界。再说明交付结果和量化指标。团队已有能力、开源框架和个人新增能力要分开描述。

8. 参考通用 Coding Agent 的实现思路，当前业务 Agent 还有哪些可以改进的地方？

参考答案：从上下文管理、工具选择、任务持久化、工作空间、产物编辑、错误恢复、权限和评测中选择真实缺口。每项改进都要说明当前问题、预期收益、实现成本和验证方式，不能直接照搬通用 Coding Agent 的全部能力。

9. 如何将具体业务场景和固定 Workflow 加入 ReAct 执行过程？

参考答案：规则稳定、审计严格、失败代价高的步骤用 Workflow。路径未知、需要探索和动态选工具的部分用 ReAct。常见做法是外层确定性状态机约束阶段与门禁，局部节点交给 Agent。两者通过结构化输入输出、挂起/恢复和完成校验衔接。复杂不等于必须多 Agent。

10. 追问：有哪些具体参考例子？

参考答案：不要只报产品名称。应指出参考的是哪类设计、解决什么问题、与当前系统的差异，以及哪些部分已经验证、哪些部分因业务约束没有采用。内容必须来自真实阅读或实验。

11. 是否有后端或平台研发经验？如有，涉及哪些方面？

参考答案：按真实经历说明服务拆分、接口设计、数据库、缓存、队列、并发、监控和部署中实际负责的部分。只描述亲自做过的模块，并说明规模、故障处理和结果。

12. 是否有 NLP 相关研究经历？

参考答案：如实区分论文研究、工程应用和自学验证。说明涉及的任务、数据、模型、评测方法和最终产出。如果没有正式研究经历，应直接说明，并补充已有的文本处理和大模型应用经验。

13. 工作中是否参与过大模型训练？

参考答案：如实区分应用调用、推理部署和训练经验。多数企业 Agent 先用 Prompt、工具、RAG 与评测解决问题。只有错误模式稳定、数据量足且收益可验证时才考虑 SFT/DPO，并建立数据授权、离线评测和灰度回滚。

14. 当前 Agent 项目涉及模型训练，还是主要调用模型 API 或本地部署模型？

参考答案：部署时通常把入口服务、任务队列、Worker 池和状态存储分开。入口负责接收请求，队列保存待执行任务，Worker 负责实际运行，数据库或对象存储保存配置、进度和结果。公有云部署更便于弹性扩缩。私有化部署则要确保模型、数据、日志和密钥都留在客户规定的边界内。

十一、同花顺

一面

1. 近期的主力开发语言是什么？如使用过 Python 或 Java，分别用于哪些场景？

参考答案：分别说明两种语言的使用年限、最近使用时间、承担过的模块和能够独立完成的工作。Python 可从异步服务、Agent 和数据处理说明，Java
可从微服务、并发、数据库与中间件说明。只写真实熟练度，不用把两种语言说成同一水平。

2. 公司内部的技术栈是否受到合规等限制？

参考答案：按真实情况说明语言、依赖、模型、网络访问、数据存储和开源许可证方面的限制。重点说明这些限制如何影响技术选型，以及团队怎样通过审批、私有部署、依赖扫描和数据分级满足要求。

3. 团队使用哪些 AI Coding 工具和模型？

参考答案：AI 负责样板、测试草案和局部重构，人对架构、接口、安全和最终合并负责。多人协作按模块/接口拆分，小批次提交，强制代码审查、静态检查、测试和依赖扫描。统计比例不如说明哪些结果由人验证、出现错误怎样发现更有意义。

4. AI 应用平台是什么产品，解决什么问题？

参考答案：建议答法：平台面向【内部开发者/企业客户】，把模型、知识、工具和流程统一成可发布、可治理的 AI 应用。核心解决重复接入、权限审计、长任务可靠性和效果回归。交付物应明确为产品实例、配置或扩展包、接口与运维指标，不能只说“交付一个 Agent”。

5. 已有很多低代码 AI 平台，为什么仍然选择自研？

参考答案：先用现成方案验证需求。只有在强权限/私有化、长任务恢复、领域流程、现有数据动作接入或规模成本上存在持续缺口时才自研。差异化必须用同模型、同工具、同任务对照，量化完成率、合规率、接入周期和单位成本。通用能力产品化，客户差异放配置/插件，避免项目制失控。

6. 平台是否上线，用户量、调用量和并发量如何？

参考答案：只报真实、可解释的数据，并区分注册用户/活跃用户、日任务量、峰值 QPS、同时运行任务数、Worker 数和单任务资源。注明统计周期和环境。概念验证（PoC）、灰度、生产要分开，容量测试结果也不能冒充线上规模。

7. LangGraph、FastAPI 和 Python Worker 为什么能够支撑商业化平台？

参考答案：部署时通常把入口服务、任务队列、Worker
池和状态存储分开。入口负责接收请求，队列保存待执行任务，Worker 负责实际运行，数据库或对象存储保存配置、进度和结果。公有云部署更便于弹性扩缩。私有化部署则要确保模型、数据、日志和密钥都留在客户规定的边界内。

8. 如果 Workflow 没有 LLM 节点，而是大量代码节点，应该怎样执行？

参考答案：应按普通确定性任务执行，不必进入 ReAct。代码节点由 Workflow 引擎按依赖调度，在沙箱或受控 Worker 中运行，设置超时、资源配额、重试和检查点。使用 LLM 与否不影响流程的状态和观测。

9. 追问：代码中出现死循环怎么办？

参考答案：执行放到短生命周期、无特权的隔离环境：只读基础镜像、独立用户和命名空间、CPU/内存/进程数/时长配额、临时工作目录和默认断网。文件与网络按白名单开放。调度器在沙箱外做鉴权、审计和 Kill，死循环由超时与资源配额终止。

10. ReAct 的核心执行逻辑是什么？

参考答案：核心循环是 Thought/Decision→Action→工具结果（Observation）→更新状态，直到 Final 或触发预算/取消/错误边界。生产实现还必须加入工具鉴权、参数校验、超时、幂等、Checkpoint、上下文预算和可观测事件。循环迟迟不结束时用最大轮次、重复调用检测和无进展判定收敛。

11. 以查询天气、控制电视和空调为例，从用户输入到 Function Call、Observation 和 Final Answer，完整链路怎样运行？

参考答案：注册表保存稳定 tool_id、版本、JSON 参数格式、权限、超时/重试、幂等性和执行适配器。模型只看到经权限与场景过滤后的短描述。解析后在服务端做参数格式校验、参数规范化、鉴权和风险审批，再执行并返回结构化工具结果（Observation）。用调用正确率、参数合法率和任务完成率评估。

12. 工具数量增加后，怎样避免选错工具或参数错误？

参考答案：注册表保存稳定 tool_id、版本、JSON 参数格式、权限、超时/重试、幂等性和执行适配器。模型只看到经权限与场景过滤后的短描述。解析后在服务端做参数格式校验、参数规范化、鉴权和风险审批，再执行并返回结构化工具结果（Observation）。用调用正确率、参数合法率和任务完成率评估。

13. 渐进式披露具体怎样实现？

参考答案：先根据租户、身份、场景和当前阶段筛掉无权或无关能力，只向模型展示少量能力摘要。模型选中后再加载完整说明、参数格式和资源。执行前仍由服务端做鉴权，能力可见和允许执行必须分开。

14. 几十个 Skill 如何被发现和激活？

参考答案：为 Skill 建结构化清单：名称、意图、前置条件、输入输出、权限、成本和版本。先按租户/权限/场景做确定性过滤，再用关键词+Embedding 召回少量候选，必要时让路由模型重排。离线用命中率/误选率，线上用完成率、纠正率和成本评估，并支持显式选择兜底。

15. Skill 同时包含 Markdown、脚本、代码和工具时，运行时怎样执行？

参考答案：Markdown 作为说明按需进入上下文。脚本和代码由受控执行器在沙箱运行。工具通过统一注册表调用。资源文件按权限加载。运行时根据清单确定加载顺序，并统一记录版本、权限和结果。

16. Workflow-as-Skill 如何与 ReAct 主循环衔接？

参考答案：Skill 包含声明、说明、资源与可执行入口。运行时只把必要说明渐进式注入上下文。Workflow Skill 以受版本管理的确定性流程执行：ReAct
挂起并传入结构化参数，子流程返回结果/错误/产物后主循环恢复。脚本和代码必须经过签名、权限与沙箱策略。

17. 为什么有了 MCP 之后，还需要 Skill?

参考答案：MCP 提供统一的能力发现、参数格式、调用与传输约定，降低不同 Host 与工具服务的适配成本。Skill 则描述何时用、怎样组合、业务规则和产物要求，两者互补。MCP 的治理价值并非独有，鉴权、审计、限流仍需平台实现。简单稳定 API 可直接接 OpenAPI，跨客户端复用时 MCP 更有价值。

18. 反过来，有了 Skill，MCP 能不能完全不用?

参考答案：MCP 提供统一的能力发现、参数格式、调用与传输约定，降低不同 Host 与工具服务的适配成本。Skill 则描述何时用、怎样组合、业务规则和产物要求，两者互补。MCP 的治理价值并非独有，鉴权、审计、限流仍需平台实现。简单稳定 API 可直接接 OpenAPI，跨客户端复用时 MCP 更有价值。

19. Skill 直接调用 OpenAPI、CLI 或临时生成代码，为什么不能替代 MCP?

参考答案：MCP 提供统一的能力发现、参数格式、调用与传输约定，降低不同 Host 与工具服务的适配成本。Skill 则描述何时用、怎样组合、业务规则和产物要求，两者互补。MCP 的治理价值并非独有，鉴权、审计、限流仍需平台实现。简单稳定 API 可直接接 OpenAPI，跨客户端复用时 MCP 更有价值。

20. 追问：如果 API Key 已在环境变量中，模型只需要执行简化命令，MCP 还提供什么额外价值?

参考答案：MCP 提供统一的能力发现、参数格式、调用与传输约定，降低不同 Host 与工具服务的适配成本。Skill 则描述何时用、怎样组合、业务规则和产物要求，两者互补。MCP 的治理价值并非独有，鉴权、审计、限流仍需平台实现。简单稳定 API 可直接接 OpenAPI，跨客户端复用时 MCP 更有价值。

21. MCP 相比标准 OpenAPI 真正增加了什么?

参考答案：MCP 提供统一的能力发现、参数格式、调用与传输约定，降低不同 Host 与工具服务的适配成本。Skill 则描述何时用、怎样组合、业务规则和产物要求，两者互补。MCP 的治理价值并非独有，鉴权、审计、限流仍需平台实现。简单稳定 API 可直接接 OpenAPI，跨客户端复用时 MCP 更有价值。

22. 你提到的安全、治理和复用价值，是 MCP 独有的吗?

参考答案：MCP 提供统一的能力发现、参数格式、调用与传输约定，降低不同 Host 与工具服务的适配成本。Skill 则描述何时用、怎样组合、业务规则和产物要求，两者互补。MCP 的治理价值并非独有，鉴权、审计、限流仍需平台实现。简单稳定 API 可直接接 OpenAPI，跨客户端复用时 MCP 更有价值。

23. 面向千万用户的云端 Agent，整体怎样设计?

参考答案：入口层无状态并按用户和租户限流，任务进入队列后由共享 Worker 池执行，状态外置到数据库和对象存储。模型调用通过统一网关做配额、路由和成本控制，代码与文件任务进入隔离沙箱。系统按队列积压和任务类型弹性扩容，并对热点租户做隔离。

24. 是否需要给每个用户运行一个独立 Agent 实例?

参考答案：通常不需要。Agent 是配置、状态和任务记录，不等同于常驻进程。用户请求可由共享 Worker 池执行，状态按用户和任务隔离。只有持续占用本地资源、需要强隔离或长连接的场景才考虑独占实例。

25. 用户自定义 Skill、长期记忆、偏好和上传文件如何加载、隔离?

参考答案：用户身份和租户信息作为不可伪造的执行上下文贯穿网关、运行时和工具层。执行层按 RBAC/ABAC 对"主体-资源-动作"再次鉴权。存储按租户编号分区并加行级策略，缓存/队列/工作目录带租户命名空间，密钥用短期凭证。可见性过滤不能替代真正鉴权。

26. 公共资产与租户私有资产如何做到物理复用和逻辑隔离?

参考答案：用户身份和租户信息作为不可伪造的执行上下文贯穿网关、运行时和工具层。执行层按 RBAC/ABAC 对"主体-资源-动作"再次鉴权。存储按租户编号分区并加行级策略，缓存/队列/工作目录带租户命名空间，密钥用短期凭证。可见性过滤不能替代真正鉴权。

27. 用户要求执行 ps -ef 或终止某个 Java 进程时，如何保证安全?

参考答案：执行放到短生命周期、无特权的隔离环境：只读基础镜像、独立用户和命名空间、CPU/内存/进程数/时长配额、临时工作目录和默认断网。文件与网络按白名单开放。调度器在沙箱外做鉴权、审计和 Kill，死循环由超时与资源配额终止。

28. Agent 在沙箱内运行与在沙箱外调度，架构有什么不同?

参考答案：调度器、任务状态、权限决策和审计运行在沙箱外，保持稳定并控制生命周期。模型生成的代码、脚本、浏览器和高风险工具在沙箱内执行。沙箱只拿短期凭证和最小资源，结果经过校验后返回调度器。

29. 云端 Agent 的性能、资源成本、隔离和安全如何权衡?

参考答案：用户身份和租户信息作为不可伪造的执行上下文贯穿网关、运行时和工具层。执行层按 RBAC/ABAC 对"主体-资源-动作"再次鉴权。存储按租户编号分区并加行级策略，缓存/队列/工作目录带租户命名空间，密钥用短期凭证。可见性过滤不能替代真正鉴权。

30. 【Python 编码题】 编写一个协程示例。

参考答案：用 async def 定义协程，在 I/O 等待位置使用 await。多个互不依赖的任务可通过 asyncio.gather 或 TaskGroup 并发运行。协程中不能直接执行长时间阻塞操作，完整实现见附录 I。

31. 【Java 编码题】 给出两种单例实现方案。

参考答案：优先使用枚举单例，它天然具备线程安全能力，也能避免反序列化产生新实例。使用双重检查锁时，instance 必须加 volatile，锁应加在类对象上，避免其他线程读到尚未完成初始化的对象。代码见附录 J。

32. 【Java 编码题·追问】 双重检查锁为什么需要 volatile，锁应该加在哪里?

参考答案：优先使用枚举单例，它天然具备线程安全能力，也能避免反序列化产生新实例。使用双重检查锁时，instance 必须加 volatile，锁应加在类对象上，避免其他线程读到尚未完成初始化的对象。代码见附录 J。

33. 平时是否在本机使用 Codex 等 AI Coding 工具？

参考答案：AI 负责样板、测试草案和局部重构，人对架构、接口、安全和最终合并负责。多人协作按模块/接口拆分，小批次提交，强制代码审查、静态检查、测试和依赖扫描。统计比例不如说明哪些结果由人验证、出现错误怎样发现更有意义。

34. 项目代码由谁编写，AI Coding 占多大比例？

参考答案：AI 负责样板、测试草案和局部重构，人对架构、接口、安全和最终合并负责。多人协作按模块/接口拆分，小批次提交，强制代码审查、静态检查、测试和依赖扫描。统计比例不如说明哪些结果由人验证、出现错误怎样发现更有意义。

编码题参考实现

以下实现对应原文可确认题面的编码题。面试现场应先复述输入输出、边界和复杂度，再写代码并补最小测试。

附录 A：Two Sum（Python）

```
def two_sum(nums, target):
seen = {}
for i, x in enumerate(nums):
y = target - x
if y in seen:
return [seen[y], i]
seen[x] = i
return []
```

附录 B：原地合并有序数组（Python）

```
def merge(nums1, m, nums2, n):
i, j, k = m - 1, n - 1, m + n - 1
while j >= 0:
if i >= 0 and nums1[i] > nums2[j]:
nums1[k] = nums1[i]
i -= 1
else:
nums1[k] = nums2[j]
j -= 1
k -= 1
```

附录 C：简化 ReAct Loop（伪代码）

```
state = load_or_create(run_id)
while not state.done:
check_cancel_and_budget(state)
context = build_context(state, skills, memory)
decision = model.invoke(context, visible_tools(state.identity))
if decision.final:
validate_completion(state, decision.final)
state.done = True
else:
call = validate_schema_and_authorize(decision.tool_call)
result = execute_with_timeout_and_idempotency(call, state.run_id)
state.append_observation(result)
checkpoint(state)
return state.final
```

附录 D：Plan-and-Execute 核心结构（Python）

```
steps = [
{"id": "a", "type": "tool", "name": "weather", "args": {"city": "  "}},
{"id": "b", "type": "tool", "name": "weather", "args": {"city": "  "}},
{"id": "c", "type": "tool", "name": "temperature_diff",
"args": {"x": "$a.temp", "y": "$b.temp"}, "deps": ["a", "b"]},
]

results = {}
for step in topological_sort(steps):
args = resolve_refs(step["args"], results)
results[step["id"]] = registry[step["name"]](**args)
checkpoint(step["id"], results[step["id"]])
answer = summarize(results)
```

附录 E：重排数组使相邻不同（Python）

```
import heapq

def rearrange(nums):
freq = {}
```

```
for x in nums:
freq[x] = freq.get(x, 0) + 1
if max(freq.values(), default=0) > (len(nums) + 1) // 2:
return None
heap = [(-c, x) for x, c in freq.items()]
heapq.heapify(heap)
ans, prev = [], (0, None)
while heap:
c, x = heapq.heappop(heap)
ans.append(x)
if prev[0] < 0:
heapq.heappush(heap, prev)
prev = (c + 1, x)
return ans
```

附录 F：二叉树层序遍历（Python）

```
from collections import deque

def level_order(root):
if not root:
return []
q, ans = deque([root]), []
while q:
level = []
for _ in range(len(q)):
node = q.popleft()
level.append(node.val)
if node.left: q.append(node.left)
if node.right: q.append(node.right)
ans.append(level)
return ans
```

附录 G：LRU Cache（Python）

```
from collections import OrderedDict

class LRUCache:
def __init__(self, capacity):
self.capacity = capacity
self.data = OrderedDict()
def get(self, key):
if key not in self.data:
return -1
self.data.move_to_end(key)
return self.data[key]
def put(self, key, value):
self.data[key] = value
self.data.move_to_end(key)
if len(self.data) > self.capacity:
self.data.popitem(last=False)
```

附录 H：线程池执行流程（伪代码）

```
submit(task):
if state != RUNNING: reject(task)
else if worker_count < core_size: start_worker(task)
else if queue.offer(task): signal_worker()
else if worker_count < max_size: start_worker(task)
else: rejection_policy(task)

worker_loop():
while state permits or queue not empty:
task = queue.take()
try: task.run()
catch error: record(error)
finally: update_metrics()
```

附录 I：Python 协程实现

```
import asyncio

async def fetch(name, delay):
await asyncio.sleep(delay)
return f"{name}: done"

async def main():
results = await asyncio.gather(fetch("a", 0.2), fetch("b", 0.1))
print(results)

if __name__ == "__main__":
asyncio.run(main())
```

附录 J：Java 单例

```
//
public enum Singleton {
INSTANCE;
}

//
public final class DclSingleton {
private static volatile DclSingleton instance;
private DclSingleton() {}
public static DclSingleton getInstance() {
if (instance == null) {
synchronized (DclSingleton.class) {
if (instance == null) instance = new DclSingleton();
}
}
return instance;
}
}
```

答案边界

原文中部分算法题明确注明题面不完整，因此这里只给典型版本，不额外虚构约束。个人经历、用户规模、绩效、薪酬、模型版本等问题必须按候选人的真实情况替换。